PROJECT - 2

Deploy Your Code on a Docker Container using Jenkins on AWS



Step 1 : Application Setup

Overview

The first step was to run and validate the application locally before moving to Docker. This ensures that the application works correctly in its original environment before introducing containerization.

Application Setup

The provided repository was cloned from GitHub. The project structure and package.json confirmed that the application is built with React and TypeScript, using Vite as the build tool and npm for package management.

The required dependencies were installed using npm install. The application was then started using npm run dev and was available at http://localhost:5173

Runtime Configuration

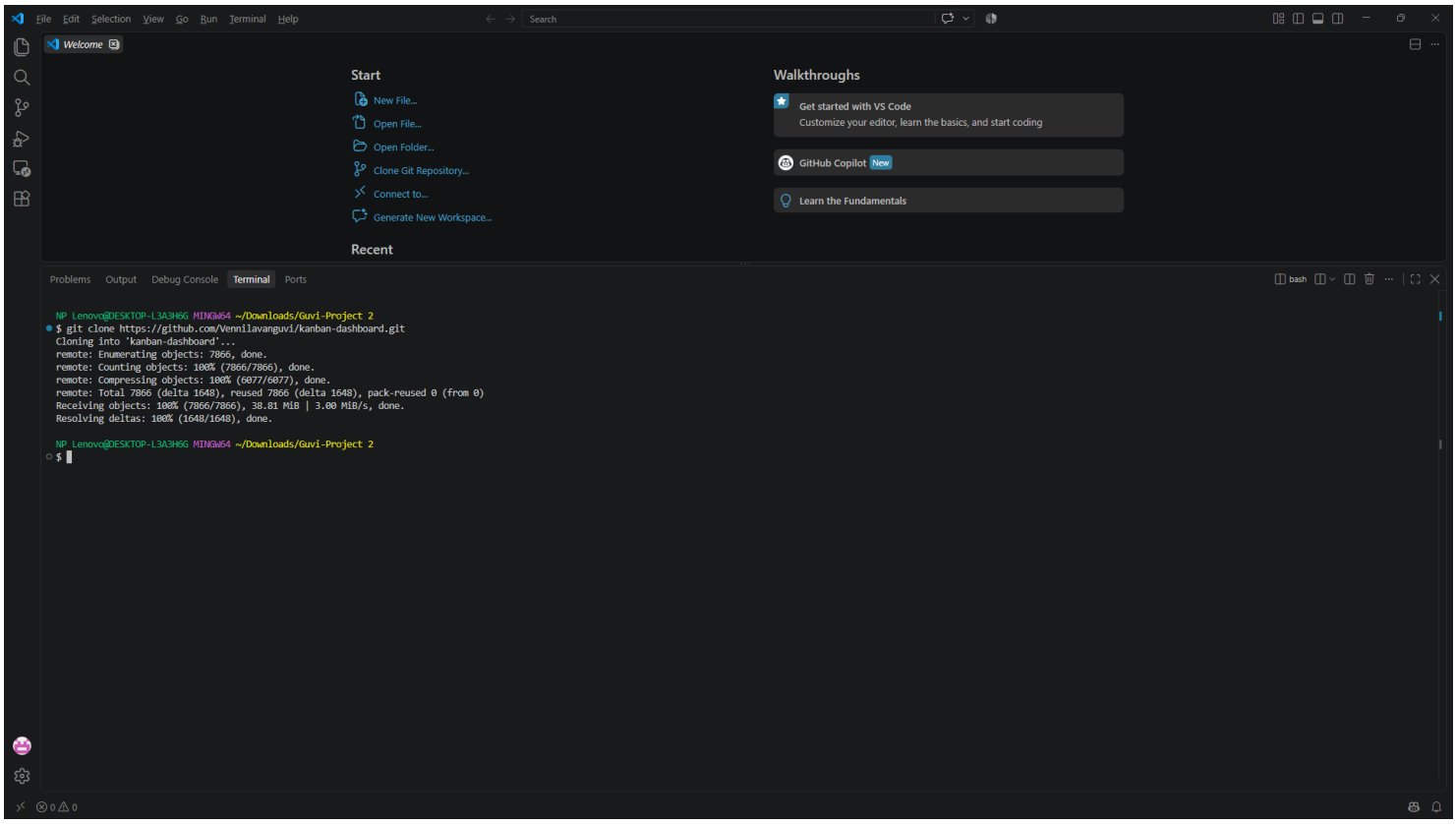
- Start Command: npm run dev
- Port: 5173
- Local URL: http://localhost:5173/
- Environment Variables: No `.env` file was found in the repository.

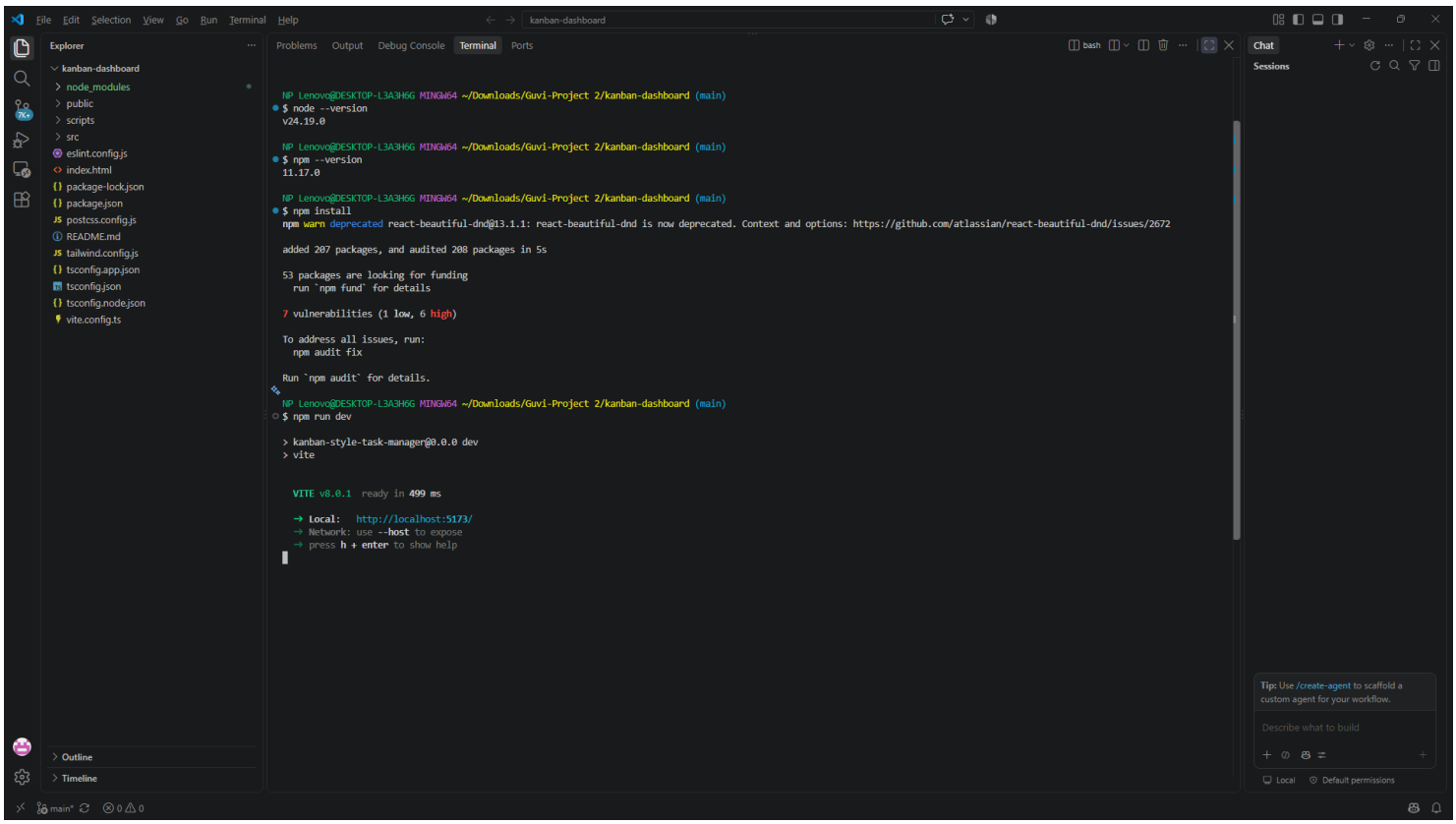
Validation

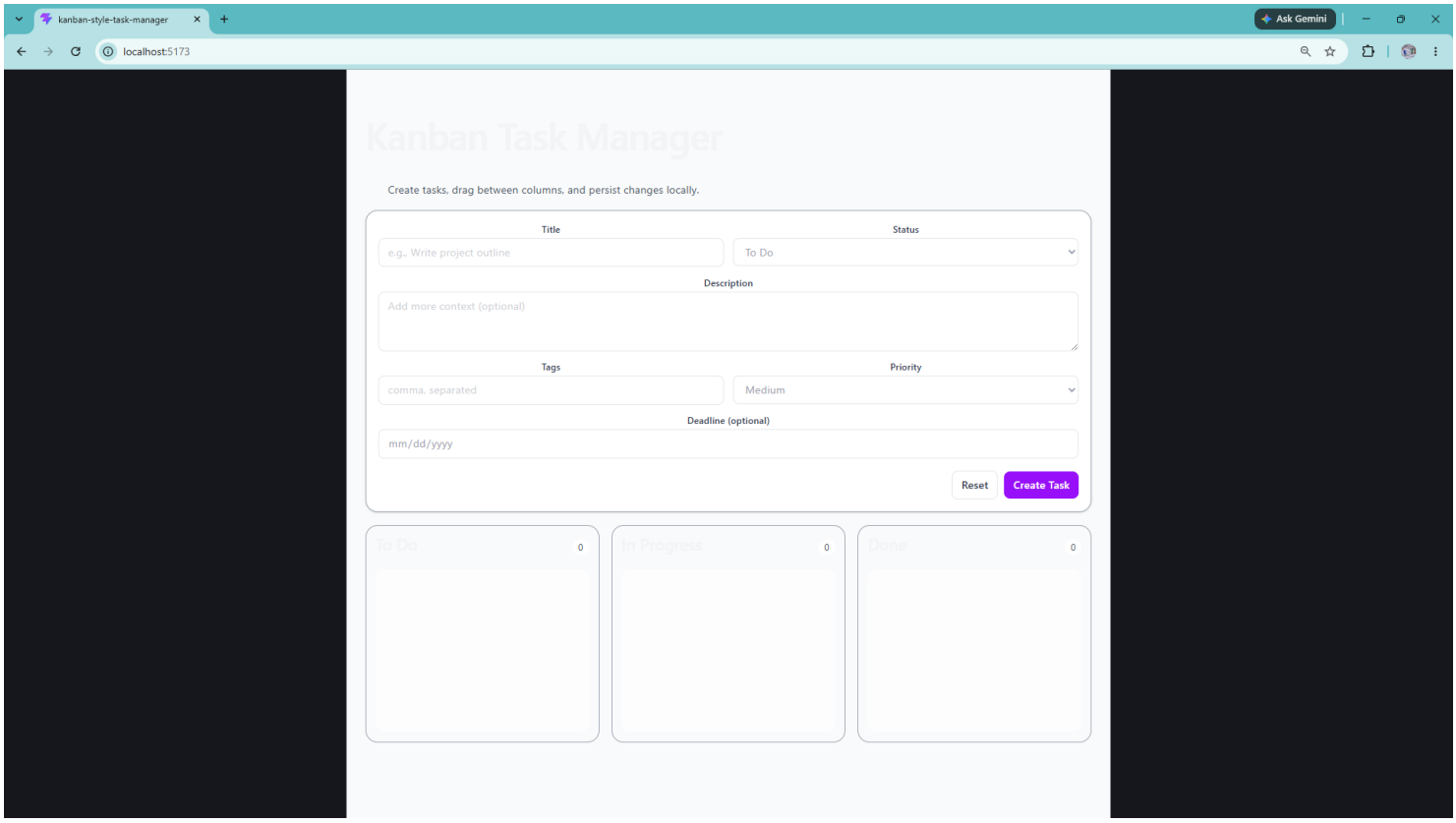
The Kanban Task Manager loaded successfully in the browser, with the To Do, In Progress, and Done columns displayed correctly.

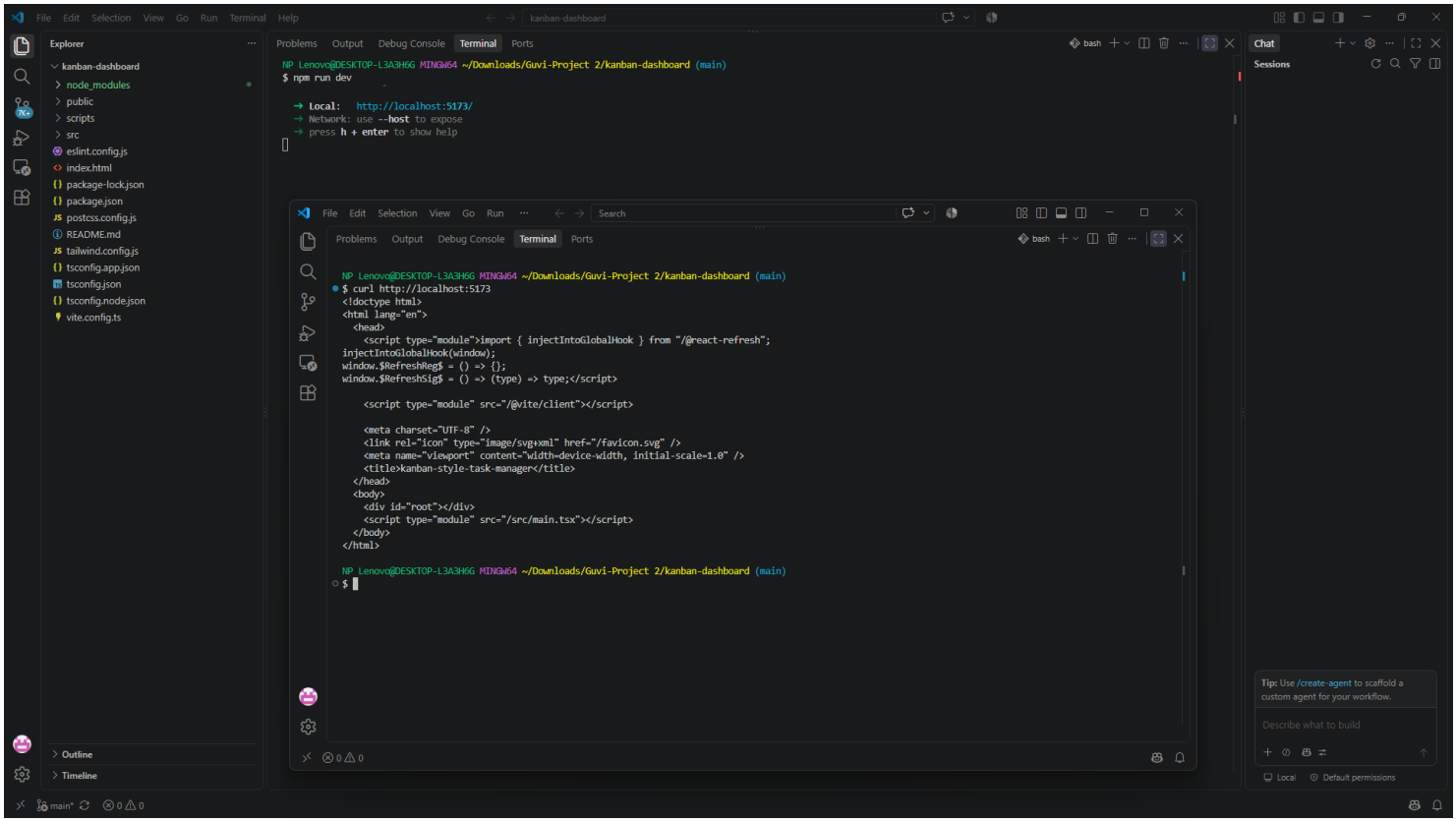
Result

Local application setup and validation were completed successfully. The application was successfully validated locally and is ready for containerization.

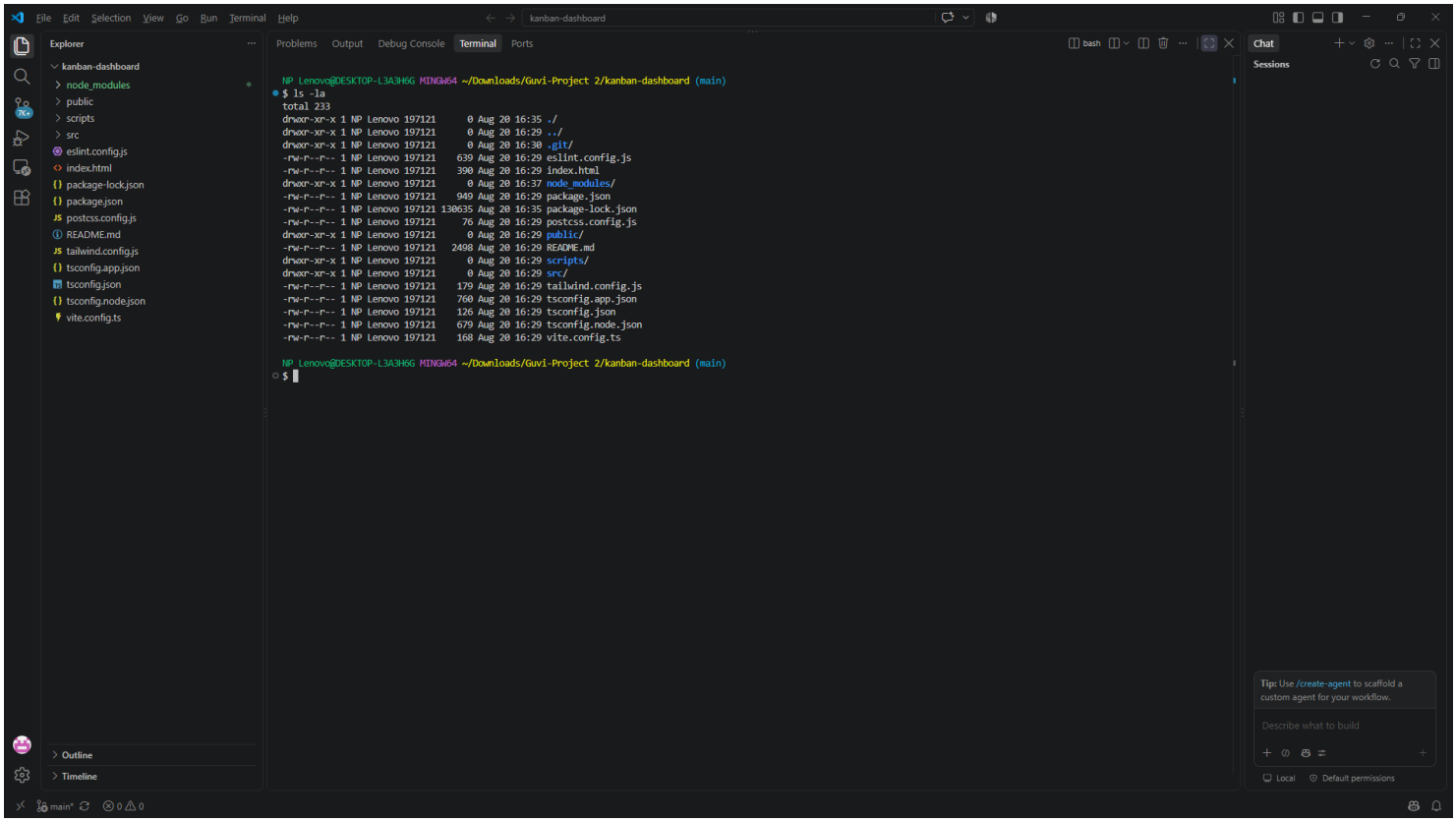

File Edit Selection View Go Run Terminal Help
Search
Welcome
Start
New File...
Open File...
Open Folder...
Clone Git Repository...
Connect to...
Generate New Workspace...
Walkthroughs
Get started with VS Code
Customize your editor, learn the basics, and start coding
GitHub Copilot New
Learn the Fundamentals
Recent
Problems Output Debug Console Terminal Ports
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2
$ git clone https://github.com/Vennilavanguvi/kanban-dashboard.git
Cloning into 'kanban-dashboard'...
remote: Enumerating objects: 7866, done.
remote: Counting objects: 100% (7866/7866), done.
remote: Compressing objects: 100% (6077/6077), done.
remote: Total 7866 (delta 1648), reused 7866 (delta 1648), pack-reused 0 (from 0)
Receiving objects: 100% (7866/7866), 38.81 MiB | 3.00 MiB/s, done.
Resolving deltas: 100% (1648/1648), done.
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2
$


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
node_modules
public
scripts
src
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Problems Output Debug Console Terminal Ports
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ node --version
v24.19.0
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ npm --version
11.17.0
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ npm install
npm warn deprecated react-beautiful-dnd@13.1.1: react-beautiful-dnd is now deprecated. Context and options: https://github.com/atlassian/react-beautiful-dnd/issues/2672
added 207 packages, and audited 208 packages in 5s
53 packages are looking for funding
run `npm fund` for details
7 vulnerabilities (1 low, 6 high)
To address all issues, run:
npm audit fix
Run `npm audit` for details.
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ npm run dev
> kanban-style-task-manager@0.0.0 dev
> vite
VITE v8.0.1 ready in 499 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Local
Default permissions
main*


kanban-style-task-manager
Ask Gemini
localhost:5173
Kanban Task Manager
Create tasks, drag between columns, and persist changes locally.
Title
e.g., Write project outline
Status
To Do
Description
Add more context (optional)
Tags
comma, separated
Priority
Medium
Deadline (optional)
mm/dd/yyyy
Reset
Create Task
To Do
0
In Progress
0
Done
0


Explorer
kanban-dashboard
node_modules
public
scripts
src
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Problems
Output
Debug Console
Terminal
Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ npm run dev
➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help
$ curl http://localhost:5173
<!doctype html>
<html lang="en">
<head>
<script type="module">import { injectIntoGlobalHook } from "/@react-refresh";
injectIntoGlobalHook(window);
window.$RefreshReg$ = () => {};
window.$RefreshSig$ = () => (type) => type;</script>
<script type="module" src="/@vite/client"></script>
<meta charset="UTF-8" />
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>kanban-style-task-manager</title>
</head>
<body>

<script type="module" src="/src/main.tsx"></script>
</body>
</html>
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
node_modules
public
scripts
src
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Problems Output Debug Console Terminal Ports
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ ls -la
total 233
drwxr-xr-x 1 NP Lenovo 197121 0 Aug 20 16:35 ./
drwxr-xr-x 1 NP Lenovo 197121 0 Aug 20 16:29 ../
drwxr-xr-x 1 NP Lenovo 197121 0 Aug 20 16:30 .git/
-rw-r--r-- 1 NP Lenovo 197121 639 Aug 20 16:29 eslint.config.js
-rw-r--r-- 1 NP Lenovo 197121 390 Aug 20 16:29 index.html
drwxr-xr-x 1 NP Lenovo 197121 0 Aug 20 16:37 node_modules/
-rw-r--r-- 1 NP Lenovo 197121 949 Aug 20 16:29 package.json
-rw-r--r-- 1 NP Lenovo 197121 130635 Aug 20 16:35 package-lock.json
-rw-r--r-- 1 NP Lenovo 197121 76 Aug 20 16:29 postcss.config.js
drwxr-xr-x 1 NP Lenovo 197121 0 Aug 20 16:29 public/
-rw-r--r-- 1 NP Lenovo 197121 2498 Aug 20 16:29 README.md
drwxr-xr-x 1 NP Lenovo 197121 0 Aug 20 16:29 scripts/
drwxr-xr-x 1 NP Lenovo 197121 0 Aug 20 16:29 src/
-rw-r--r-- 1 NP Lenovo 197121 179 Aug 20 16:29 tailwind.config.js
-rw-r--r-- 1 NP Lenovo 197121 760 Aug 20 16:29 tsconfig.app.json
-rw-r--r-- 1 NP Lenovo 197121 126 Aug 20 16:29 tsconfig.json
-rw-r--r-- 1 NP Lenovo 197121 679 Aug 20 16:29 tsconfig.node.json
-rw-r--r-- 1 NP Lenovo 197121 168 Aug 20 16:29 vite.config.ts
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Local
Default permissions
main*
0 0

Step 2 : Docker Containerization

Overview

With the application validated locally, the next step was to containerize it using Docker. A multi-stage Dockerfile was written to keep the final image lean — the build stage compiles the React/TypeScript app, and the runtime stage serves only the built artifacts using a lightweight Nginx base image.

Dockerfile Setup

The first stage uses node:24-alpine to install dependencies and run the Vite build. The second stage uses nginx:alpine and copies only the compiled dist into the Nginx web root. No source code, node modules, or dev dependencies are carried into the final image.

The EXPOSE instruction was added to declare port 80. A HEALTHCHECK instruction was included to allow Docker to automatically monitor container health by periodically pinging localhost.

Runtime Configuration

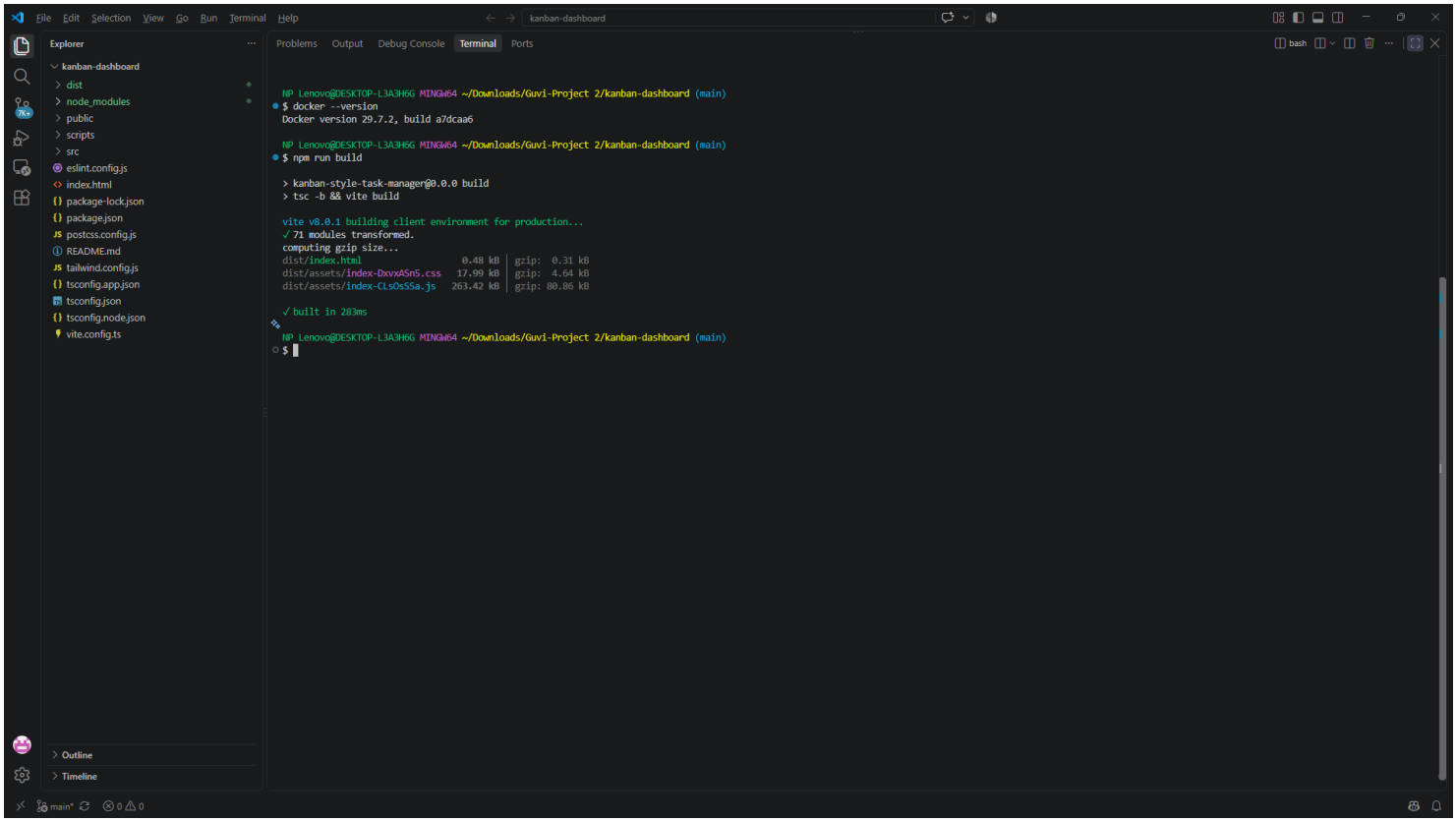
- Build Command: docker build -t kanban-dashboard:v1 .
- Run Command: docker run -d --name kanban-dashboard -p 8080:80 kanban-dashboard:v1
- Port: 8080 (mapped to container port 80)
- Local URL: http://localhost:8080
- Image Size: 93.8 MB (26.4 MB content size)

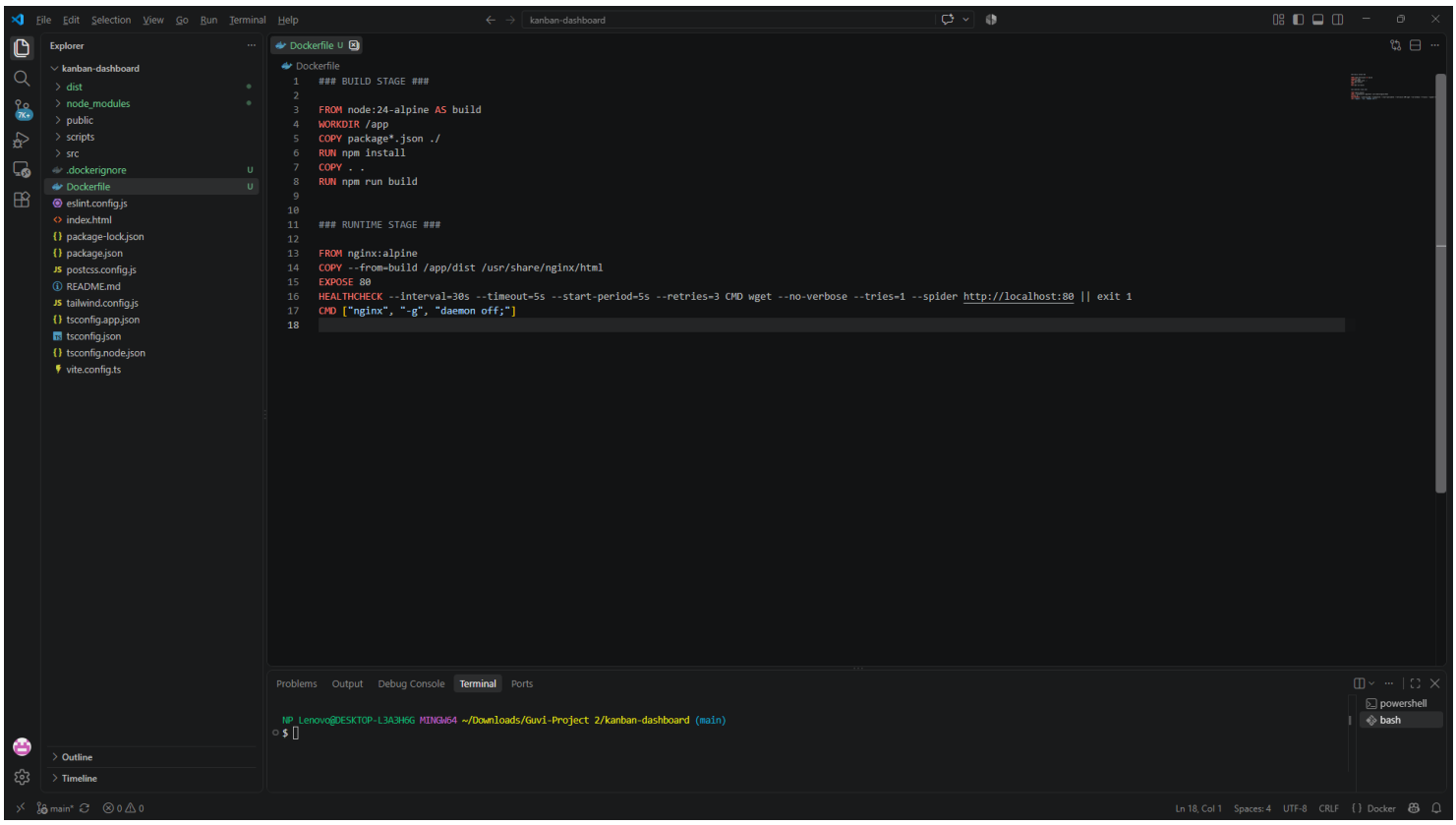
Validation

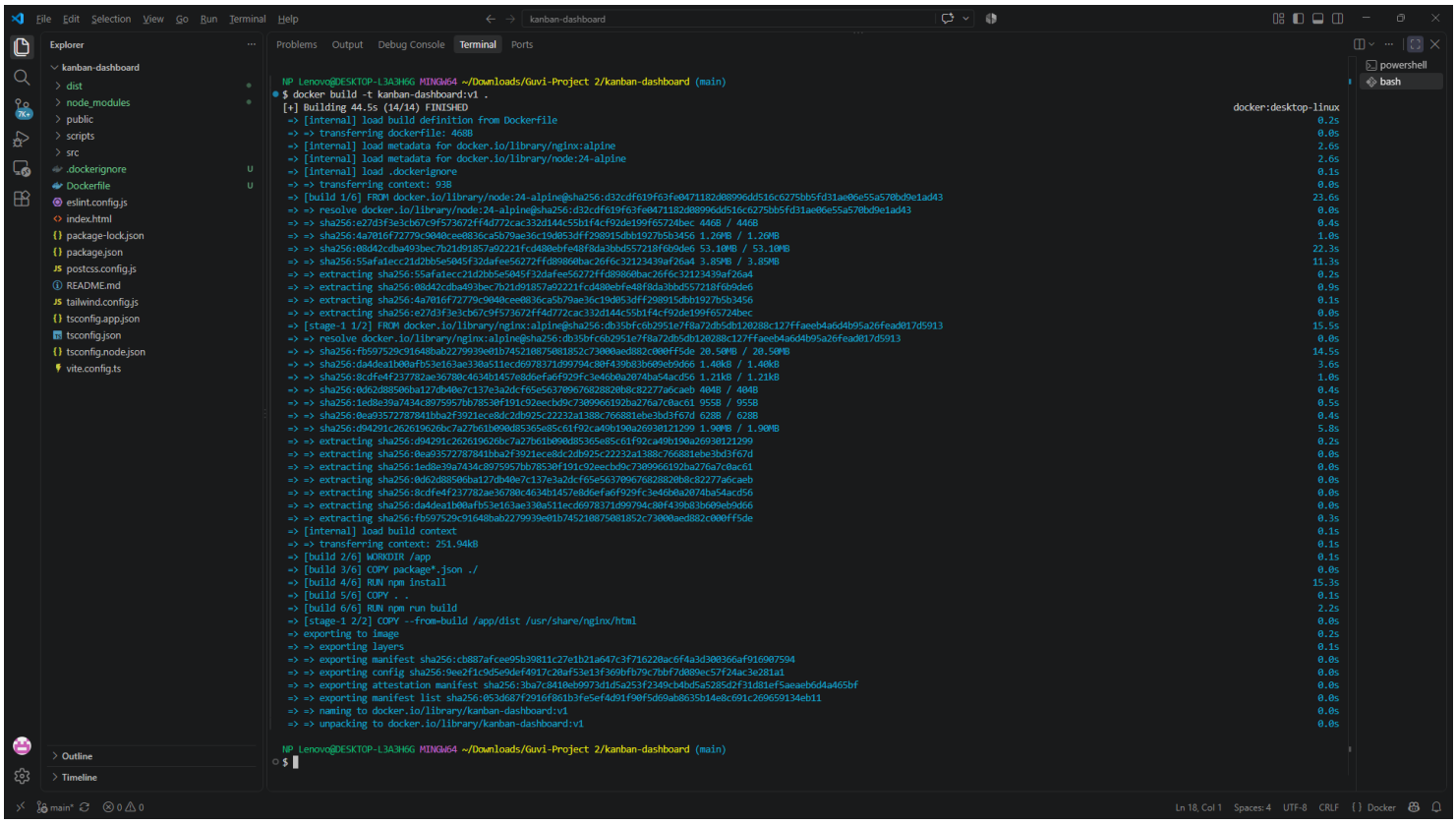
The docker build completed successfully across 14 steps in 44.5 seconds. Running docker ps confirmed the container status as healthy, with ports correctly mapped as 0.0.0.0:8080->80/tcp. The Kanban Task Manager loaded successfully in the browser at http://localhost:8080, with the To Do, In Progress, and Done columns displayed correctly.

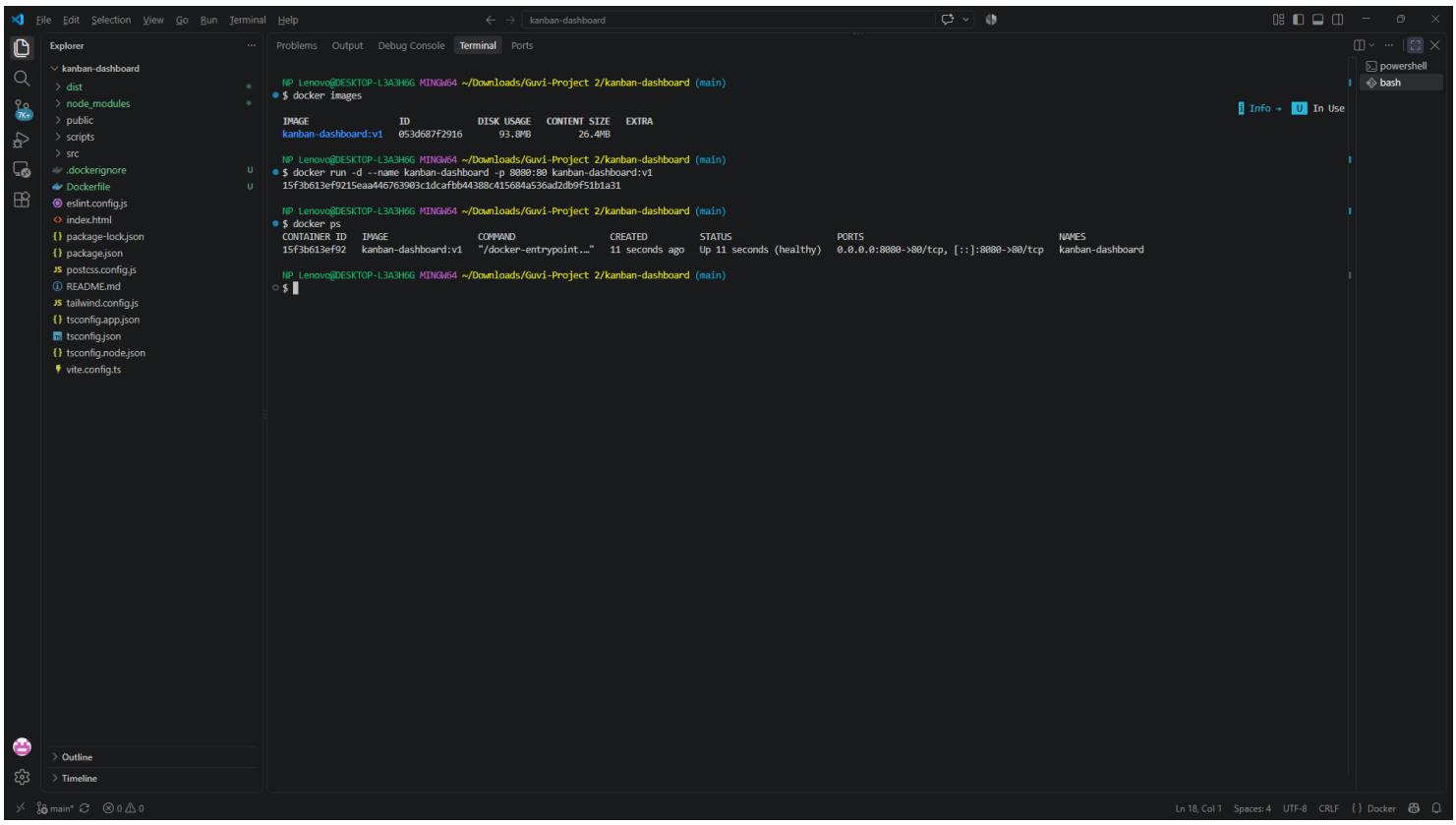
Result

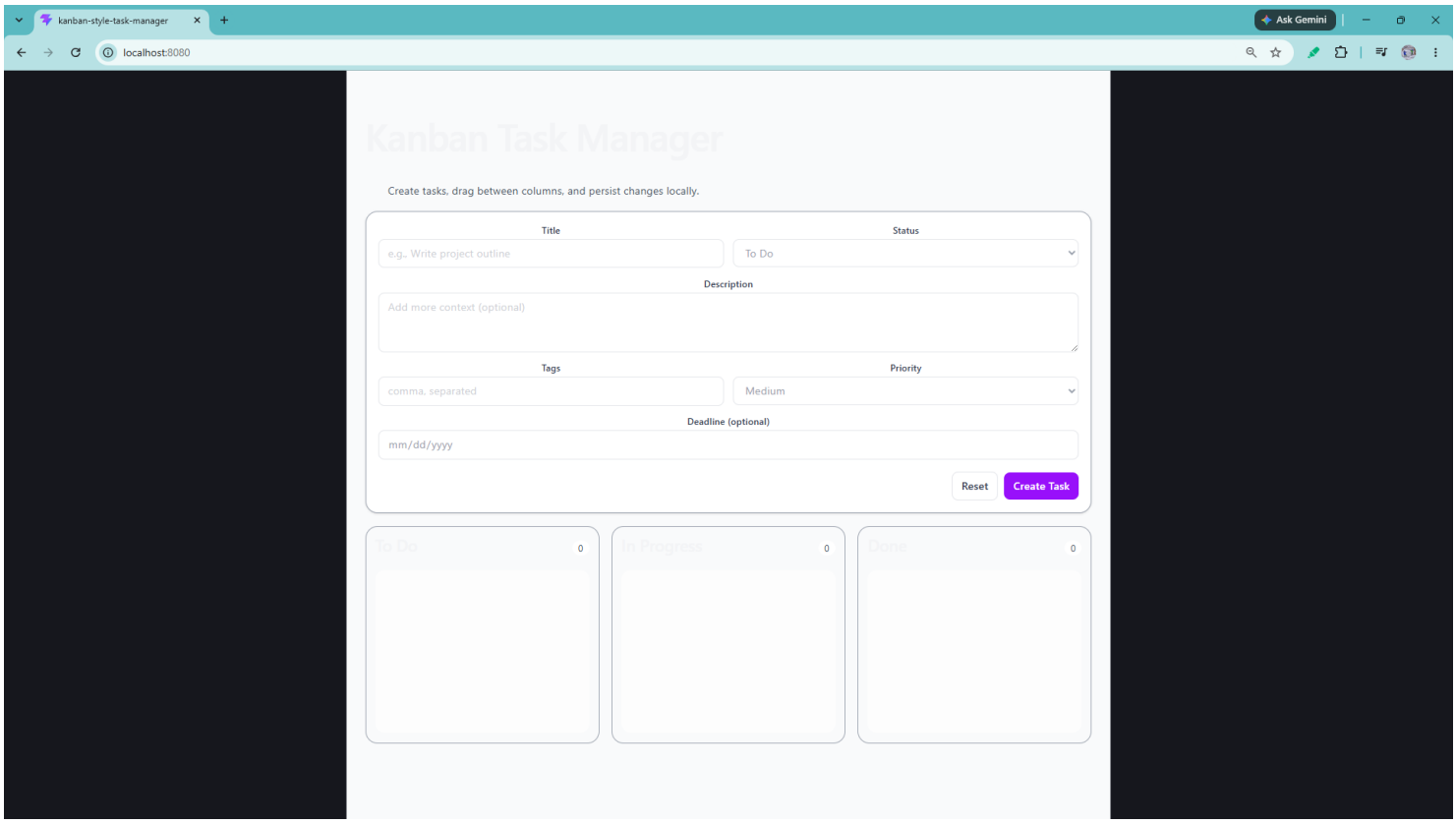
Docker containerization was completed successfully. The multi-stage build produced a slim, production-ready image with health checks passing. And the application is running inside a container.


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Problems Output Debug Console Terminal Ports
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker --version
Docker version 29.7.2, build a7dcaa6
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ npm run build
> kanban-style-task-manager@0.0.0 build
> tsc -b && vite build
vite v8.0.1 building client environment for production...
✓ 71 modules transformed.
computing gzip size...
dist/index.html 0.48 kB | gzip: 0.31 kB
dist/assets/index-DxvxASn5.css 17.99 kB | gzip: 4.64 kB
dist/assets/index-CLsOsSSa.js 263.42 kB | gzip: 80.86 kB
✓ built in 283ms
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
Outline
Timeline
main*
0 0


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore U
Dockerfile U
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Dockerfile U
Dockerfile
1 ### BUILD STAGE ###
2
3 FROM node:24-alpine AS build
4 WORKDIR /app
5 COPY package*.json ./
6 RUN npm install
7 COPY . .
8 RUN npm run build
9
10
11 ### RUNTIME STAGE ###
12
13 FROM nginx:alpine
14 COPY --from=build /app/dist /usr/share/nginx/html
15 EXPOSE 80
16 HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 CMD wget --no-verbose --tries=1 --spider http://localhost:80 || exit 1
17 CMD ["nginx", "-g", "daemon off;"]
18
Problems Output Debug Console Terminal Ports
powershell
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
Outline
Timeline
main*
0 0
Ln 18, Col 1 Spaces: 4 UTF-8 CRLF Docker


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore
Dockerfile
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Problems Output Debug Console Terminal Ports
powershell
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker build -t kanban-dashboard:v1 .
[+] Building 44.5s (14/14) FINISHED docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.2s
=> => transferring dockerfile: 468B 0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 2.6s
=> [internal] load metadata for docker.io/library/node:24-alpine 2.6s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 93B 0.0s
=> [build 1/6] FROM docker.io/library/node:24-alpine@sha256:d32cdf619f63fe0471182d08996dd516c6275bb5fd31ae06e55a570bd9e1ad43 23.6s
=> => resolve docker.io/library/node:24-alpine@sha256:d32cdf619f63fe0471182d08996dd516c6275bb5fd31ae06e55a570bd9e1ad43 0.0s
=> => sha256:e27d3f3e3cb67c9f573672ff4d772cac332d144c55b1f4cf92de199f6572dbec 446B / 446B 0.4s
=> => sha256:4a7016f72779c9040cee0836ca5b79ae36c19d053dff298915dbb1927b5b3456 1.26MB / 1.26MB 1.0s
=> => sha256:08d42cdba493bec7b21d91857a92221fcd480ebfe48f8da3bbd557218f6b9de6 53.10MB / 53.10MB 22.3s
=> => sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 3.85MB / 3.85MB 11.3s
=> => extracting sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 0.2s
=> => extracting sha256:08d42cdba493bec7b21d91857a92221fcd480ebfe48f8da3bbd557218f6b9de6 0.9s
=> => extracting sha256:4a7016f72779c9040cee0836ca5b79ae36c19d053dff298915dbb1927b5b3456 0.1s
=> => extracting sha256:e27d3f3e3cb67c9f573672ff4d772cac332d144c55b1f4cf92de199f6572dbec 0.0s
=> [stage-1 1/2] FROM docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913 15.5s
=> => resolve docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913 0.0s
=> => sha256:fb597529c91648bab2279939e01b745210875081852c73000aed882c000ff5de 20.50MB / 20.50MB 14.5s
=> => sha256:da4dea1b00afb53e163ae330a511ecd6978371d99794c80f439b83b609eb9d66 1.40kB / 1.40kB 3.6s
=> => sha256:8cdfe4f237782ae36780c4634b1457e8d6efa6f929fc3e46b0a2074ba54acd56 1.21kB / 1.21kB 1.0s
=> => sha256:0d62d88506ba127db40e7c137e3a2dcf65e563709676828820b8c82277a6caeb 404B / 404B 0.4s
=> => sha256:1ed8e39a7434c8975957bb78530f191c92eecbd9c7309966192ba276a7c0ac61 955B / 955B 0.5s
=> => sha256:0ea93572787841bba2f3921ece8dc2db925c22232a1388c766881ebe3bd3f67d 628B / 628B 0.4s
=> => sha256:d94291c262619626bc7a27b61b090d85365e85c61f92ca49b190a26930121299 1.90MB / 1.90MB 5.8s
=> => extracting sha256:d94291c262619626bc7a27b61b090d85365e85c61f92ca49b190a26930121299 0.2s
=> => extracting sha256:0ea93572787841bba2f3921ece8dc2db925c22232a1388c766881ebe3bd3f67d 0.0s
=> => extracting sha256:1ed8e39a7434c8975957bb78530f191c92eecbd9c7309966192ba276a7c0ac61 0.0s
=> => extracting sha256:0d62d88506ba127db40e7c137e3a2dcf65e563709676828820b8c82277a6caeb 0.0s
=> => extracting sha256:8cdfe4f237782ae36780c4634b1457e8d6efa6f929fc3e46b0a2074ba54acd56 0.0s
=> => extracting sha256:da4dea1b00afb53e163ae330a511ecd6978371d99794c80f439b83b609eb9d66 0.0s
=> => extracting sha256:fb597529c91648bab2279939e01b745210875081852c73000aed882c000ff5de 0.3s
=> [internal] load build context 0.1s
=> => transferring context: 251.94kB 0.1s
=> [build 2/6] WORKDIR /app 0.1s
=> [build 3/6] COPY package*.json ./ 0.0s
=> [build 4/6] RUN npm install 15.3s
=> [build 5/6] COPY . . 0.1s
=> [build 6/6] RUN npm run build 2.2s
=> [stage-1 2/2] COPY --from=build /app/dist /usr/share/nginx/html 0.0s
=> exporting to image 0.2s
=> => exporting layers 0.1s
=> => exporting manifest sha256:cb887afcee95b39811c27e1b21a647c3f716220ac6f4a3d300366af916907594 0.0s
=> => exporting config sha256:9ee2f1c9d5e9def4917c20af53e13f369bfb79c7bbf7d089ec57f24ac3e281a1 0.0s
=> => exporting attestation manifest sha256:3ba7c0410eb9973d1d5a253f2349cb4bd5a5285d2f31d81ef5aeaeb6d4a465bf 0.0s
=> => exporting manifest list sha256:053d687f2916f861b3fe5ef4d91f90f5d69ab8635b14e8c691c269659134eb11 0.0s
=> => naming to docker.io/library/kanban-dashboard:v1 0.0s
=> => unpacking to docker.io/library/kanban-dashboard:v1 0.0s
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
main* 0 0
Ln 18, Col 1 Spaces: 4 UTF-8 CRLF { } Docker


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore
Dockerfile
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Problems Output Debug Console Terminal Ports
powershell
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker images
Info U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
kanban-dashboard:v1 053d687f2916 93.8MB 26.4MB
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker run -d --name kanban-dashboard -p 8080:80 kanban-dashboard:v1
15f3b613ef9215eaa446763903c1dcafbb44388c415684a536ad2db9f51b1a31
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
15f3b613ef92 kanban-dashboard:v1 "/docker-entrypoint.…" 11 seconds ago Up 11 seconds (healthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp kanban-dashboard
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
main* 0 0
Ln 18, Col 1 Spaces: 4 UTF-8 CRLF { } Docker


kanban-style-task-manager
Ask Gemini
localhost:8080
Kanban Task Manager
Create tasks, drag between columns, and persist changes locally.
Title
e.g., Write project outline
Status
To Do
Description
Add more context (optional)
Tags
comma, separated
Priority
Medium
Deadline (optional)
mm/dd/yyyy
Reset
Create Task
To Do
0
In Progress
0
Done
0

Step 3 : Docker Security

Overview

With the containerized application running, the next step was to harden the Docker image by applying security best practices. This included switching to a non-root user, adding a .dockerignore file, and scanning the image to confirm no secrets or critical vulnerabilities were embedded.

Security Hardening

The Dockerfile was updated to create a dedicated non-root user and group (appuser/appgroup) in the runtime stage. Ownership of the required Nginx directories was transferred to appuser, and the USER appuser directive was added to ensure the container process runs without root privileges.

A .dockerignore file was created to prevent sensitive and unnecessary files from being copied into the image during the build context. The following entries were added: node_modules, dist, .git, .env, .env.* and npm-debug.log*.

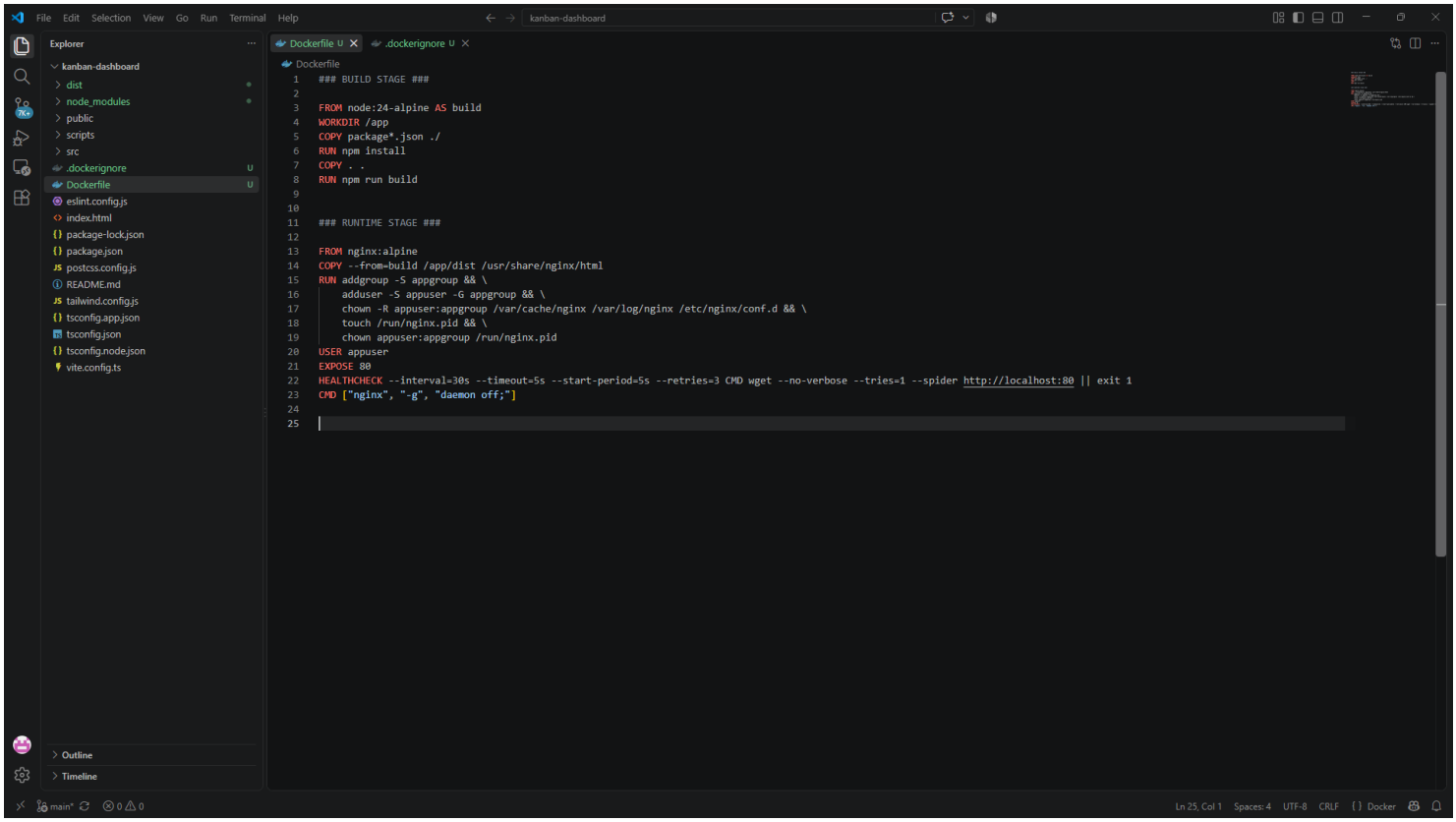
The Dockerfile was checked using grep to confirm no secrets, tokens, passwords, or API keys were present.

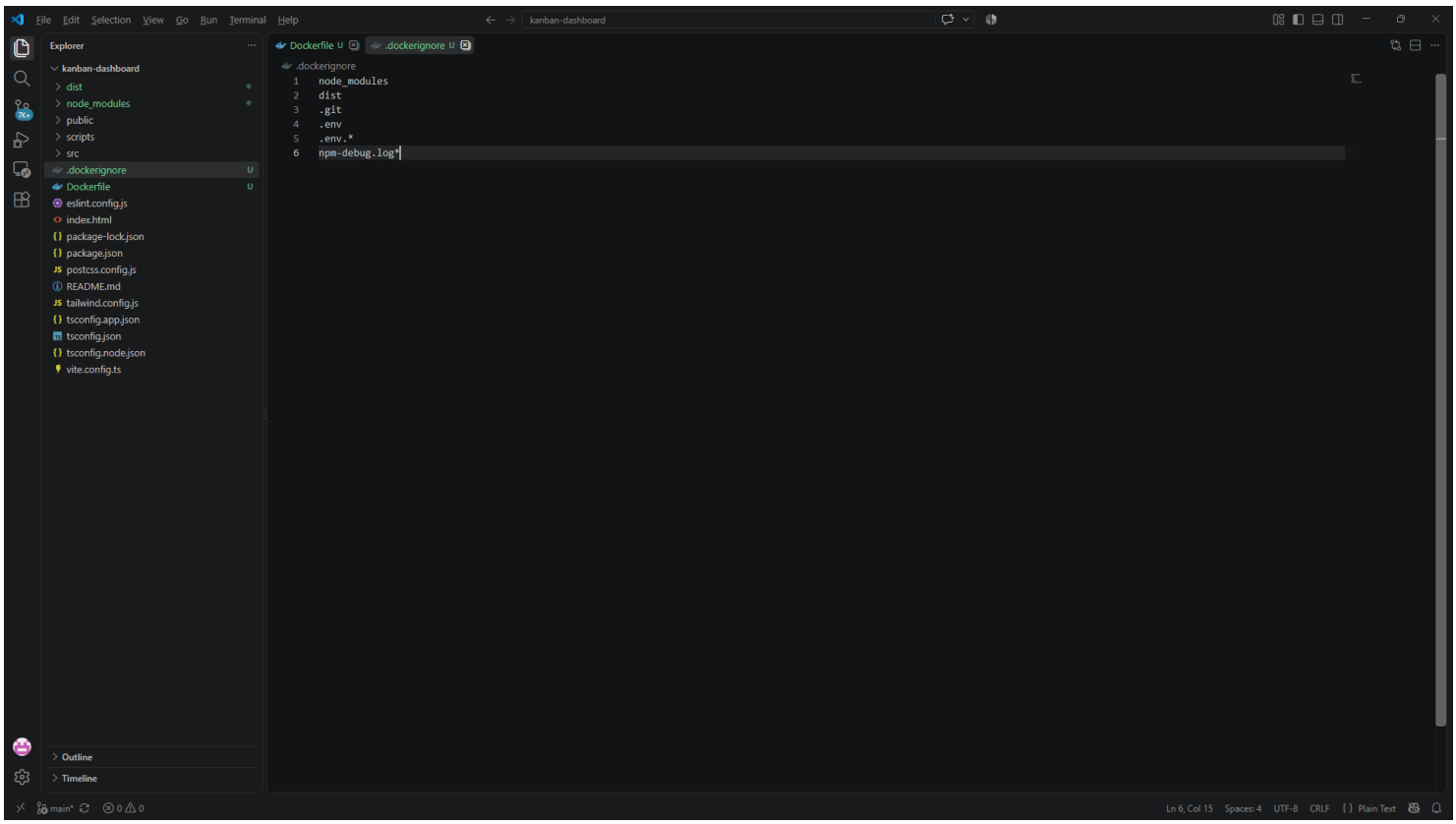
Validation

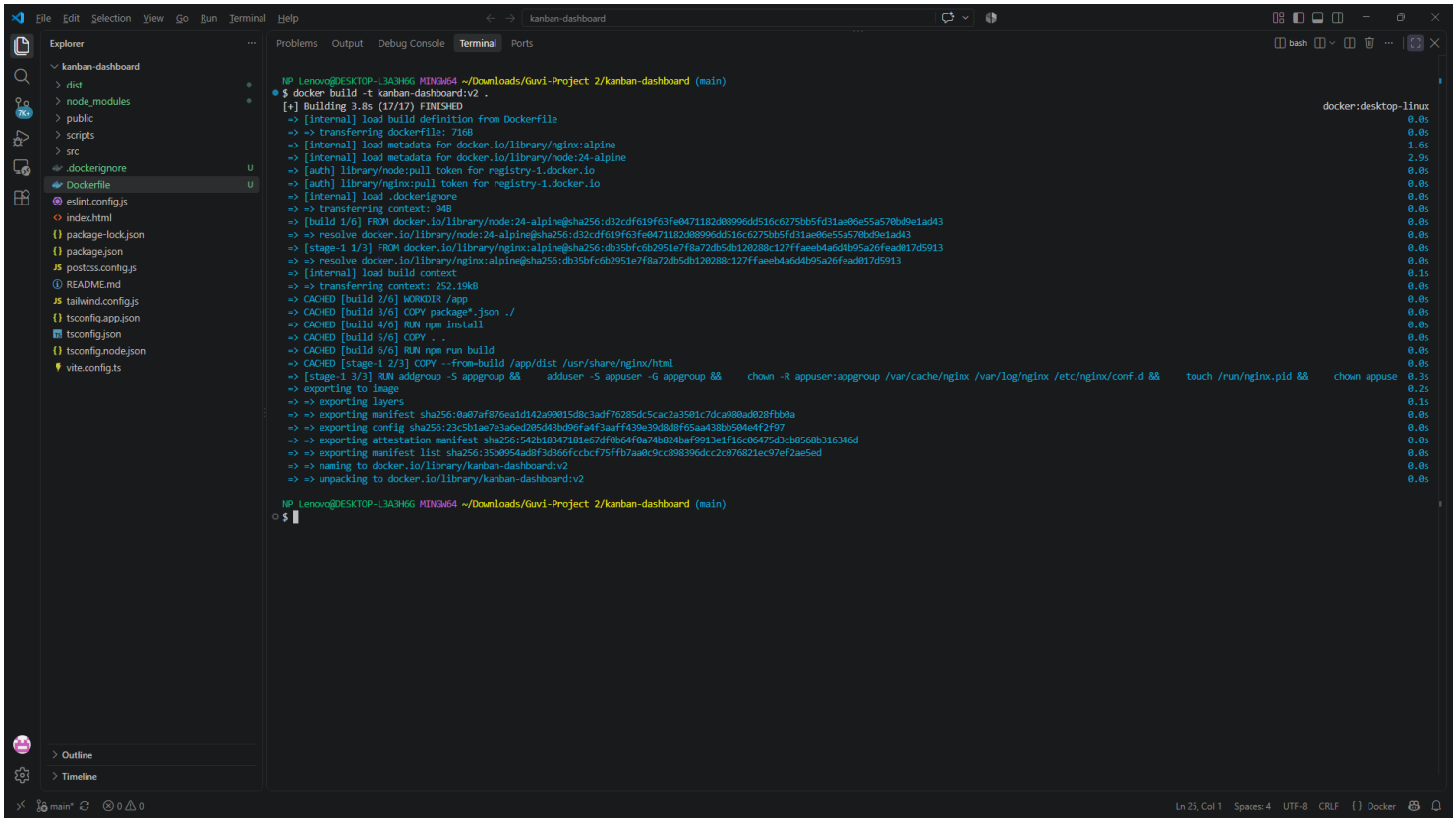
- docker inspect confirmed the container is running as appuser
- docker ps confirmed container status as healthy with ports mapped as 0.0.0.0:8080->80/tcp
- grep confirmed no secrets or tokens found in the Dockerfile
- docker scout quickview reported 0 critical, 0 high, 0 medium vulnerabilities
- Non-root user policy showed as passed in the scout report

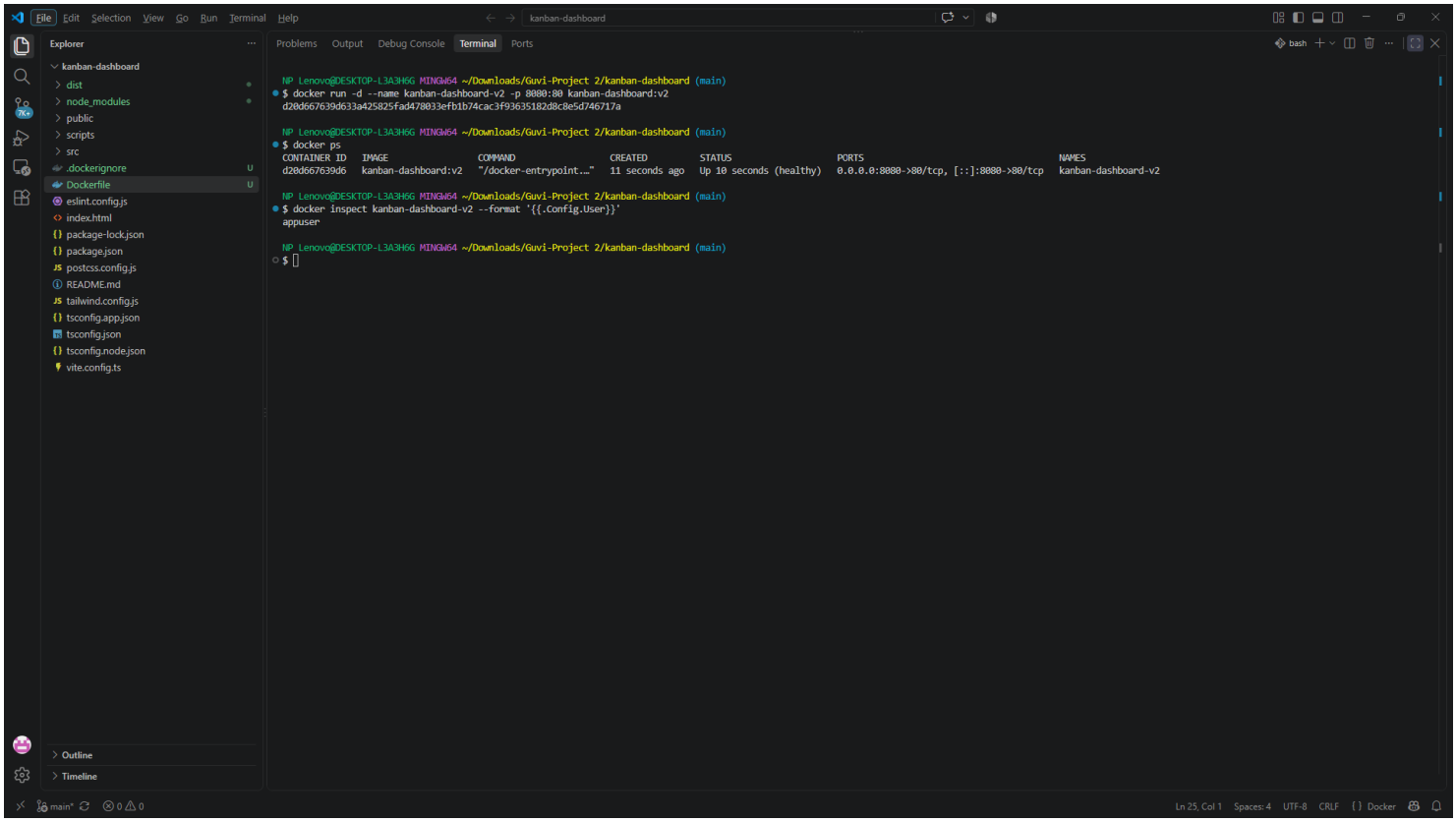
Result

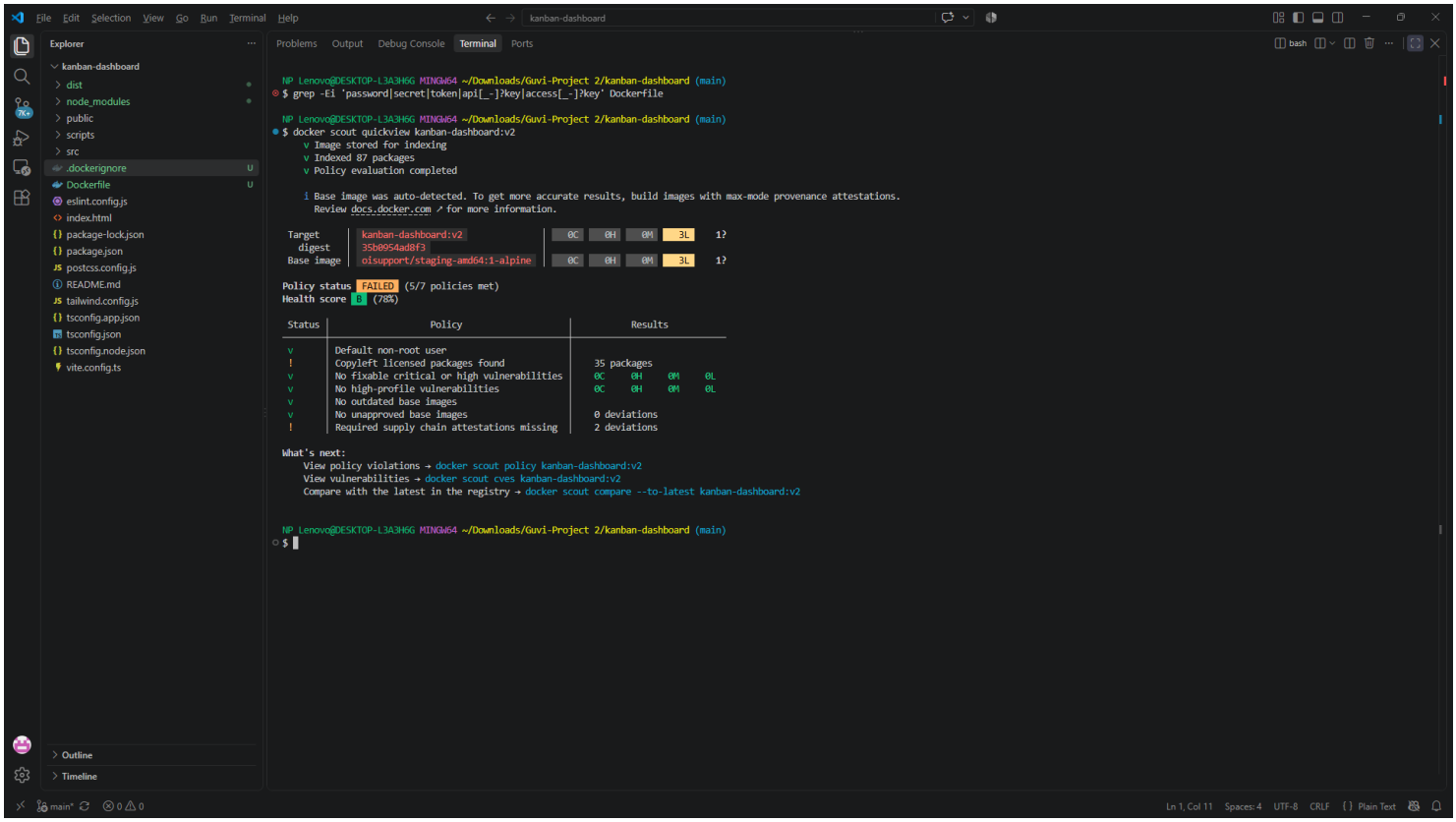
Docker security hardening was completed successfully. The container runs as a non-root user, sensitive files are excluded from the build context, no obvious secrets were found in the Dockerfile, and the image scan confirmed no critical or high vulnerabilities.


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore U
Dockerfile U
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Dockerfile U
.dockerignore U
Dockerfile
BUILD STAGE
FROM node:24-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
RUNTIME STAGE
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
RUN addgroup -S appgroup && \
adduser -S appuser -G appgroup && \
chown -R appuser:appgroup /var/cache/nginx /var/log/nginx /etc/nginx/conf.d && \
touch /run/nginx.pid && \
chown appuser:appgroup /run/nginx.pid
USER appuser
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 CMD wget --no-verbose --tries=1 --spider http://localhost:80 || exit 1
CMD ["nginx", "-g", "daemon off;"]
main*
0 0
Ln 25, Col 1 Spaces: 4 UTF-8 CRLF Docker


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore U
Dockerfile U
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Dockerfile U
.dockerignore U
.dockerignore
node_modules
dist
.git
.env
.env.*
npm-debug.log*
main*
0 0
Ln 6, Col 15 Spaces: 4 UTF-8 CRLF Plain Text


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore
Dockerfile
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Problems Output Debug Console Terminal Ports
bash
MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker build -t kanban-dashboard:v2 .
[+] Building 3.8s (17/17) FINISHED docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 716B 0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 1.6s
=> [internal] load metadata for docker.io/library/node:24-alpine 2.9s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [auth] library/nginx:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 94B 0.0s
=> [build 1/6] FROM docker.io/library/node:24-alpine@sha256:d32cdf619f63fe0471182d08996dd516c6275bb5fd31ae06e55a570bd9e1ad43 0.0s
=> => resolve docker.io/library/node:24-alpine@sha256:d32cdf619f63fe0471182d08996dd516c6275bb5fd31ae06e55a570bd9e1ad43 0.0s
=> [stage-1 1/3] FROM docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913 0.0s
=> => resolve docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 252.19kB 0.0s
=> CACHED [build 2/6] WORKDIR /app 0.0s
=> CACHED [build 3/6] COPY package*.json ./ 0.0s
=> CACHED [build 4/6] RUN npm install 0.0s
=> CACHED [build 5/6] COPY . . 0.0s
=> CACHED [build 6/6] RUN npm run build 0.0s
=> CACHED [stage-1 2/3] COPY --from=build /app/dist /usr/share/nginx/html 0.0s
=> [stage-1 3/3] RUN addgroup -S appgroup && adduser -S appuser -G appgroup && chown -R appuser:appgroup /var/cache/nginx /var/log/nginx /etc/nginx/conf.d && touch /run/nginx.pid && chown appuse 0.3s
=> exporting to image 0.2s
=> => exporting layers 0.1s
=> => exporting manifest sha256:0a07af876ea1d142a90015d8c3adf76285dc5cac2a3501c7dca980ad028fbb0a 0.0s
=> => exporting config sha256:23c5b1ae7e3a6ed205d43bd96fa4f3aaff439e39d8d8f65aa438bb504e4f2f97 0.0s
=> => exporting attestation manifest sha256:542b1834718 1e67df0b64f0a74b824baf9913e1f16c06475d3cb8568b316346d 0.0s
=> => exporting manifest list sha256:35b0954ad8f3d366fccbcf75ffb7aa0c9cc898396dcc2c076821ec97ef2ae5ed 0.0s
=> => naming to docker.io/library/kanban-dashboard:v2 0.0s
=> => unpacking to docker.io/library/kanban-dashboard:v2 0.0s
MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
main*
0 0
Ln 25, Col 1 Spaces: 4 UTF-8 CRLF { } Docker


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore
Dockerfile
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Problems Output Debug Console Terminal Ports
bash
MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker run -d --name kanban-dashboard-v2 -p 8080:80 kanban-dashboard:v2
d20d667639d633a425825fad478033efb1b74cac3f93635182d8c8e5d746717a
MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
d20d667639d6 kanban-dashboard:v2 "/docker-entrypoint.…" 11 seconds ago Up 10 seconds (healthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp kanban-dashboard-v2
MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker inspect kanban-dashboard-v2 --format '{{.Config.User}}'
appuser
MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
main*
0 0
Ln 25, Col 1 Spaces: 4 UTF-8 CRLF { } Docker


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore
Dockerfile
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
Problems Output Debug Console Terminal Ports
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ grep -Ei 'password|secret|token|api[_-]?key|access[_-]?key' Dockerfile
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker scout quickview kanban-dashboard:v2
v Image stored for indexing
v Indexed 87 packages
v Policy evaluation completed
i Base image was auto-detected. To get more accurate results, build images with max-mode provenance attestations.
Review docs.docker.com ↗ for more information.
Target kanban-dashboard:v2 0C 0H 0M 3L 1?
digest 35b0954ad8f3
Base image oisupport/staging-amd64:1-alpine 0C 0H 0M 3L 1?
Policy status FAILED (5/7 policies met)
Health score B (78%)
Status Policy Results
v Default non-root user
! Copyleft licensed packages found 35 packages
v No fixable critical or high vulnerabilities 0C 0H 0M 0L
v No high-profile vulnerabilities 0C 0H 0M 0L
v No outdated base images
v No unapproved base images 0 deviations
! Required supply chain attestations missing 2 deviations
What's next:
View policy violations → docker scout policy kanban-dashboard:v2
View vulnerabilities → docker scout cves kanban-dashboard:v2
Compare with the latest in the registry → docker scout compare --to-latest kanban-dashboard:v2
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
main*
Ln 1, Col 11 Spaces: 4 UTF-8 CRLF Plain Text

Step 4 : Container Registry & Versioning

Overview

With the secured image ready, the next step was to push it to a container registry for versioned storage and remote access. The image was tagged, pushed to Docker Hub, and then pulled and run on an AWS EC2 instance to confirm it works identically outside the local environment.

Registry Setup

A public repository named priyanp/project-kanban-dashboard was created on Docker Hub. The local image kanban-dashboard:v2 was tagged as priyanp/project-kanban-dashboard:build-1 and pushed to the registry after authenticating via docker login.

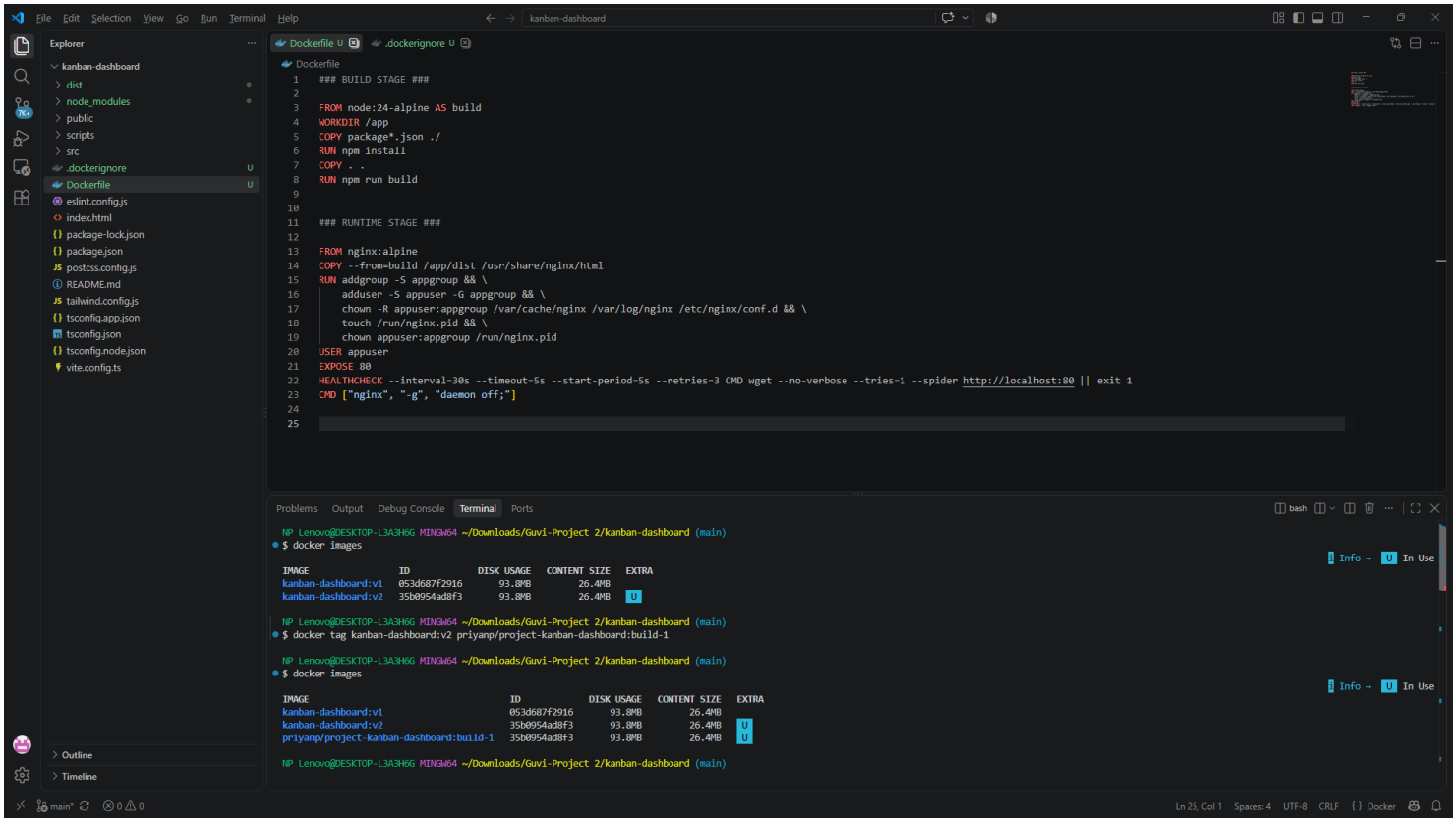
Validation

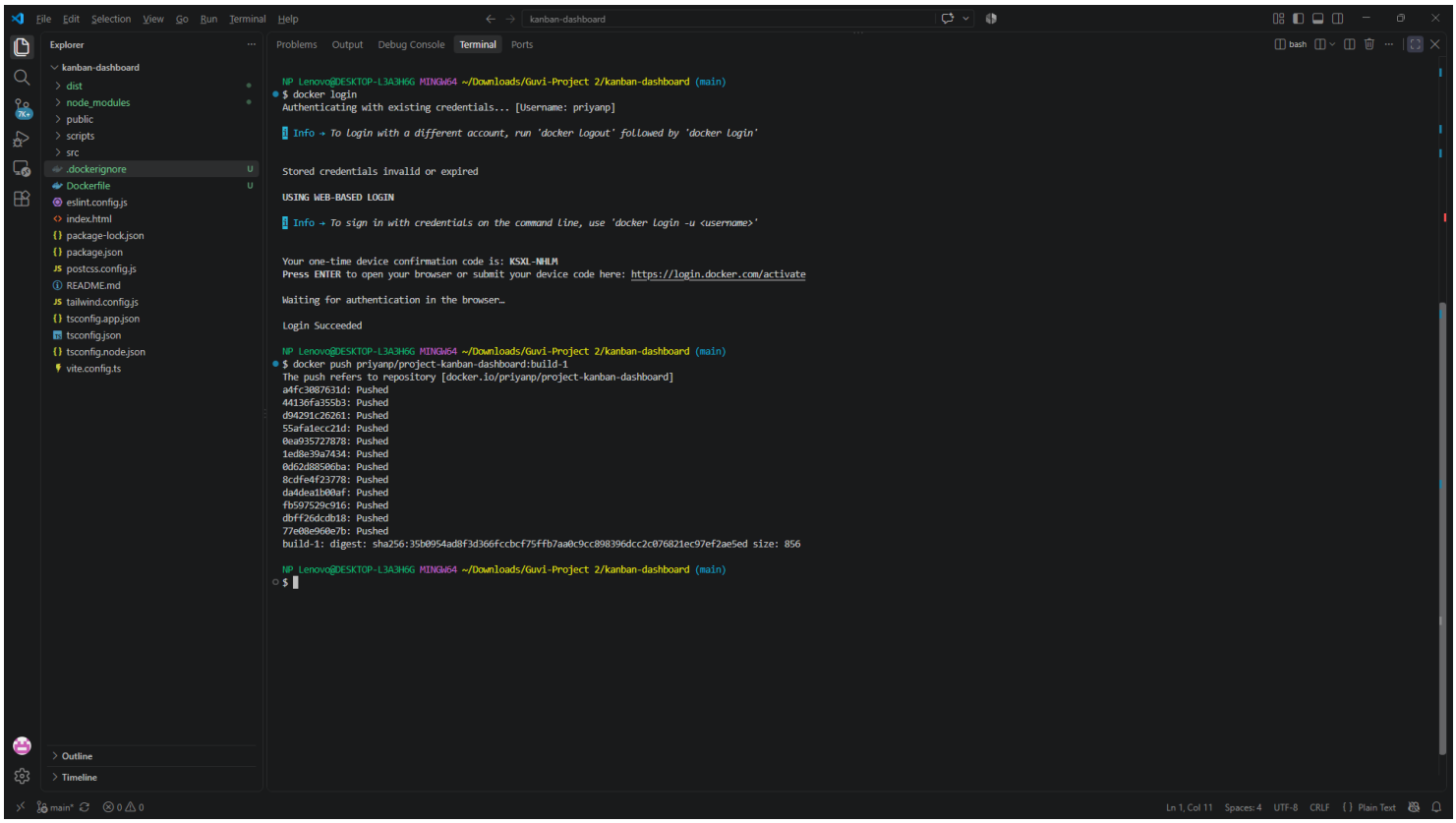
Docker Hub confirmed the repository contains 1 tag (build-1) with a repository size of 25.1 MB, pushed successfully.

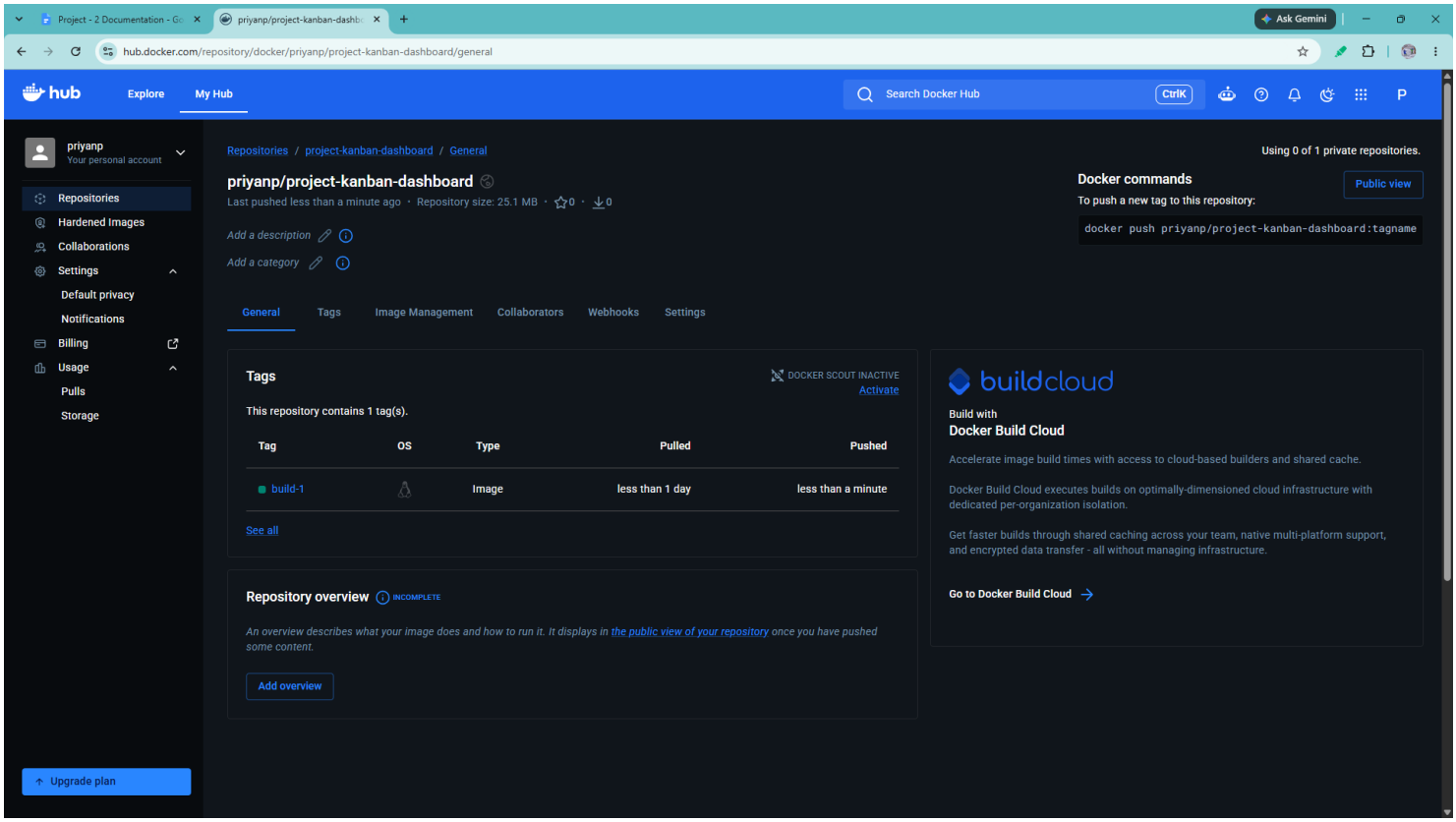
On the AWS EC2 instance, the image was pulled successfully from Docker Hub and the container was started with port mapping 8080:80. Running sudo docker ps confirmed that the container was healthy. The Kanban Task Manager loaded successfully in the browser using the EC2 public IP and port 8080.

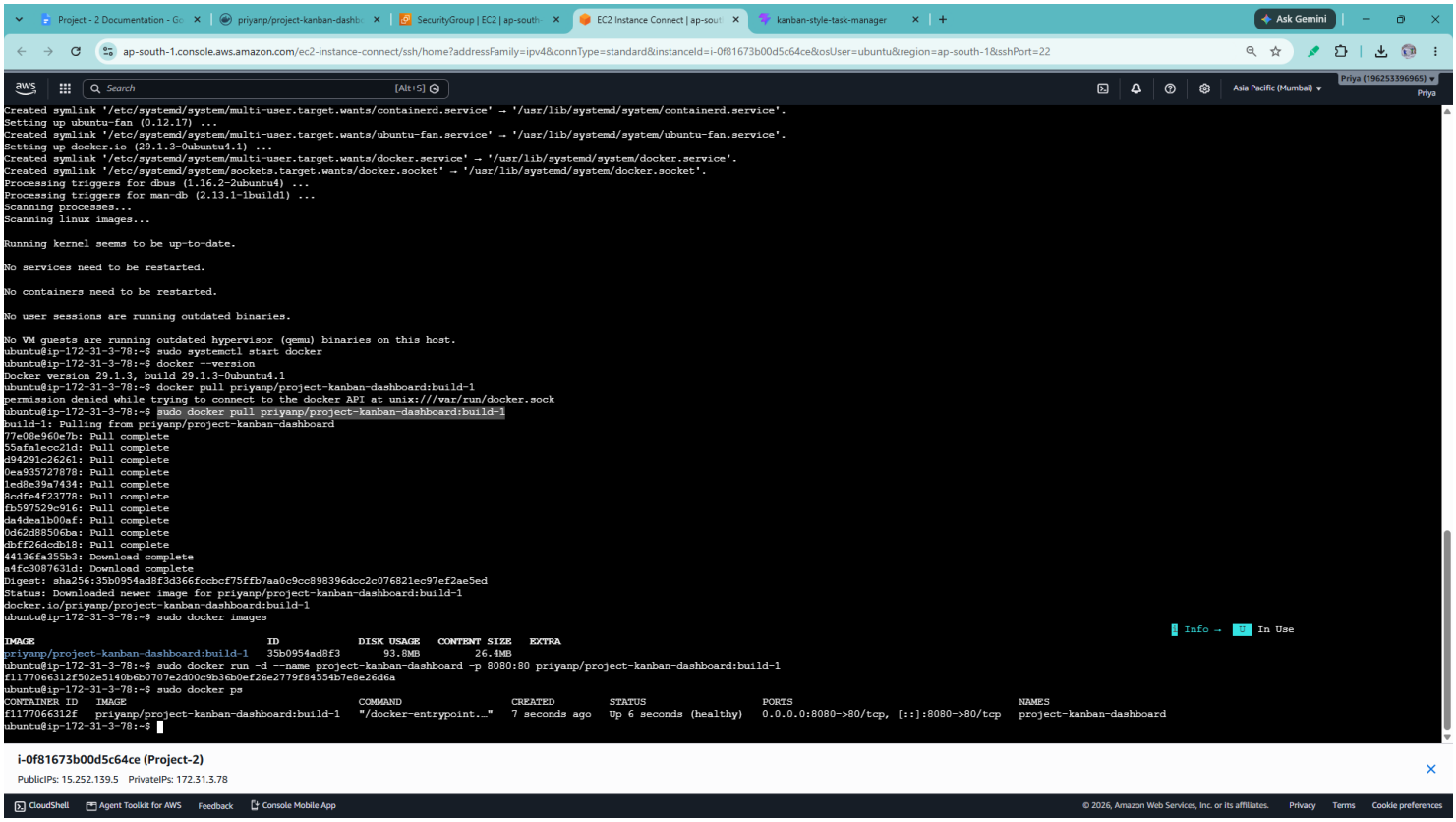
Result

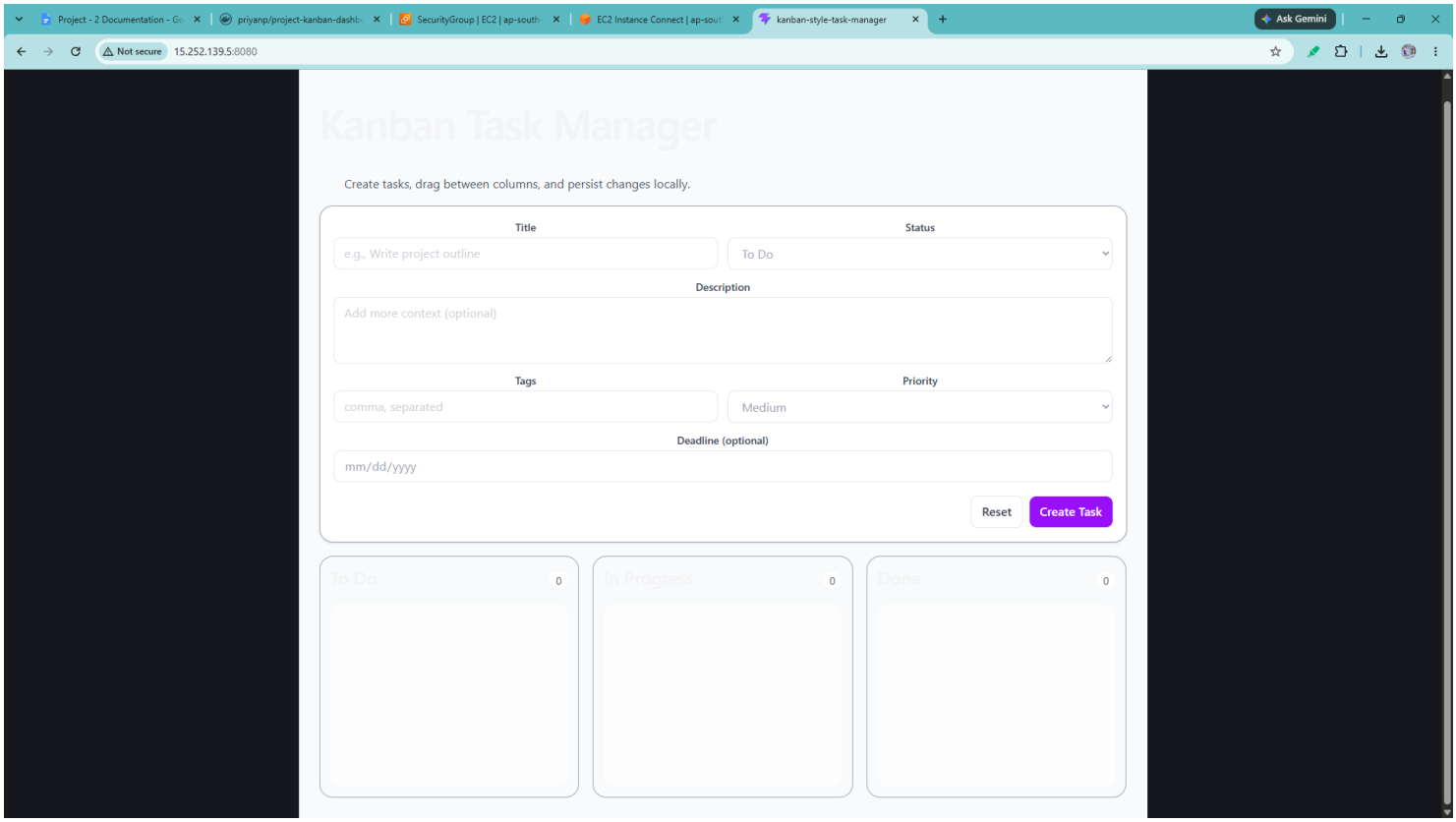
The image was successfully pushed to Docker Hub with a versioned tag (build-1) and pulled onto a remote AWS EC2 instance. The application ran identically to the local environment, confirming the registry setup is working as expected.


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore U
Dockerfile U
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Dockerfile U
.dockerignore U
Dockerfile
1 ### BUILD STAGE ###
2
3 FROM node:24-alpine AS build
4 WORKDIR /app
5 COPY package*.json ./
6 RUN npm install
7 COPY . .
8 RUN npm run build
9
10
11 ### RUNTIME STAGE ###
12
13 FROM nginx:alpine
14 COPY --from=build /app/dist /usr/share/nginx/html
15 RUN addgroup -S appgroup && \
16 adduser -S appuser -G appgroup && \
17 chown -R appuser:appgroup /var/cache/nginx /var/log/nginx /etc/nginx/conf.d && \
18 touch /run/nginx.pid && \
19 chown appuser:appgroup /run/nginx.pid
20 USER appuser
21 EXPOSE 80
22 HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 CMD wget --no-verbose --tries=1 --spider http://localhost:80 || exit 1
23 CMD ["nginx", "-g", "daemon off;"]
24
25
Problems Output Debug Console Terminal Ports
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker images
Info → U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
kanban-dashboard:v1 053d687f2916 93.8MB 26.4MB
kanban-dashboard:v2 35b0954ad8f3 93.8MB 26.4MB U
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker tag kanban-dashboard:v2 priyanp/project-kanban-dashboard:build-1
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker images
Info → U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
kanban-dashboard:v1 053d687f2916 93.8MB 26.4MB
kanban-dashboard:v2 35b0954ad8f3 93.8MB 26.4MB U
priyanp/project-kanban-dashboard:build-1 35b0954ad8f3 93.8MB 26.4MB U
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
Outline
Timeline
main* 0 0
Ln 25, Col 1 Spaces: 4 UTF-8 CRLF { } Docker


File Edit Selection View Go Run Terminal Help
kanban-dashboard
Explorer
kanban-dashboard
dist
node_modules
public
scripts
src
.dockerignore U
Dockerfile U
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Problems Output Debug Console Terminal Ports
bash
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker login
Authenticating with existing credentials... [Username: priyanp]
Info → To login with a different account, run 'docker logout' followed by 'docker login'
Stored credentials invalid or expired
USING WEB-BASED LOGIN
Info → To sign in with credentials on the command line, use 'docker login -u <username>'
Your one-time device confirmation code is: KSXL-NHLM
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate
Waiting for authentication in the browser…
Login Succeeded
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$ docker push priyanp/project-kanban-dashboard:build-1
The push refers to repository [docker.io/priyanp/project-kanban-dashboard]
a4fc3087631d: Pushed
44136fa355b3: Pushed
d94291c26261: Pushed
55afa1ecc21d: Pushed
0ea935727878: Pushed
1ed8e39a7434: Pushed
0d62d88506ba: Pushed
8cdfe4f23778: Pushed
da4dea1b00af: Pushed
fb597529c916: Pushed
dbff26dcdb18: Pushed
77e08e960e7b: Pushed
build-1: digest: sha256:35b0954ad8f3d366fccbcf75ffb7aa0c9cc898396dcc2c076821ec97ef2ae5ed size: 856
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Guvi-Project 2/kanban-dashboard (main)
$
Outline
Timeline
main* 0 0
Ln 1, Col 11 Spaces: 4 UTF-8 CRLF { } Plain Text


hub.docker.com/repository/docker/priyanp/project-kanban-dashboard/general
Explore
My Hub
Search Docker Hub
priyanp
Your personal account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade plan
Repositories / project-kanban-dashboard / General
priyanp/project-kanban-dashboard
Last pushed less than a minute ago · Repository size: 25.1 MB · 0 · 0
Add a description
Add a category
Using 0 of 1 private repositories.
Docker commands
Public view
To push a new tag to this repository:
docker push priyanp/project-kanban-dashboard:tagname
General
Tags
Image Management
Collaborators
Webhooks
Settings
Tags
DOCKER SCOUT INACTIVE
Activate
This repository contains 1 tag(s).
Tag
OS
Type
Pulled
Pushed
build-1
Image
less than 1 day
less than a minute
See all
Repository overview
INCOMPLETE
An overview describes what your image does and how to run it. It displays in the public view of your repository once you have pushed some content.
Add overview
buildcloud
Build with
Docker Build Cloud
Accelerate image build times with access to cloud-based builders and shared cache.
Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.
Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.
Go to Docker Build Cloud


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&region=ap-south-1&sshPort=22
Created symlink '/etc/systemd/system/multi-user.target.wants/containerd.service' → '/usr/lib/systemd/system/containerd.service'.
Setting up ubuntu-fan (0.12.17) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/ubuntu-fan.service' → '/usr/lib/systemd/system/ubuntu-fan.service'.
Setting up docker.io (29.1.3-0ubuntu4.1) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service' → '/usr/lib/systemd/system/docker.service'.
Created symlink '/etc/systemd/system/sockets.target.wants/docker.socket' → '/usr/lib/systemd/system/docker.socket'.
Processing triggers for dbus (1.16.2-2ubuntu4) ...
Processing triggers for man-db (2.13.1-1build1) ...
Scanning processes...
Scanning linux images...
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-3-78:~$ sudo systemctl start docker
ubuntu@ip-172-31-3-78:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-3-78:~$ docker pull priyanp/project-kanban-dashboard:build-1
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-3-78:~$ sudo docker pull priyanp/project-kanban-dashboard:build-1
build-1: Pulling from priyanp/project-kanban-dashboard
77e08e960e7b: Pull complete
55afalecc21d: Pull complete
d94291c26261: Pull complete
0ea935727878: Pull complete
1ed8e39a7434: Pull complete
8cdfe4f23778: Pull complete
fb597529c916: Pull complete
da4dea1b00af: Pull complete
0d62d88506ba: Pull complete
dbff26dcdb18: Pull complete
44136fa355b3: Download complete
a4fc3087631d: Download complete
Digest: sha256:35b0954ad8f3d366fccbcf75ffb7aa0c9cc898396dcc2c076821ec97ef2ae5ed
Status: Downloaded newer image for priyanp/project-kanban-dashboard:build-1
docker.io/priyanp/project-kanban-dashboard:build-1
ubuntu@ip-172-31-3-78:~$ sudo docker images
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
priyanp/project-kanban-dashboard:build-1 35b0954ad8f3 93.8MB 26.4MB
ubuntu@ip-172-31-3-78:~$ sudo docker run -d --name project-kanban-dashboard -p 8080:80 priyanp/project-kanban-dashboard:build-1
f1177066312f502e5140b6b0707e2d00c9b36b0ef26e2779f84554b7e8e26d6a
ubuntu@ip-172-31-3-78:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
f1177066312f priyanp/project-kanban-dashboard:build-1 "/docker-entrypoint..." 7 seconds ago Up 6 seconds (healthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp project-kanban-dashboard
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App


Project - 2 Documentation - Go
priyanp/project-kanban-dashb
SecurityGroup | EC2 | ap-south
EC2 Instance Connect | ap-sout
kanban-style-task-manager
Ask Gemini
Not secure
15.252.139.5:8080
Kanban Task Manager
Create tasks, drag between columns, and persist changes locally.
Title
e.g., Write project outline
Status
To Do
Description
Add more context (optional)
Tags
comma, separated
Priority
Medium
Deadline (optional)
mm/dd/yyyy
Reset
Create Task
To Do
0
In Progress
0
Done
0

Step 5 : AWS Infrastructure

Overview

With the Docker image available on Docker Hub, the next step was to set up the AWS infrastructure. An EC2 instance was launched and configured with Docker and Jenkins, with Security Group rules restricting access to only the required ports.

EC2 Instance Setup

An Ubuntu-based EC2 instance named Project-2 was launched. Docker and Jenkins were installed on the instance, with Java installed as a Jenkins dependency.

Security Group Configuration

The Security Group was configured with the following inbound rules:

- Port 22 — SSH access
- Port 8080 — Jenkins
- Port 8081 — Kanban application

Initially, the application container was mapped to host port 8080. Since Jenkins also uses port 8080, this resulted in a port conflict and Jenkins failed to start with an Address already in use error. The conflict was resolved by changing the application mapping to:

EC2 port 8081 → Container port 80

This allowed Jenkins and the Kanban application to run simultaneously on the same EC2 instance.

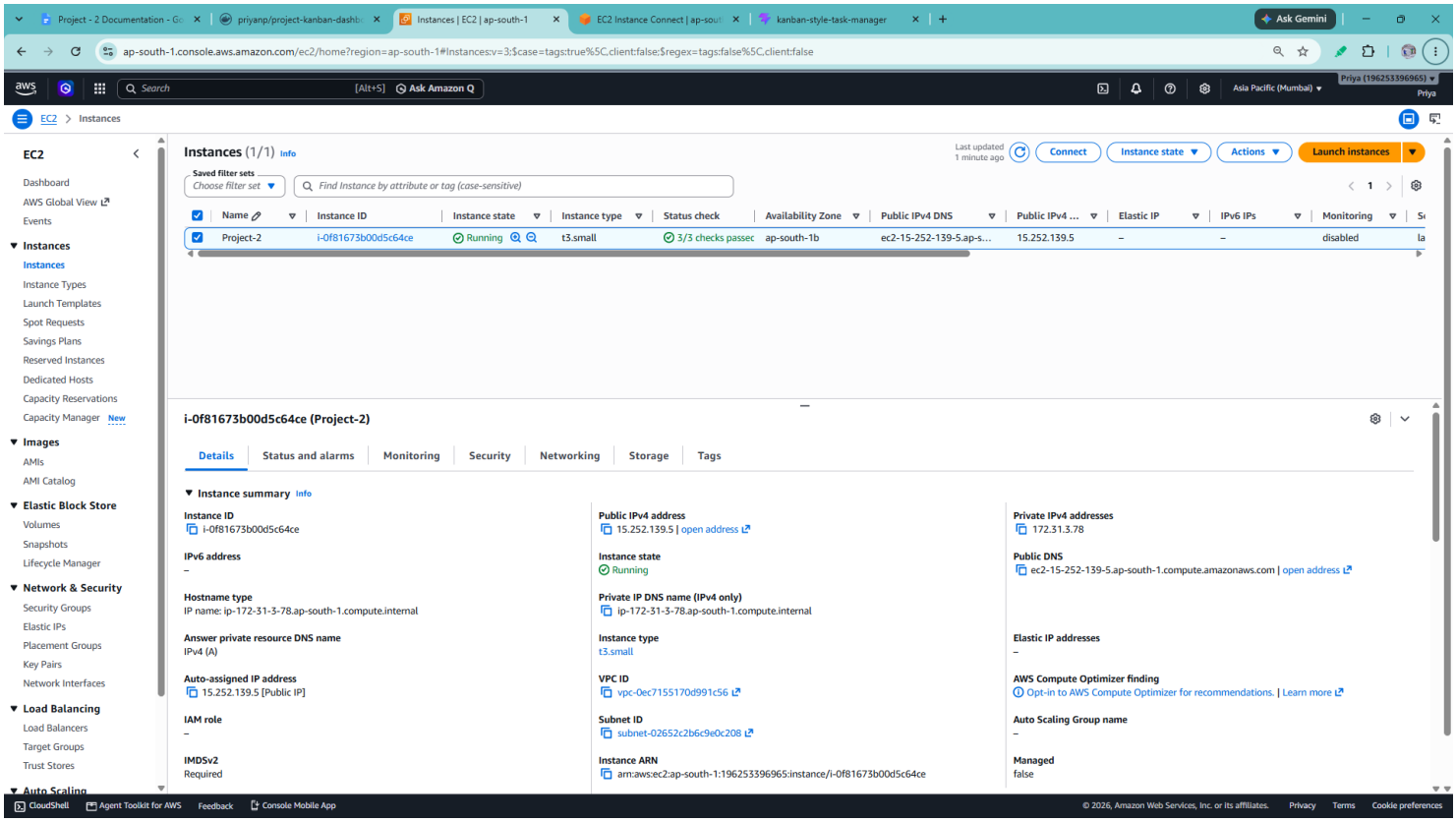
Validation

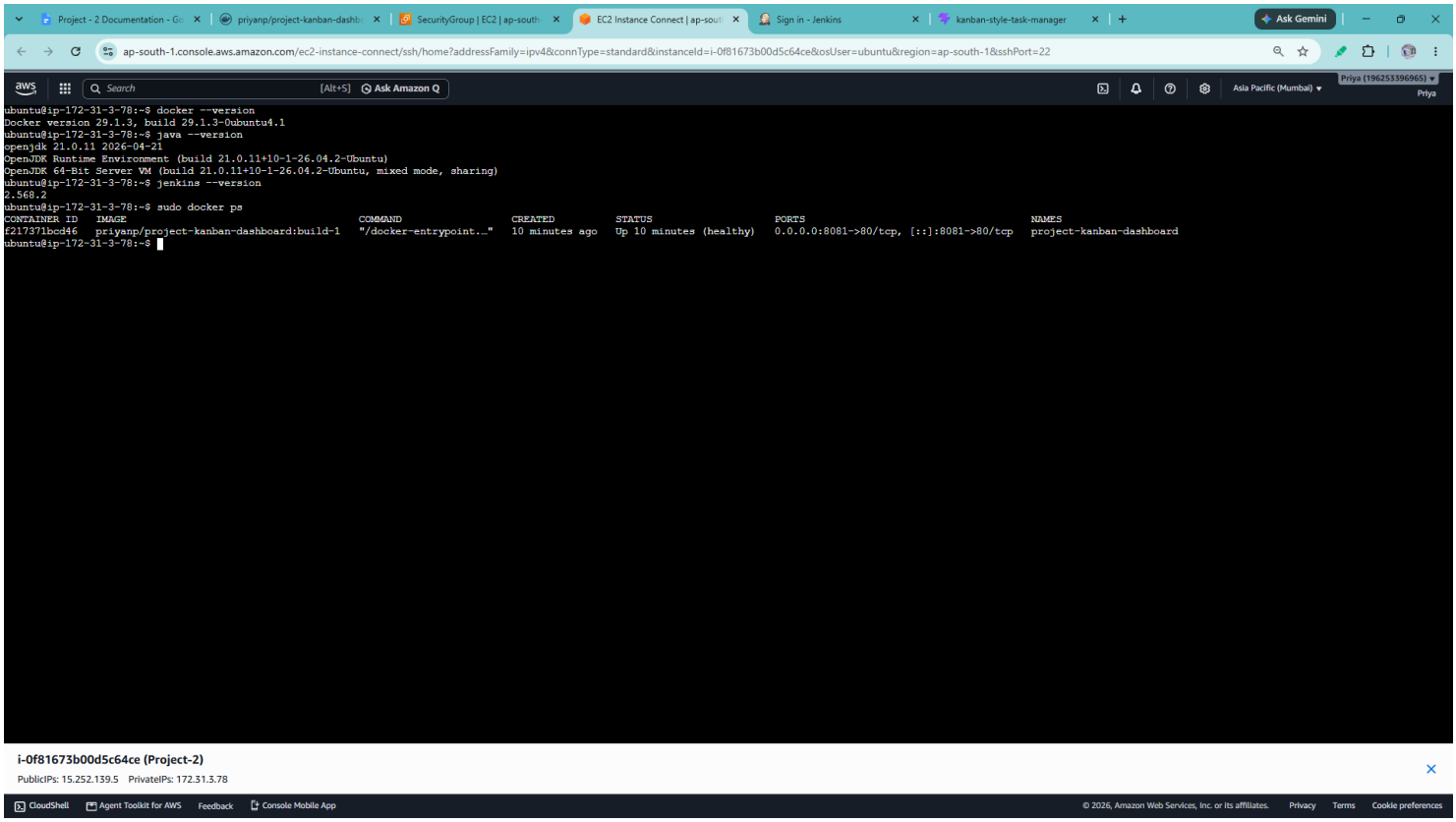
- EC2 instance confirmed running with public IP 15.252.139.5
- Jenkins UI accessible at 15.252.139.5:8080 and displayed the Unlock Jenkins setup page
- sudo docker ps confirmed the container status as healthy
- Container port mapping confirmed as 0.0.0.0:8081 → 80/tcp
- Kanban application loaded successfully in the browser at 15.252.139.5:8081

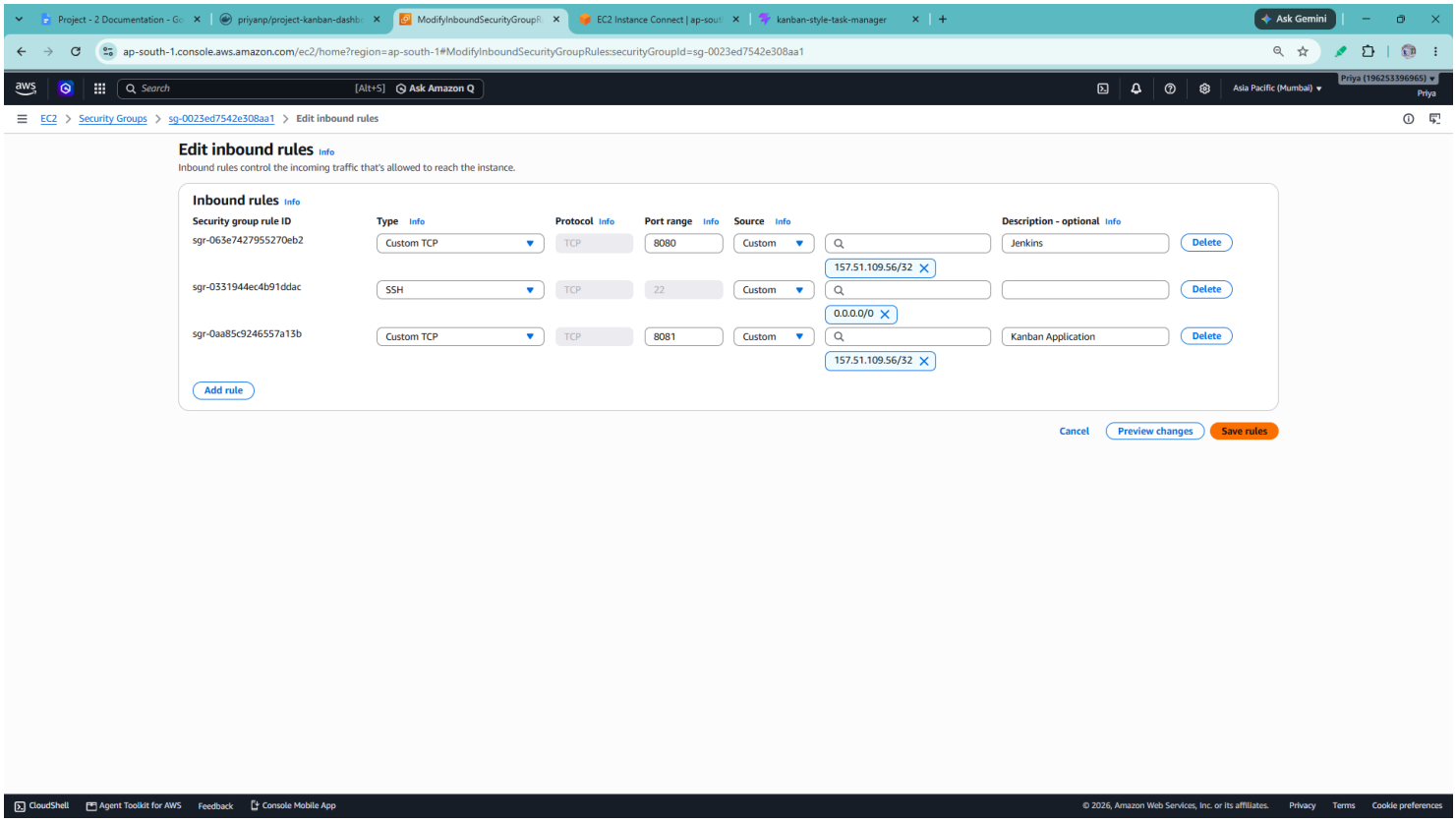
Result

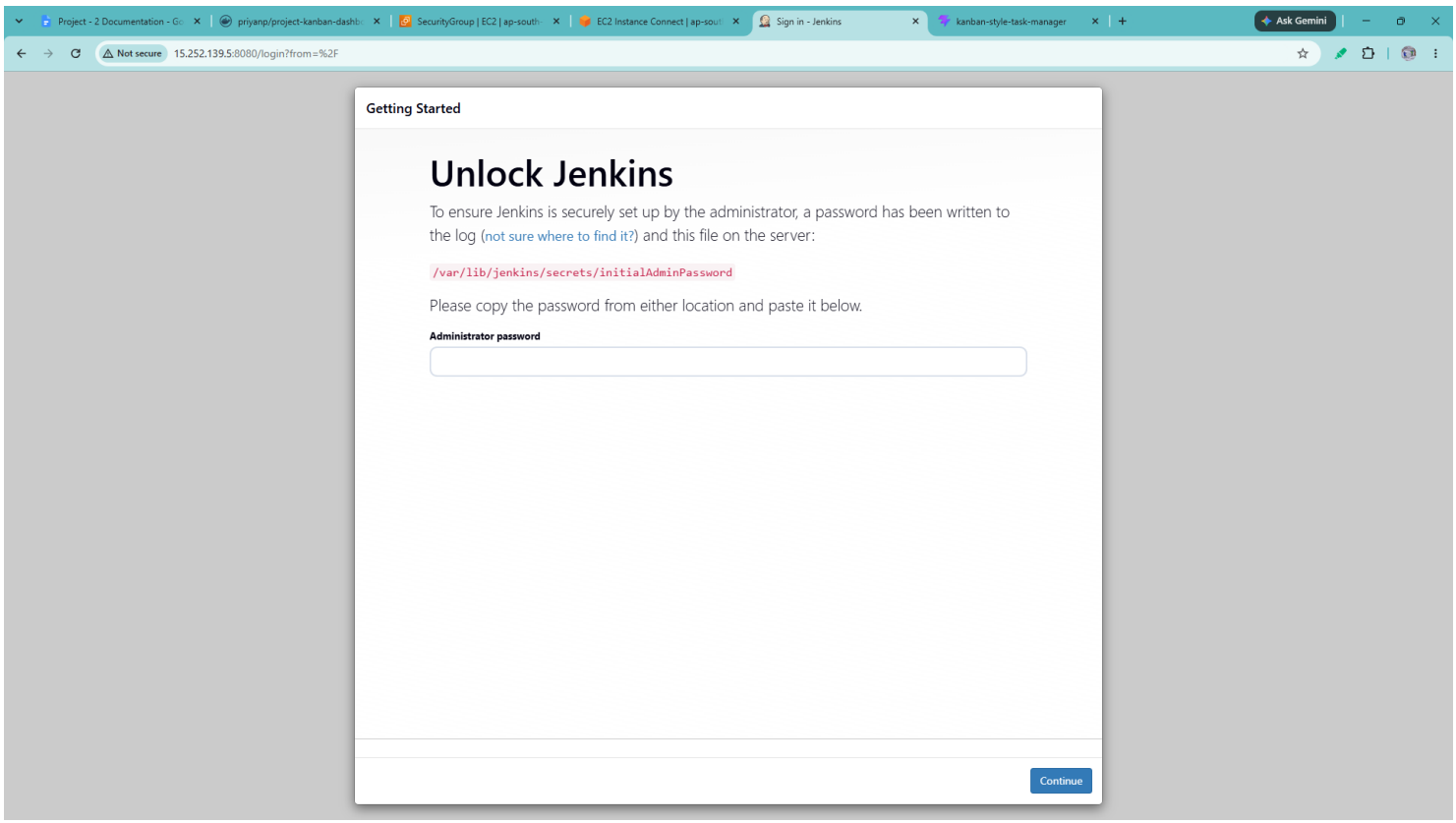
AWS infrastructure setup was completed successfully. The EC2 instance is hosting both Jenkins and the running Docker container, with access configured through the required Security Group

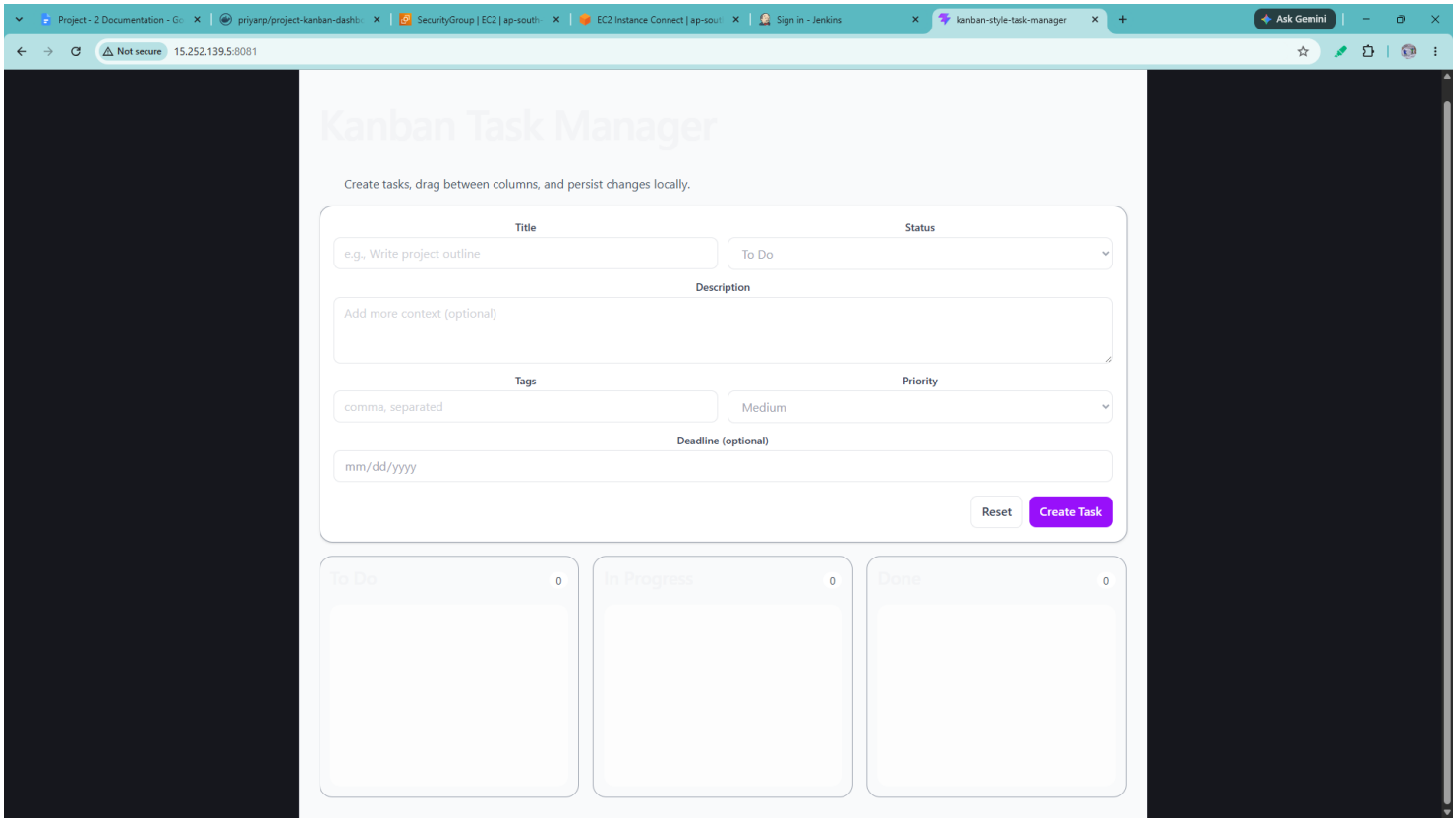
ports. The initial port conflict between Jenkins and the application was resolved by moving the application to port 8081. The infrastructure is now ready for the Jenkins CI/CD pipeline setup.






Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source Info
Description - optional Info
sgr-063e7427955270eb2
Custom TCP
TCP
8080
Custom
157.51.109.56/32
Jenkins
Delete
sgr-0331944ec4b91ddac
SSH
TCP
22
Custom
0.0.0.0/0
Delete
sgr-0aa85c9246557a13b
Custom TCP
TCP
8081
Custom
157.51.109.56/32
Kanban Application
Delete
Add rule
Cancel
Preview changes
Save rules


Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
/var/lib/jenkins/secrets/initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password
Continue


Not secure
15.252.139.5:8081
kanban-style-task-manager
Ask Gemini
Kanban Task Manager
Create tasks, drag between columns, and persist changes locally.
Title
e.g., Write project outline
Status
To Do
Description
Add more context (optional)
Tags
comma, separated
Priority
Medium
Deadline (optional)
mm/dd/yyyy
Reset
Create Task
To Do
0
In Progress
0
Done
0

Step 6 : Jenkins Configuration

Overview

With the EC2 infrastructure in place, Jenkins was configured to connect to the GitHub repository, store the required credentials securely, and run the pipeline from the Jenkinsfile committed to the repository.

Credentials Setup

Three credentials were added to the Jenkins Global credentials store — no secrets were stored in code or the Jenkinsfile:

- github-credentials — GitHub personal access token
- dockerhub-credentials — Docker Hub credentials
- aws-credentials — AWS access key

Pipeline Configuration

A pipeline job named Project-2-Pipeline was created in Jenkins. The pipeline definition was set to Pipeline script from SCM with the following configuration:

- SCM: Git
- Repository URL: https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git
- Credentials: github-credentials
- Branch: */main
- Script Path: Jenkinsfile

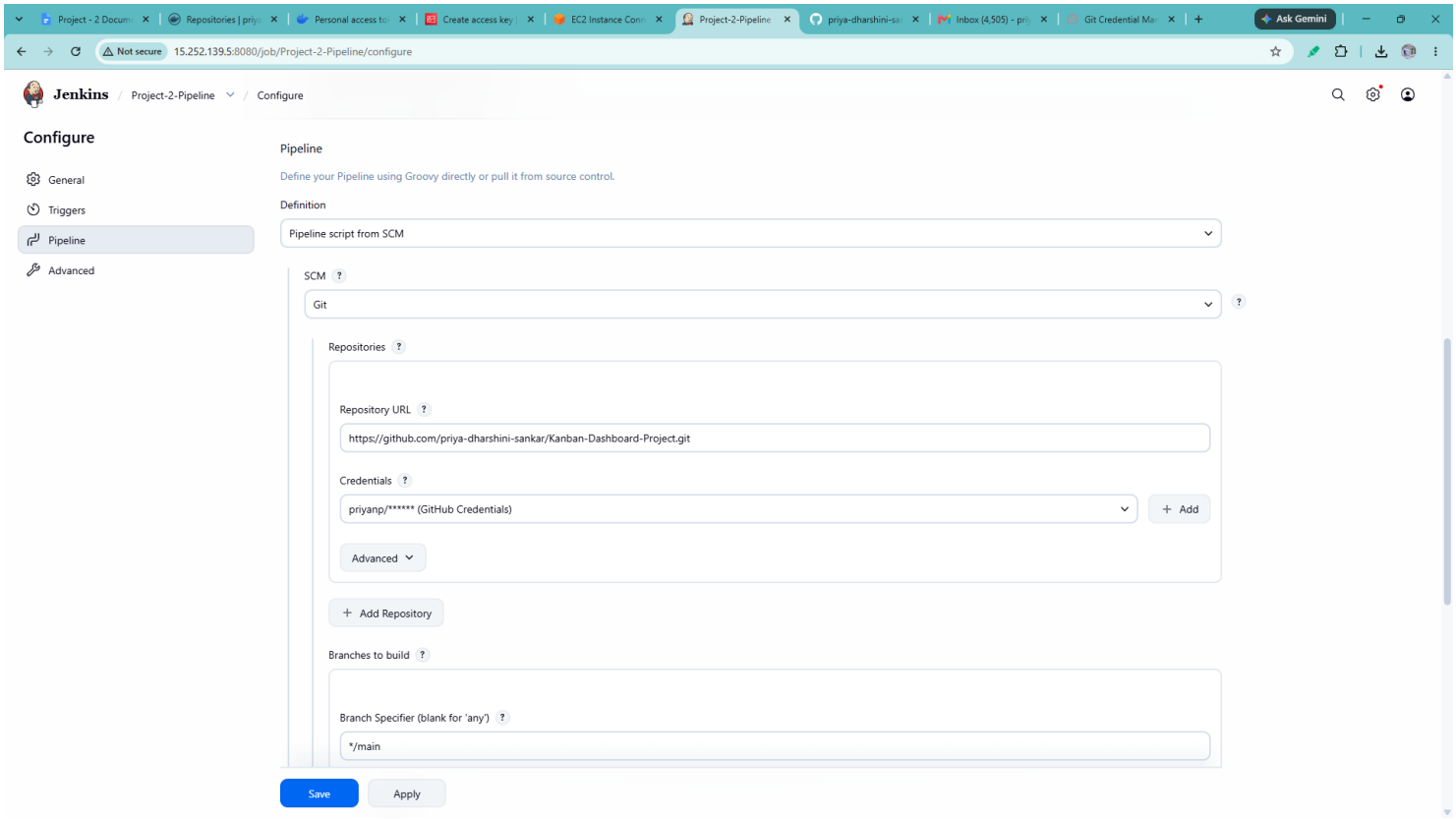
A Jenkinsfile was created in the repository with an initial Checkout Source stage and pushed to the main branch via git commit and git push.

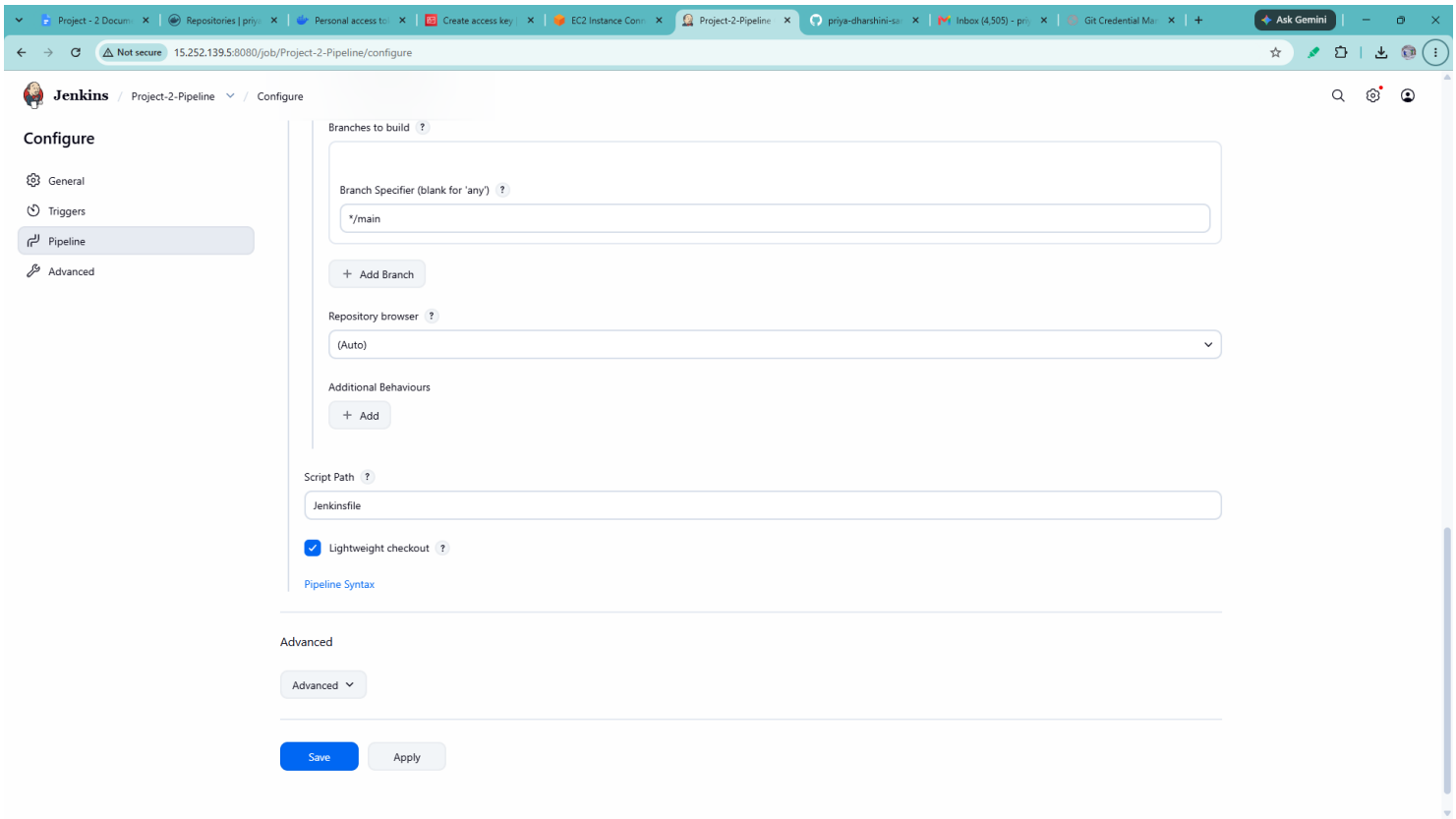
Validation

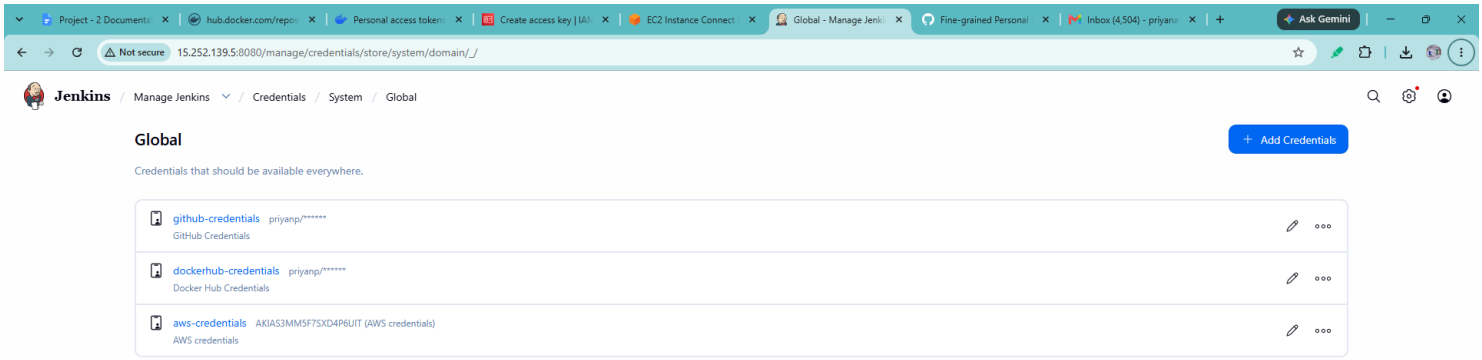
The Jenkins dashboard showed Project-2-Pipeline with last success 1 min 9 sec, last failure N/A, and last duration 17 seconds — confirming the pipeline ran successfully on the first attempt with no failures.

Result

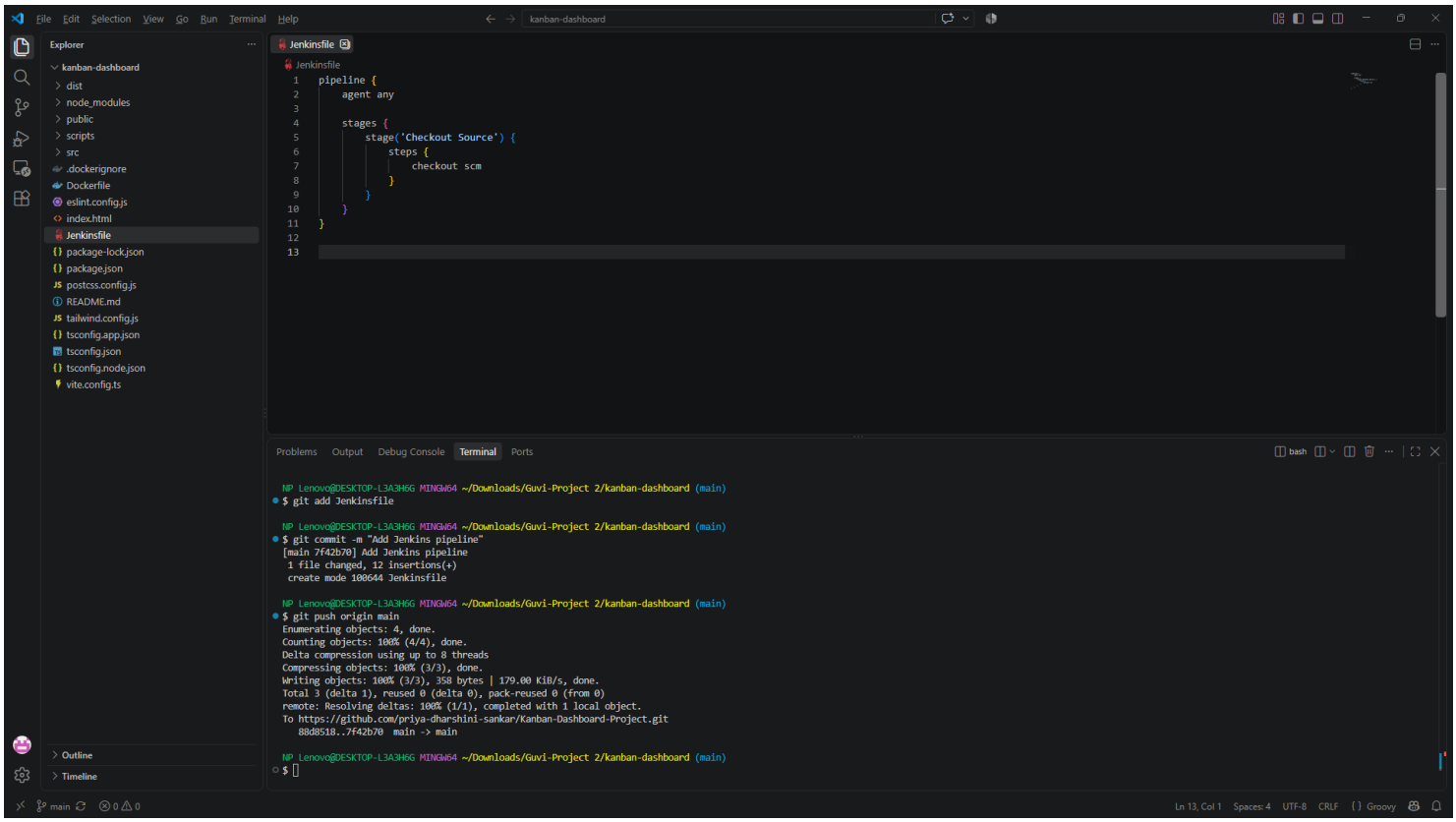
Jenkins was configured successfully. Credentials are stored securely in the Jenkins credentials store, the pipeline is connected to the GitHub repository, and the Jenkinsfile is being picked up correctly from the main branch.

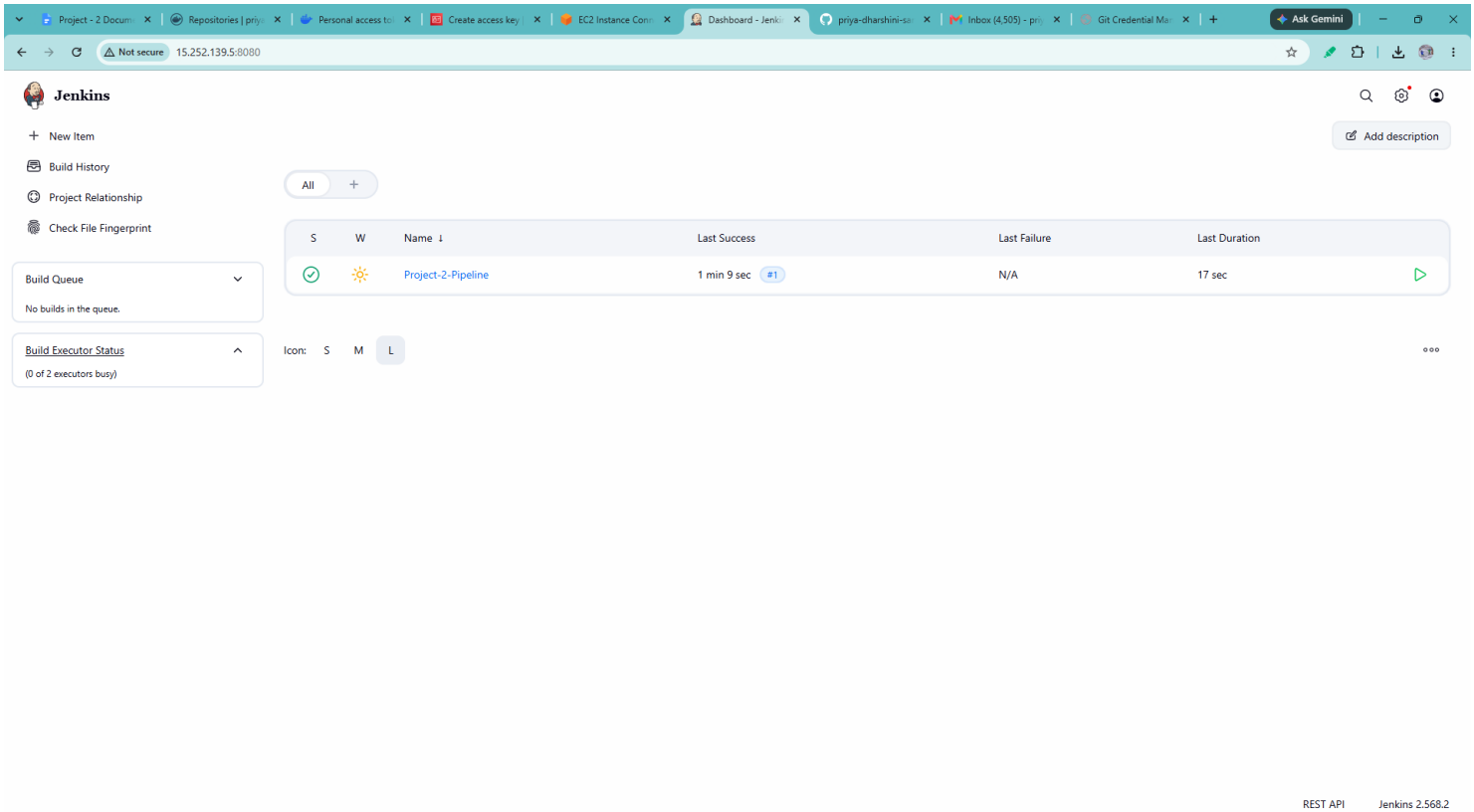

Jenkins / Project-2-Pipeline / Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git
Credentials
priyanp/****** (GitHub Credentials)
Add
Advanced
Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
Save
Apply

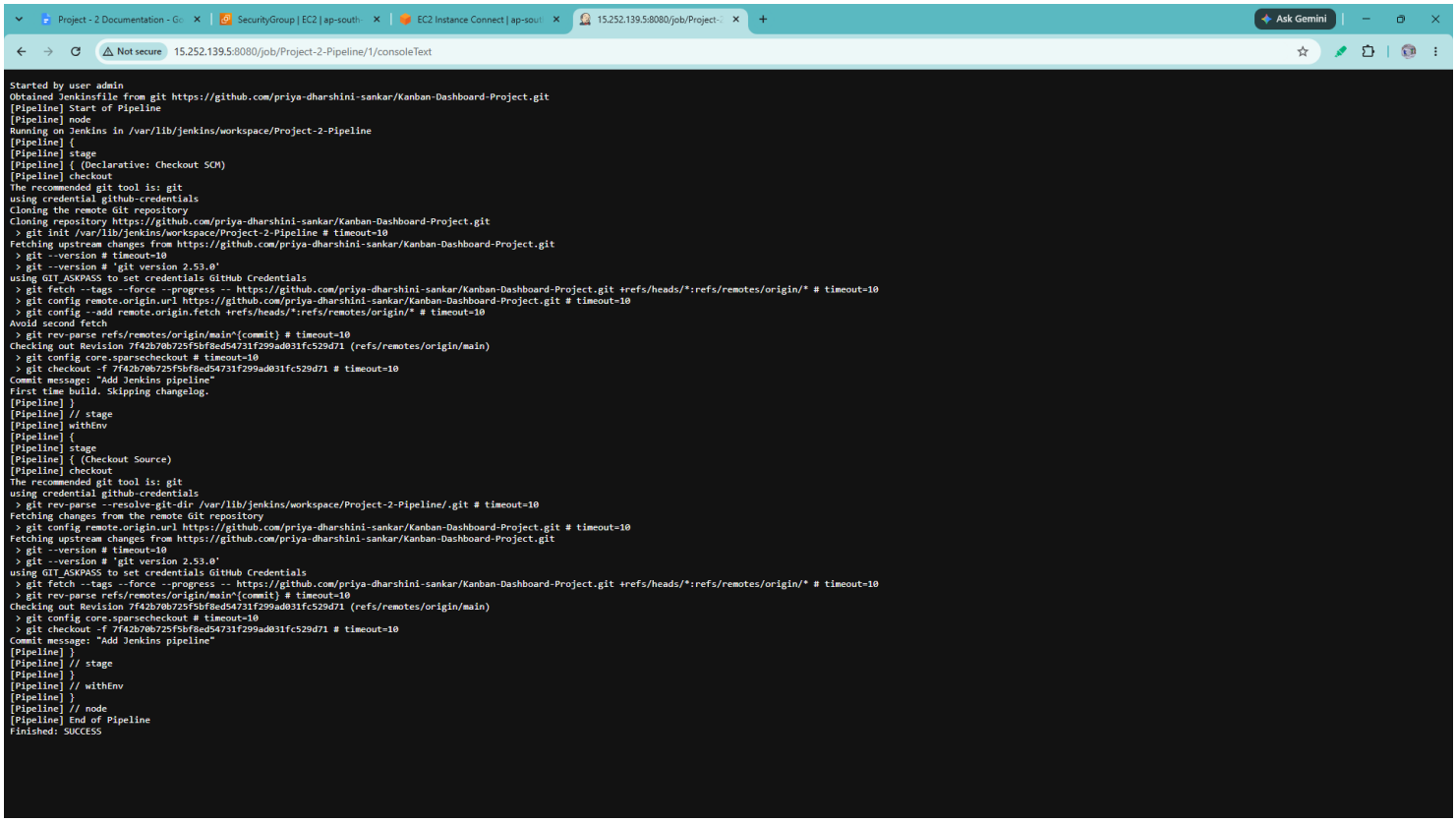

Jenkins / Project-2-Pipeline / Configure
Configure
General
Triggers
Pipeline
Advanced
Branches to build
Branch Specifier (blank for 'any')
*/main
Add Branch
Repository browser
(Auto)
Additional Behaviours
Add
Script Path
Jenkinsfile
Lightweight checkout
Pipeline Syntax
Advanced
Advanced
Save
Apply



REST API Jenkins 2.568.2




Jenkins
New Item
Build History
Project Relationship
Check File Fingerprint
Build Queue
No builds in the queue.
Build Executor Status
(0 of 2 executors busy)
Add description
All
S
W
Name ↓
Last Success
Last Failure
Last Duration
Project-2-Pipeline
1 min 9 sec
#1
N/A
17 sec
Icon: S M L
REST API
Jenkins 2.568.2


Started by user admin
Obtained Jenkinsfile from git https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Project-2-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git
using credential github-credentials
Cloning the remote Git repository
Cloning repository https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git
> git init /var/lib/jenkins/workspace/Project-2-Pipeline # timeout=10
Fetching upstream changes from https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git
> git --version # timeout=10
> git --version # 'git version 2.53.0'
using GIT_ASKPASS to set credentials GitHub Credentials
> git fetch --tags --force --progress -- https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 7f42b70b725f5bf8ed54731f299ad031fc529d71 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 7f42b70b725f5bf8ed54731f299ad031fc529d71 # timeout=10
Commit message: "Add Jenkins pipeline"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Source)
[Pipeline] checkout
The recommended git tool is: git
using credential github-credentials
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Project-2-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git # timeout=10
Fetching upstream changes from https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git
> git --version # timeout=10
> git --version # 'git version 2.53.0'
using GIT_ASKPASS to set credentials GitHub Credentials
> git fetch --tags --force --progress -- https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 7f42b70b725f5bf8ed54731f299ad031fc529d71 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 7f42b70b725f5bf8ed54731f299ad031fc529d71 # timeout=10
Commit message: "Add Jenkins pipeline"
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

Step 7 : CI/CD Pipeline (Jenkinsfile)

Overview

The Jenkinsfile defines the automated CI/CD pipeline that Jenkins executes on every code push. Once the pipeline is configured and triggered, Jenkins automates the deployment process from source checkout through production deployment and health verification.

Pipeline Configuration

The pipeline is configured with the following environment variables:

- Image: priyanp/project-kanban-dashboard
- Container: project-kanban-dashboard
- Port: 8081

Pipeline Stages

The pipeline consists of six stages executed in sequence

Checkout Source — Jenkins pulls the latest code from GitHub and captures the short Git commit SHA for image tagging.

Docker Build — A new Docker image is built and tagged with both the build number and Git SHA, for example build-3-98f7a77. This ensures every image is unique and fully traceable to its source commit.

Registry Login + Push — Jenkins retrieves Docker Hub credentials securely from its credentials store, logs in, pushes the image, and immediately logs out.

Deploy to EC2 — The current running container's image is saved for rollback. The new image is pulled from Docker Hub, the old container is stopped and removed, and a new container is started with the updated image.

Health Check — After a 10-second startup wait, the container's health status is verified. If the status is not healthy, the pipeline fails and triggers automatic rollback.

Rollback Strategy — If any stage fails, the post-failure block automatically stops the broken container and restarts the previously running image, ensuring the application remains available.

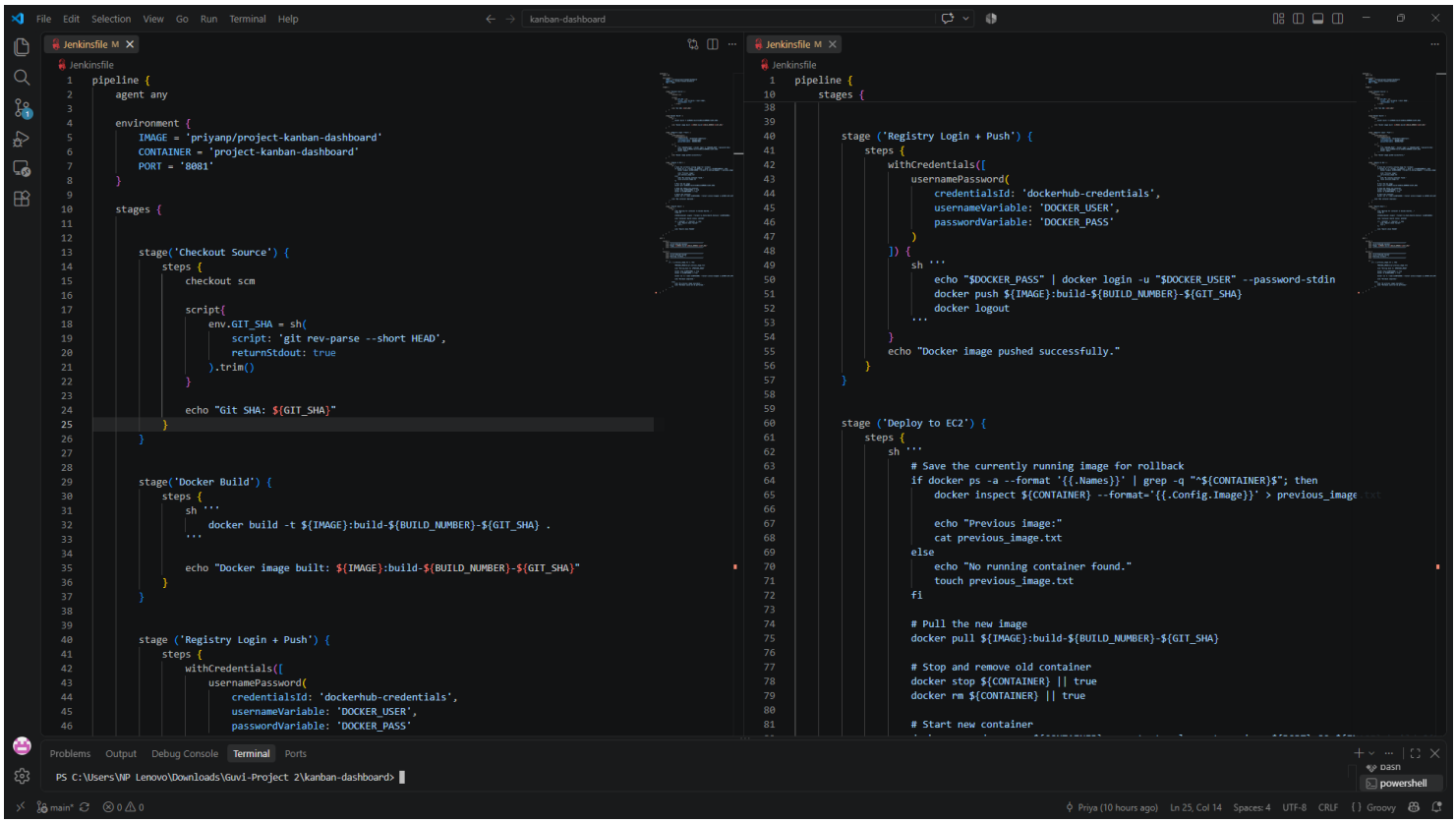
Evidence

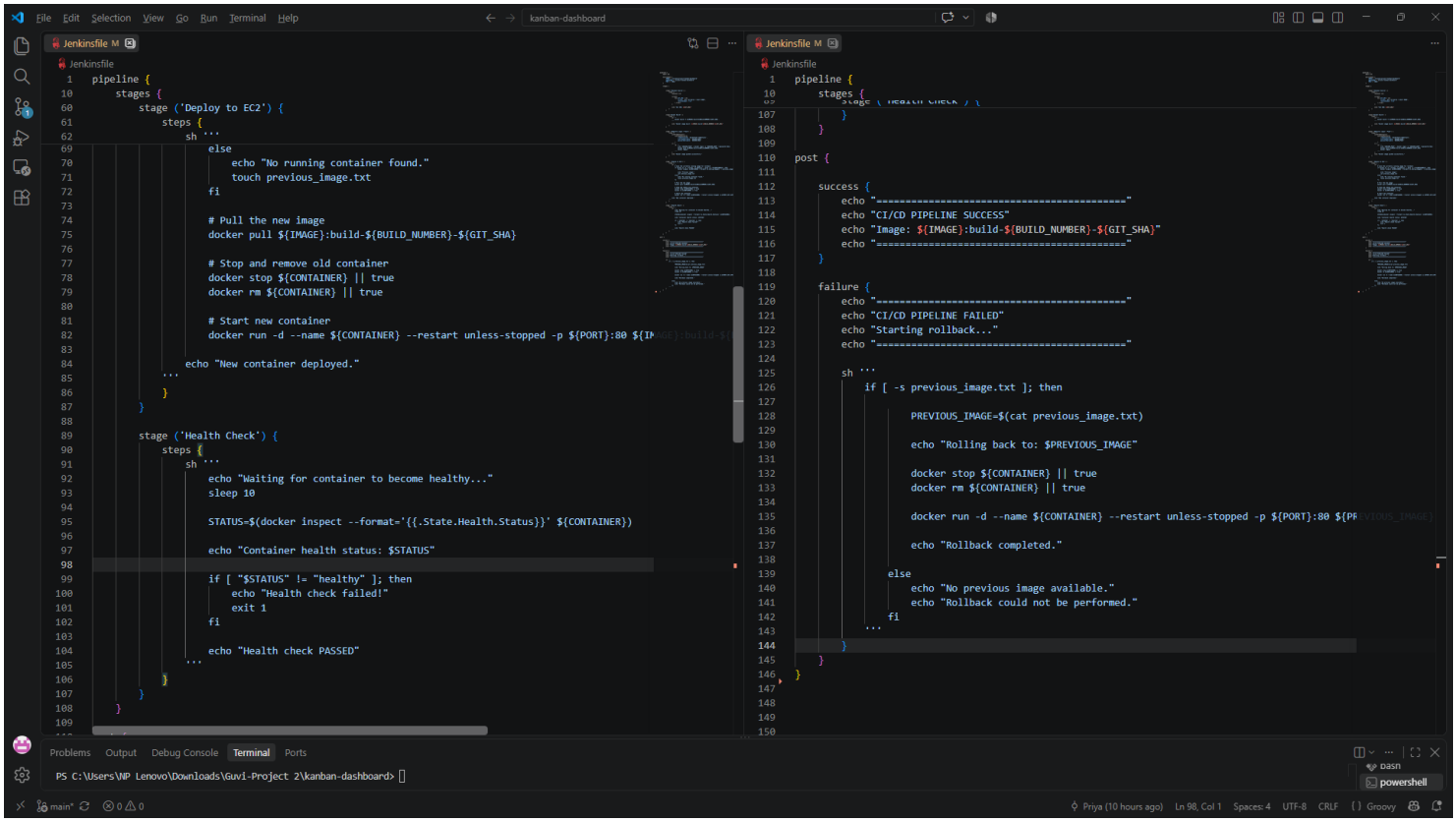
Jenkins pipeline showing all stages passing :

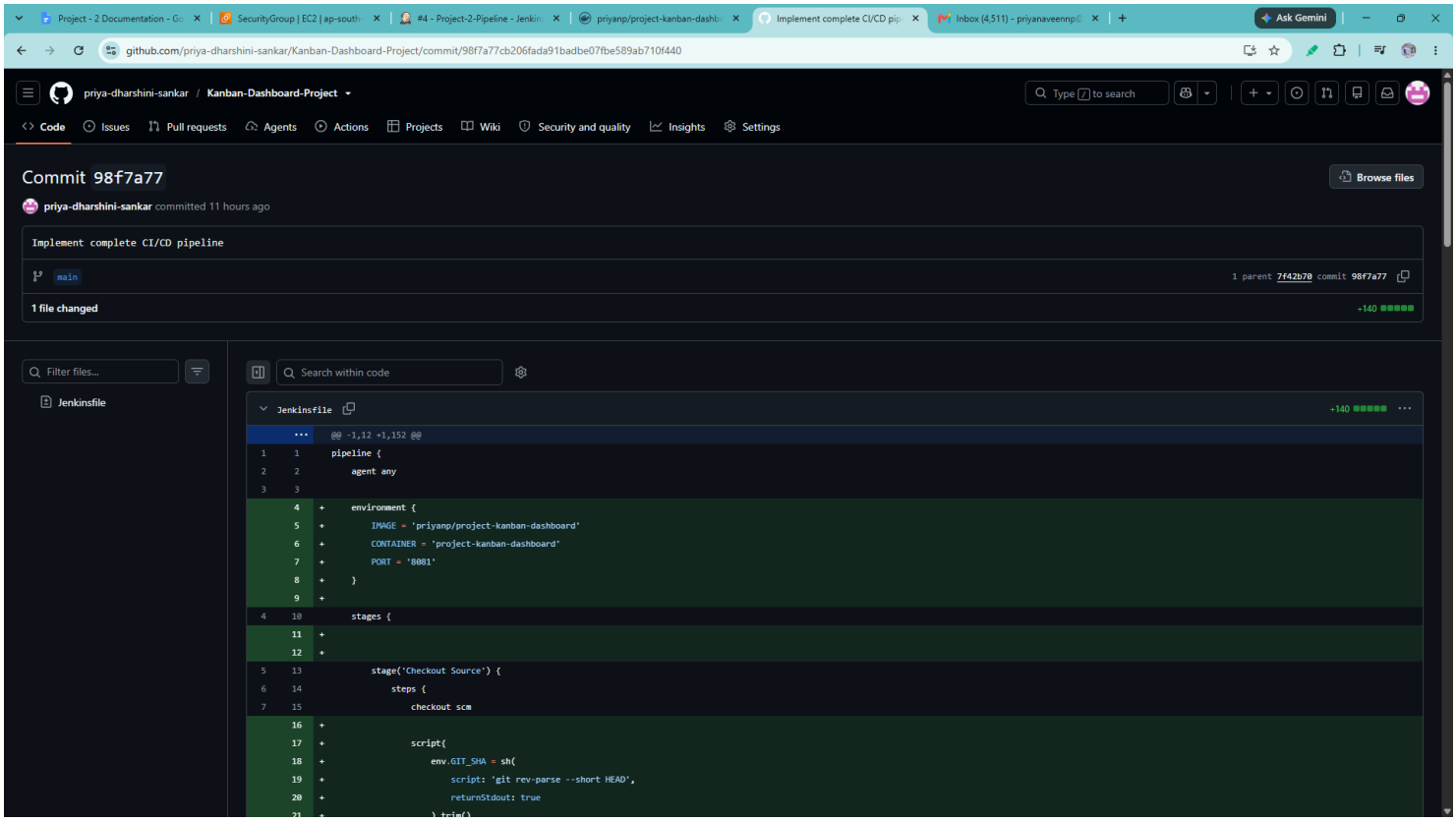
Build #3 — priyanp/project-kanban-dashboard:build-3-98f7a77

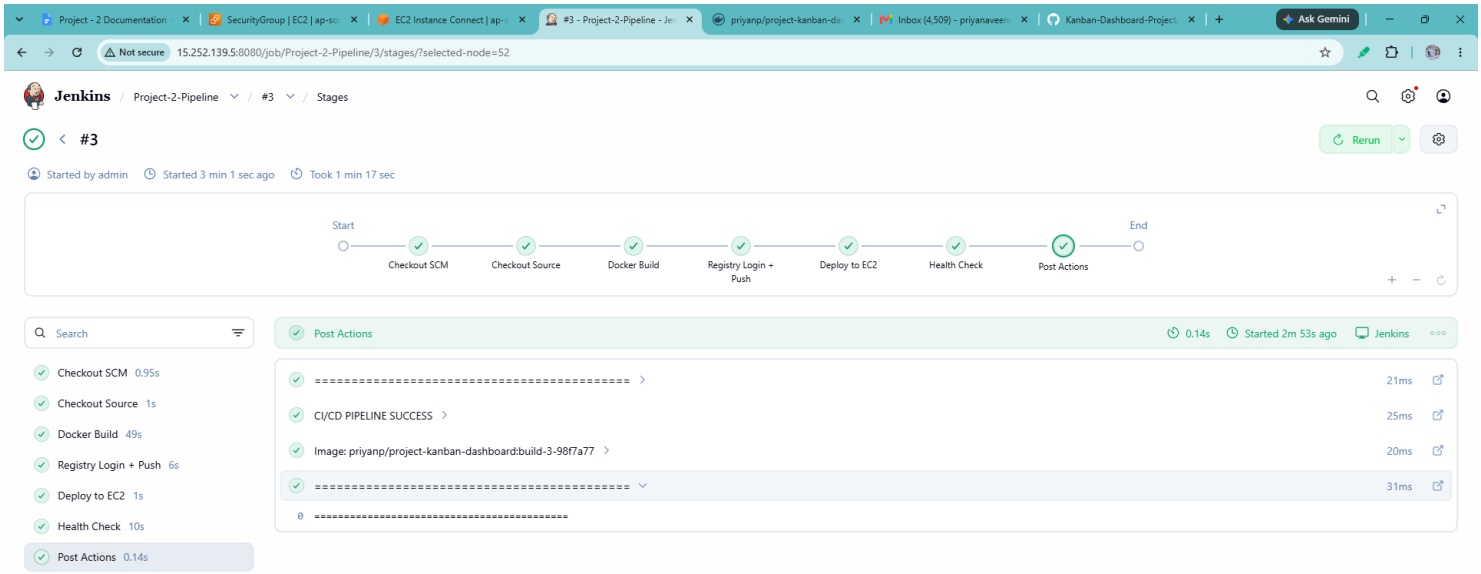
Result

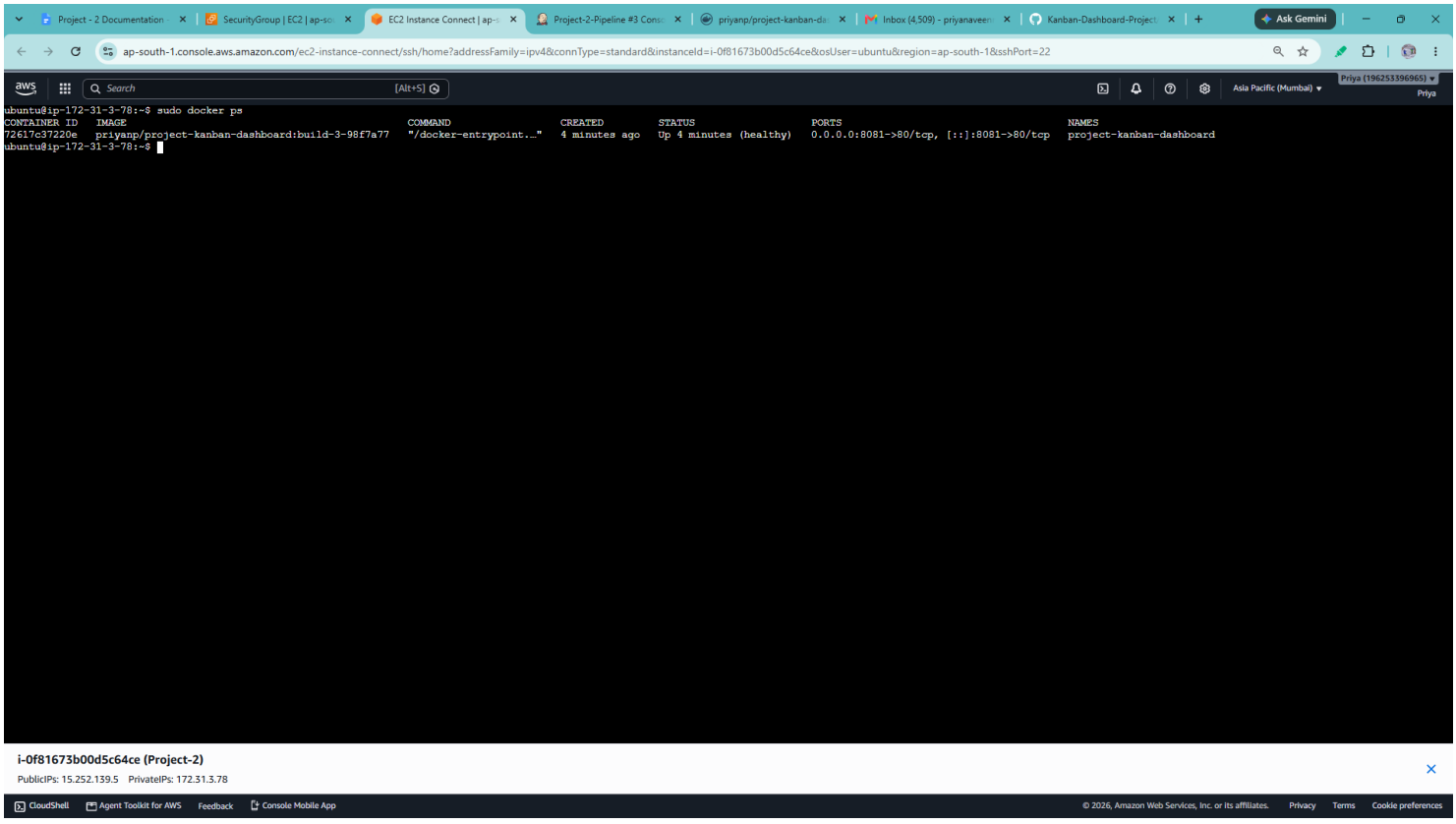
The CI/CD pipeline executed successfully end-to-end. All six stages passed, and the application was automatically built, pushed to Docker Hub, deployed to AWS EC2, and health-checked through Jenkins without any manual intervention.




File Edit Selection View Go Run Terminal Help
kanban-dashboard
Jenkinsfile M
Jenkinsfile
pipeline {
stages {
stage ('Deploy to EC2') {
steps {
sh '''
else
echo "No running container found."
touch previous_image.txt
fi
Pull the new image
docker pull ${IMAGE}:build-${BUILD_NUMBER}-${GIT_SHA}
Stop and remove old container
docker stop ${CONTAINER} || true
docker rm ${CONTAINER} || true
Start new container
docker run -d --name ${CONTAINER} --restart unless-stopped -p ${PORT}:80 ${IM
echo "New container deployed."
'''
}
}
stage ('Health Check') {
steps {
sh '''
echo "Waiting for container to become healthy..."
sleep 10
STATUS=$(docker inspect --format='{{.State.Health.Status}}' ${CONTAINER})
echo "Container health status: $STATUS"
if ["$STATUS" != "healthy"]; then
echo "Health check failed!"
exit 1
fi
echo "Health check PASSED"
'''
}
}
}
post {
success {
echo "==="
echo "CI/CD PIPELINE SUCCESS"
echo "Image: ${IMAGE}:build-${BUILD_NUMBER}-${GIT_SHA}"
echo "==="
}
failure {
echo "==="
echo "CI/CD PIPELINE FAILED"
echo "Starting rollback..."
echo "==="
sh '''
if [-s previous_image.txt]; then
PREVIOUS_IMAGE=$(cat previous_image.txt)
echo "Rolling back to: $PREVIOUS_IMAGE"
docker stop ${CONTAINER} || true
docker rm ${CONTAINER} || true
docker run -d --name ${CONTAINER} --restart unless-stopped -p ${PORT}:80 ${PR
echo "Rollback completed."
else
echo "No previous image available."
echo "Rollback could not be performed."
fi
'''
}
}
Problems Output Debug Console Terminal Ports
PS C:\Users\NP Lenovo\Downloads\Guvi-Project 2\kanban-dashboard>
powershell
main*
Priya (10 hours ago) Ln 98, Col 1 Spaces: 4 UTF-8 CRLF Groovy


github.com/priya-dharshini-sankar/Kanban-Dashboard-Project/commit/98f7a77cb206fada91badbe07fbe589ab710f440
priya-dharshini-sankar / Kanban-Dashboard-Project
Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings
Commit 98f7a77
Browse files
priya-dharshini-sankar committed 11 hours ago
Implement complete CI/CD pipeline
main
1 parent 7f42b70 commit 98f7a77
1 file changed
+140
Filter files...
Jenkinsfile
Search within code
Jenkinsfile
+140
@@ -1,12 +1,152 @@
pipeline {
agent any
+ environment {
+ IMAGE = 'priyanp/project-kanban-dashboard'
+ CONTAINER = 'project-kanban-dashboard'
+ PORT = '8081'
+ }
+
stages {
+
+
stage('Checkout Source') {
steps {
checkout scm
+
+ script{
+ env.GIT_SHA = sh(
+ script: 'git rev-parse --short HEAD',
+ returnStdout: true


Jenkins / Project-2-Pipeline / #3 / Stages
#3
Rerun
Started by admin
Started 3 min 1 sec ago
Took 1 min 17 sec
Start
Checkout SCM
Checkout Source
Docker Build
Registry Login + Push
Deploy to EC2
Health Check
Post Actions
End
Search
Checkout SCM 0.95s
Checkout Source 1s
Docker Build 49s
Registry Login + Push 6s
Deploy to EC2 1s
Health Check 10s
Post Actions 0.14s
Post Actions
0.14s
Started 2m 53s ago
Jenkins
21ms
CI/CD PIPELINE SUCCESS
25ms
Image: priyanp/project-kanban-dashboard:build-3-98f7a77
20ms
31ms
Jenkins 2.568.2


ubuntu@ip-172-31-3-78:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
72617c37220e priyanp/project-kanban-dashboard:build-3-98f7a77 "/docker-entrypoint.…" 4 minutes ago Up 4 minutes (healthy) 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp project-kanban-dashboard
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78

Step 8 : Github Webhook

Overview

With the pipeline working manually, the next step was to configure a GitHub webhook so that every code push automatically triggers the Jenkins pipeline without any manual intervention.

Webhook Setup

The Jenkins webhook URL http://15.252.139.5:8080/github-webhook/ was added to the Kanban-Dashboard-Project repository under Settings → Webhooks. The webhook was configured to trigger on push events.

Trigger Test

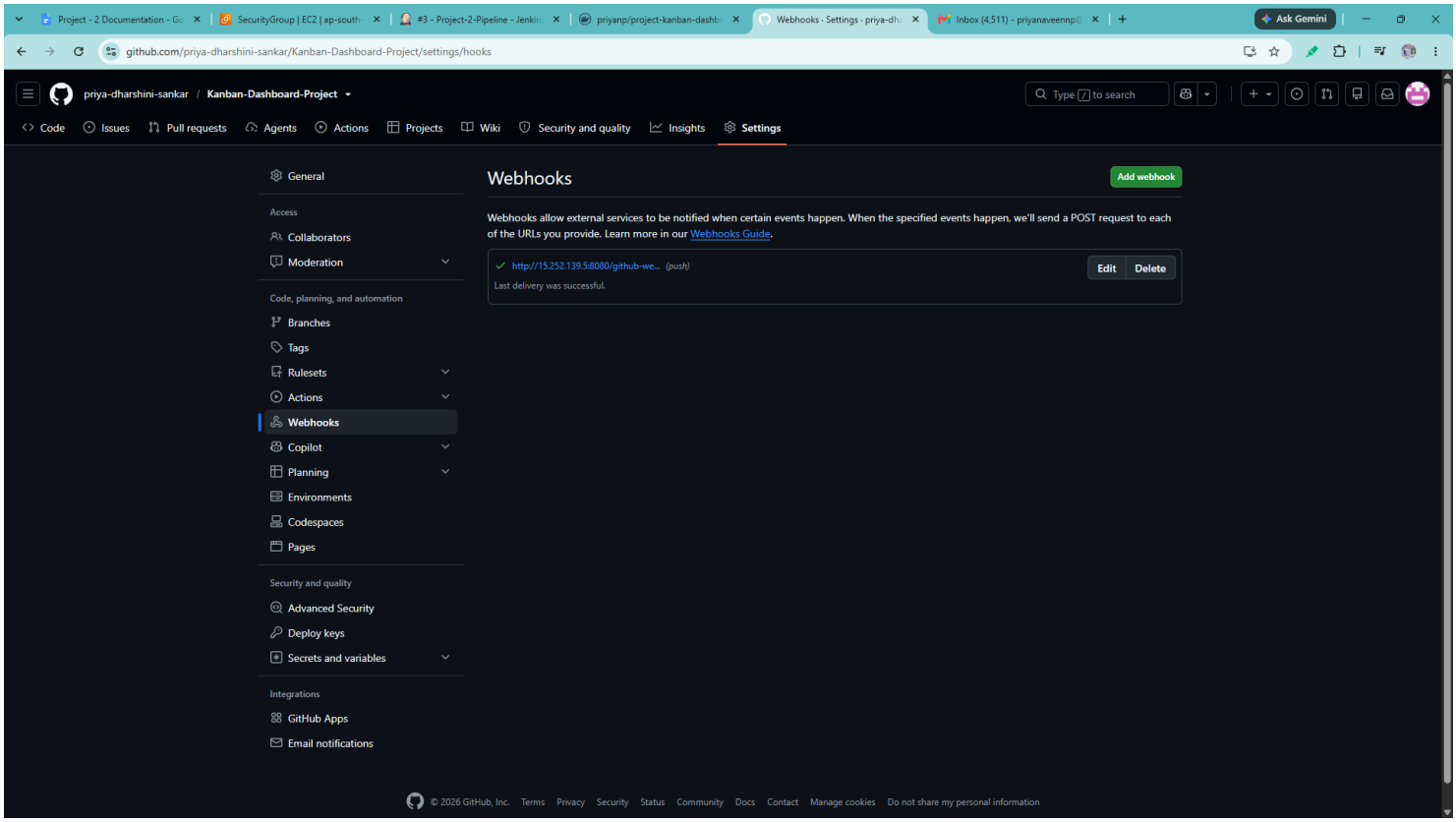
A test file named webhook-test.txt was created directly in the GitHub repository with the content "Testing GitHub webhook → Jenkins automatic trigger" and committed to the main branch with the message "Add webhook-test.txt file" (commit e89ead4). This push was used to verify that Jenkins would trigger automatically.

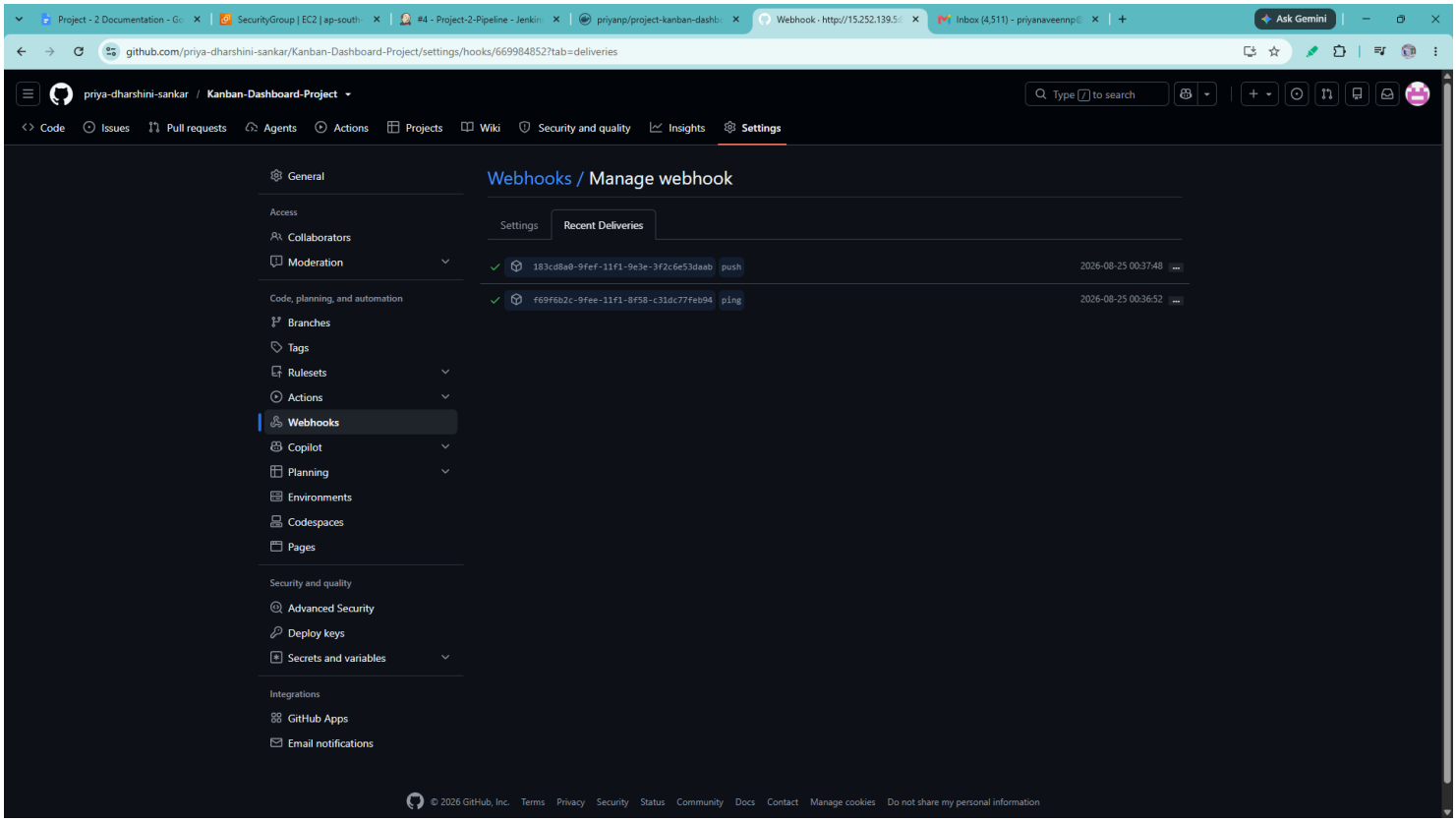
Validation

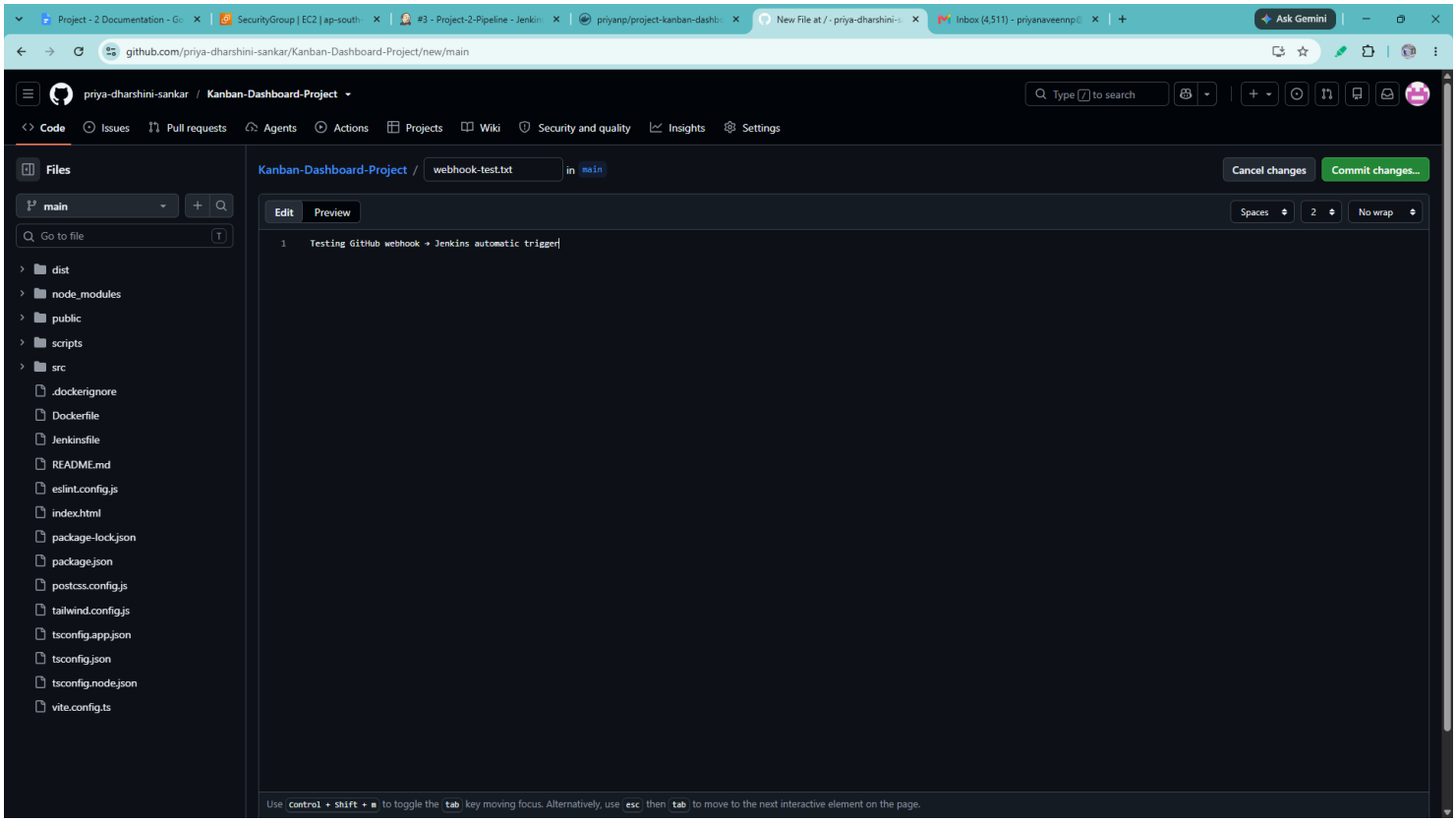
The GitHub webhook deliveries page confirmed two successful deliveries — a ping event (webhook connected) and a push event triggered by the commit. Jenkins Build #4 started automatically, triggered by GitHub push by priya-dharshini-sankar, with no manual click. All stages passed in 34 seconds and the Post Actions confirmed CI/CD PIPELINE SUCCESS with the deployed image priyanp/project-kanban-dashboard:build-4-e89ead4.

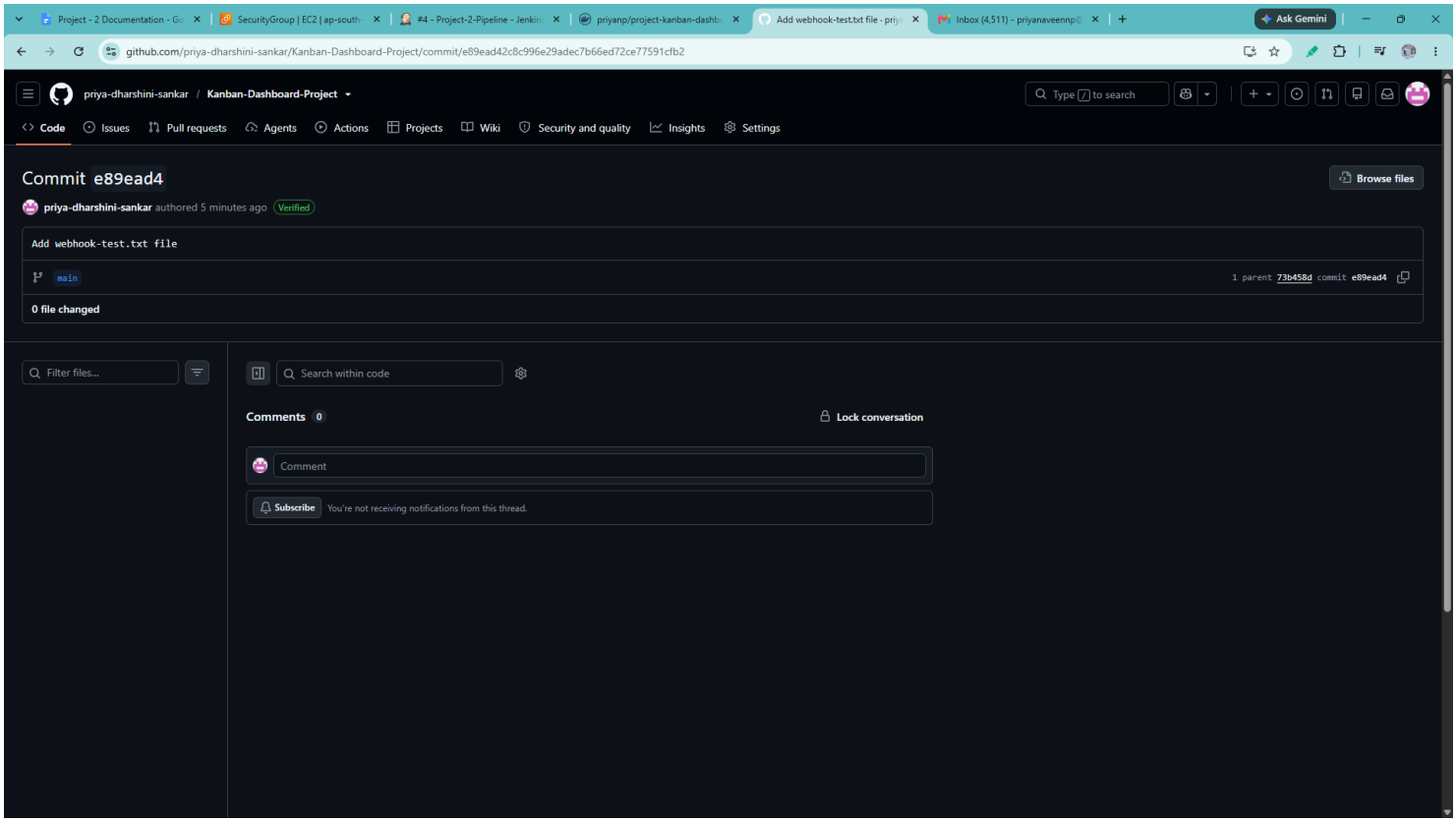
Result

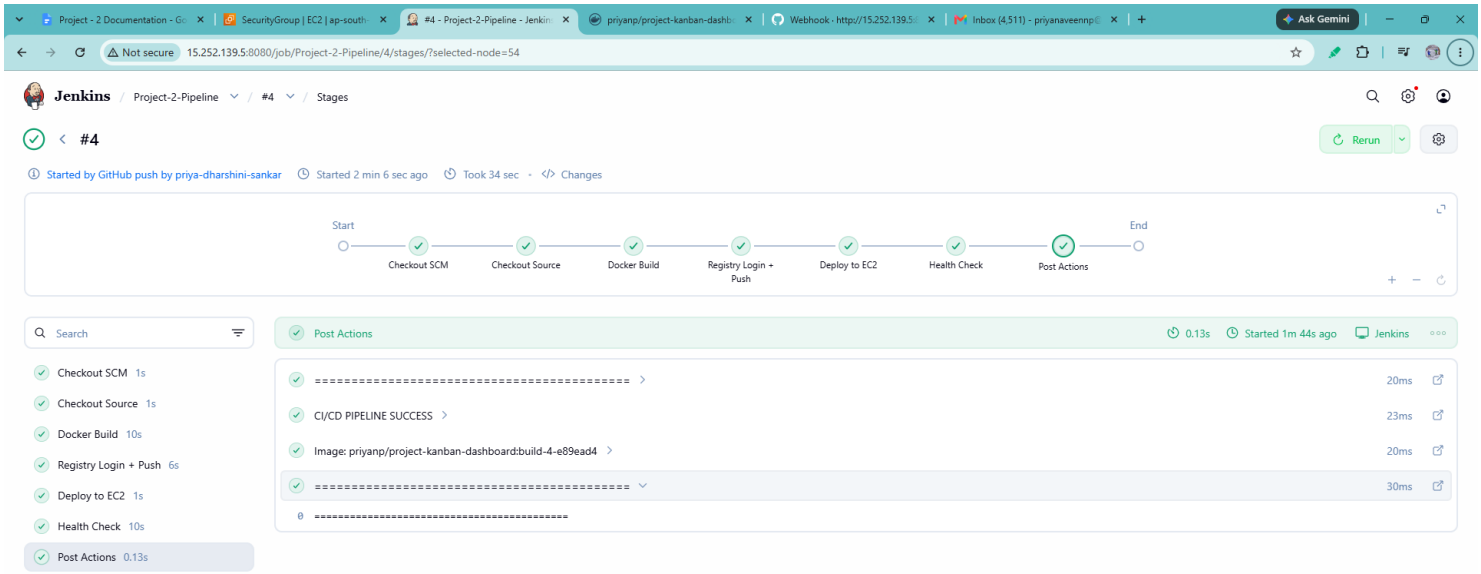
The GitHub webhook was configured successfully. A code push to the main branch automatically triggered the Jenkins pipeline and deployed the updated image without any manual intervention.


github.com/priya-dharshini-sankar/Kanban-Dashboard-Project/settings/hooks
priya-dharshini-sankar / Kanban-Dashboard-Project
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
General
Access
Collaborators
Moderation
Code, planning, and automation
Branches
Tags
Rulesets
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages
Security and quality
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Email notifications
Webhooks
Add webhook
Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our Webhooks Guide.
http://15.252.139.5:8080/github-we... (push)
Last delivery was successful.
Edit
Delete
© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information


github.com/priya-dharshini-sankar/Kanban-Dashboard-Project/settings/hooks/669984852?tab=deliveries
priya-dharshini-sankar / Kanban-Dashboard-Project
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
General
Access
Collaborators
Moderation
Code, planning, and automation
Branches
Tags
Rulesets
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages
Security and quality
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Email notifications
Webhooks / Manage webhook
Settings
Recent Deliveries
183cd8a0-9fef-11f1-9e3e-3f2c6e53daab
push
2026-08-25 00:37:48
f69f6b2c-9fee-11f1-8f58-c31dc77feb94
ping
2026-08-25 00:36:52
© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information


github.com/priya-dharshini-sankar/Kanban-Dashboard-Project/new/main
priya-dharshini-sankar / Kanban-Dashboard-Project
Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings
Files
main
Go to file
dist
node_modules
public
scripts
src
.dockerignore
Dockerfile
Jenkinsfile
README.md
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
tailwind.config.js
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Kanban-Dashboard-Project / webhook-test.txt in main
Cancel changes
Commit changes...
Edit Preview
Spaces 2 No wrap
1 Testing GitHub webhook → Jenkins automatic trigger
Use Control + Shift + m to toggle the tab key moving focus. Alternatively, use esc then tab to move to the next interactive element on the page.


github.com/priya-dharshini-sankar/Kanban-Dashboard-Project/commit/e89ead42c8c996e29adec7b66ed72ce77591cfb2
priya-dharshini-sankar / Kanban-Dashboard-Project
Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings
Commit e89ead4
Browse files
priya-dharshini-sankar authored 5 minutes ago Verified
Add webhook-test.txt file
main
1 parent 73b458d commit e89ead4
0 file changed
Filter files...
Search within code
Comments 0
Lock conversation
Comment
Subscribe
You're not receiving notifications from this thread.



Jenkins 2.568.2

Step 9 : Health Check & Validation

Overview

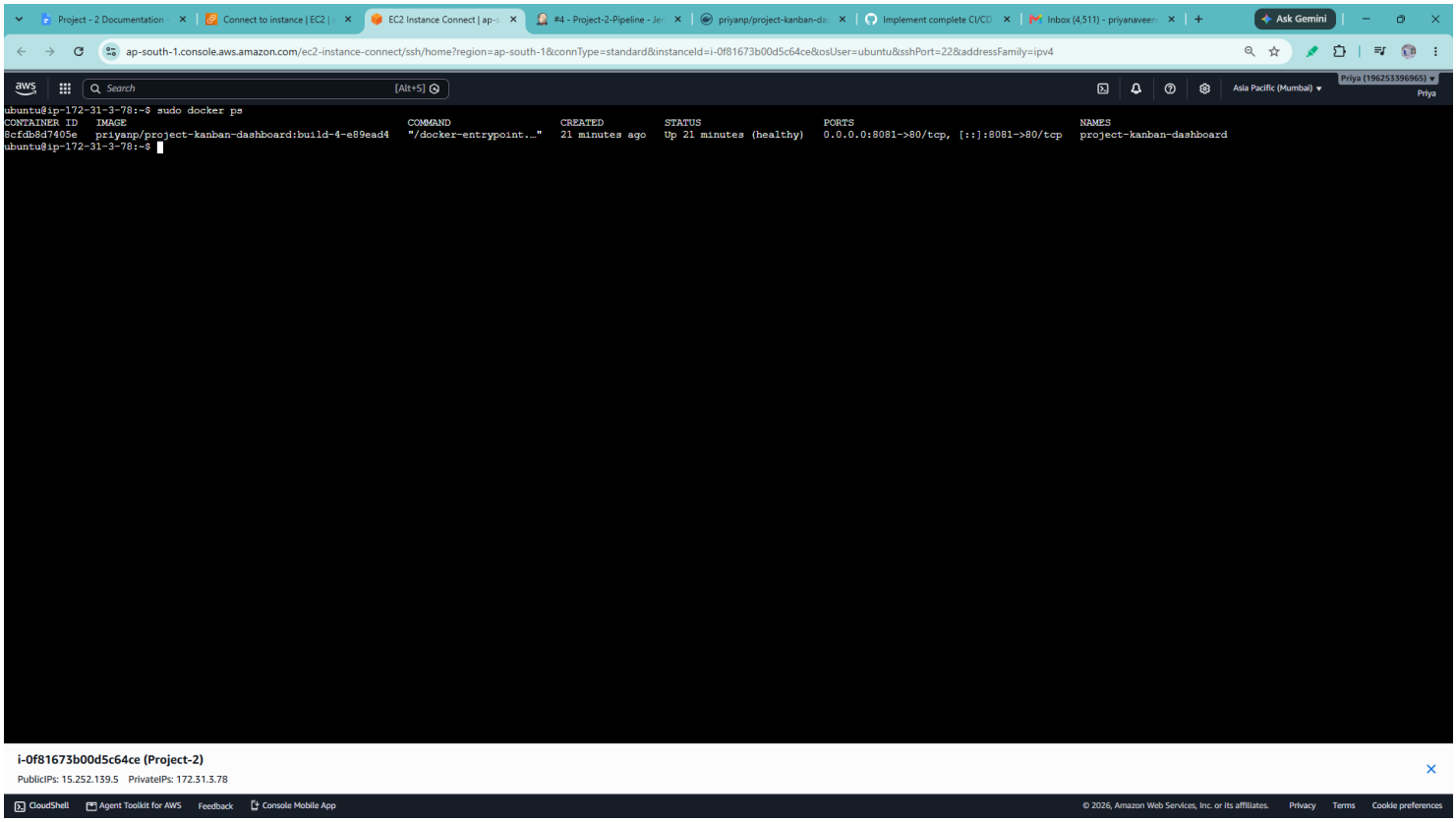
With the pipeline running end-to-end, the final step was to validate the deployment by confirming the container is running, the health check status is passing, and the application is serving a valid response.

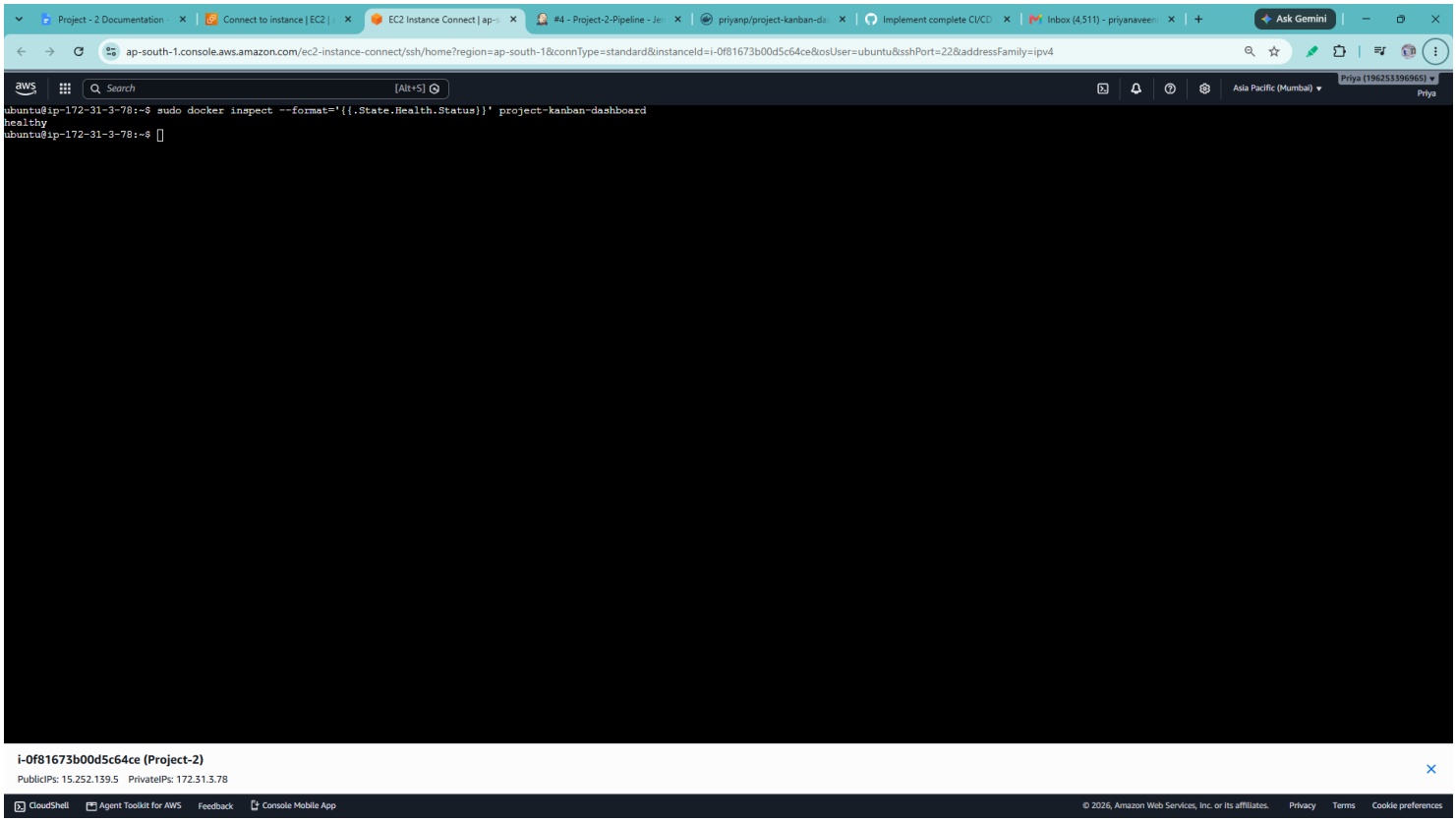
Validation

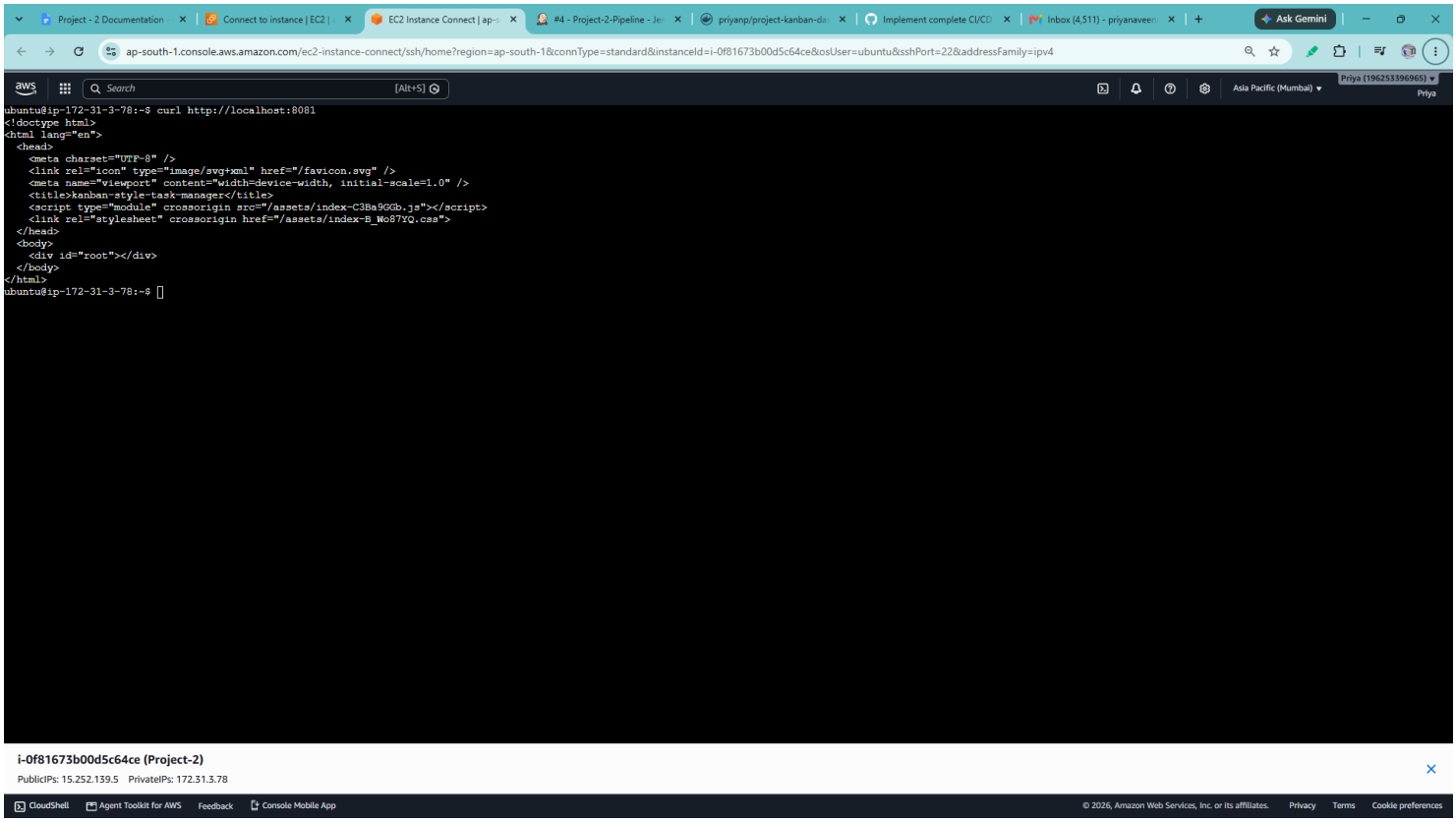
- sudo docker ps confirmed the container priyanp/project-kanban-dashboard:build-4-e89ead4 is running with status Up 21 minutes (healthy) and ports mapped as 0.0.0.0:8081->80/tcp
- sudo docker inspect --format='{{.State.Health.Status}}' project-kanban-dashboard returned healthy
- curl http://localhost:8081 returned the full HTML response of the Kanban application, confirming the app is serving correctly at the endpoint

Result

All health checks passed. The container is running, the Docker HEALTHCHECK status is healthy, and the application endpoint is returning a valid response. Deployment is confirmed successful.


ubuntu@ip-172-31-3-78:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
8cfdb8d7405e priyanp/project-kanban-dashboard:build-4-e89ead4 "/docker-entrypoint.…" 21 minutes ago Up 21 minutes (healthy) 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp project-kanban-dashboard
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78


ubuntu@ip-172-31-3-78:~$ sudo docker inspect --format='{{.State.Health.Status}}' project-kanban-dashboard
healthy
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78


Project - 2 Documentation
Connect to instance | EC2 |
EC2 Instance Connect | ap-
#4 - Project-2-Pipeline - Je
priyanp/project-kanban-da
Implement complete CI/CD
Inbox (4,511) - priyanaveen
Ask Gemini
ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&sshPort=22&addressFamily=ipv4
aws
Search
[Alt+S]
Asia Pacific (Mumbai)
Priya (196253396965)
Priya
ubuntu@ip-172-31-3-78:~$ curl http://localhost:8081
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>kanban-style-task-manager</title>
<script type="module" crossorigin src="/assets/index-C3Ba9GGb.js"></script>
<link rel="stylesheet" crossorigin href="/assets/index-B_Wo87YQ.css">
</head>
<body>

</body>
</html>
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Step 10 : Logging & Monitoring

Overview

The Jenkins build history and container logs were checked to confirm that both a passing and a failing build can be identified from the logs.

Build History

The Jenkins Build History shows four builds for Project-2-Pipeline:

- Build #1 — success
- Build #2 — failed (Docker API permission denied error)
- Build #3 — success
- Build #4 — success

Container Logs

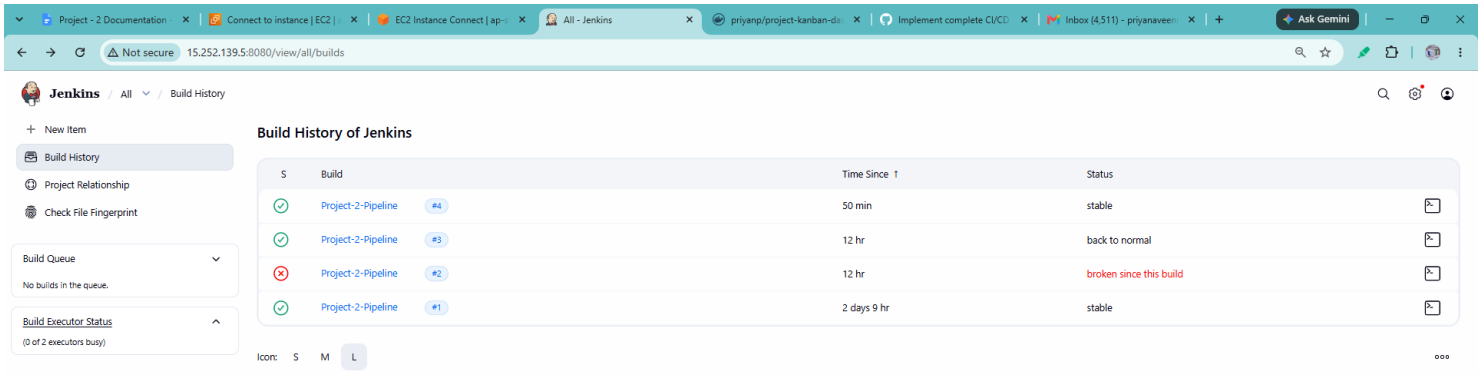
Running sudo docker logs project-kanban-dashboard on the EC2 instance showed Nginx running and continuous HTTP 200 responses, confirming the application is live and the health check is working.

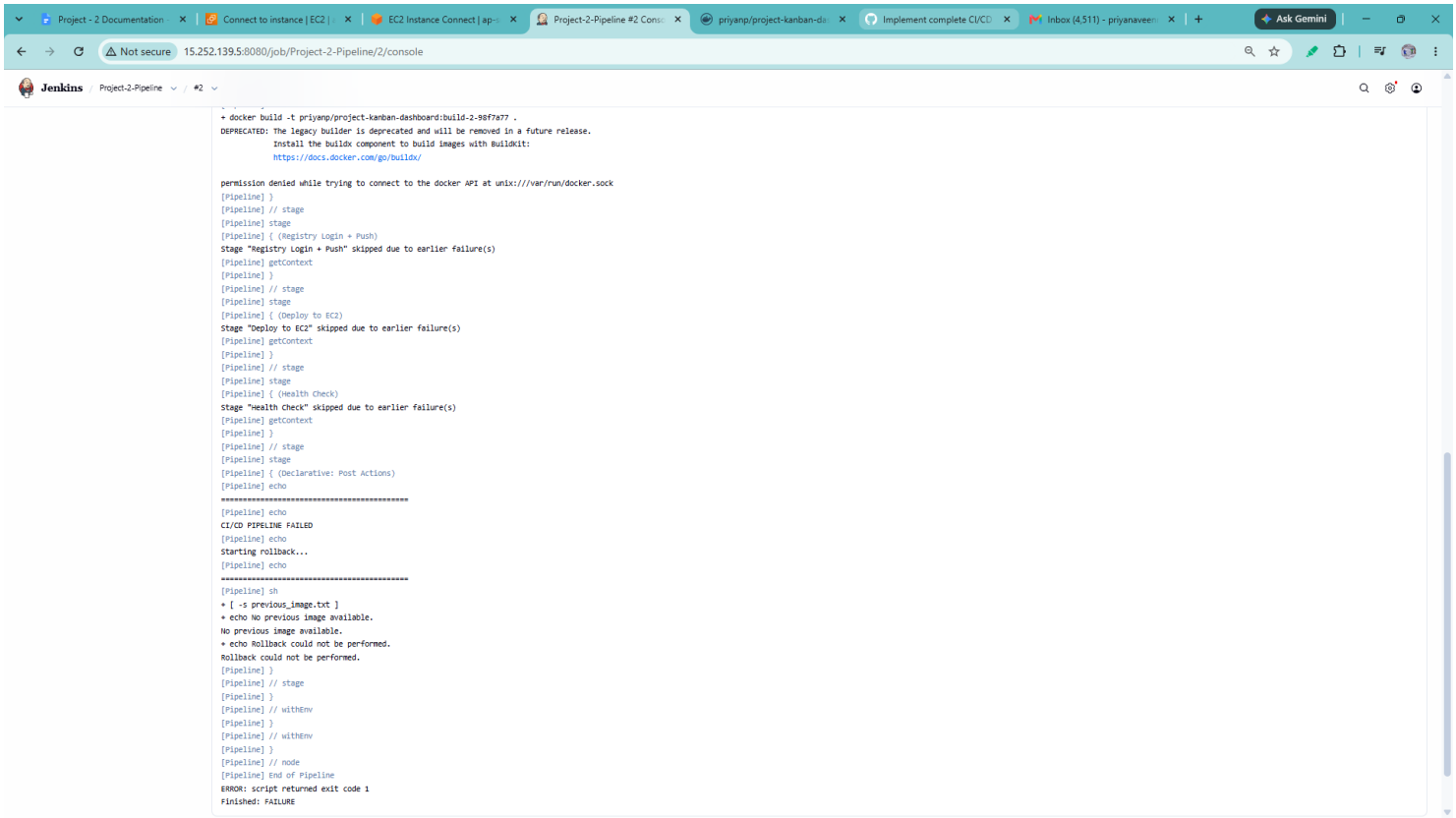
Validation

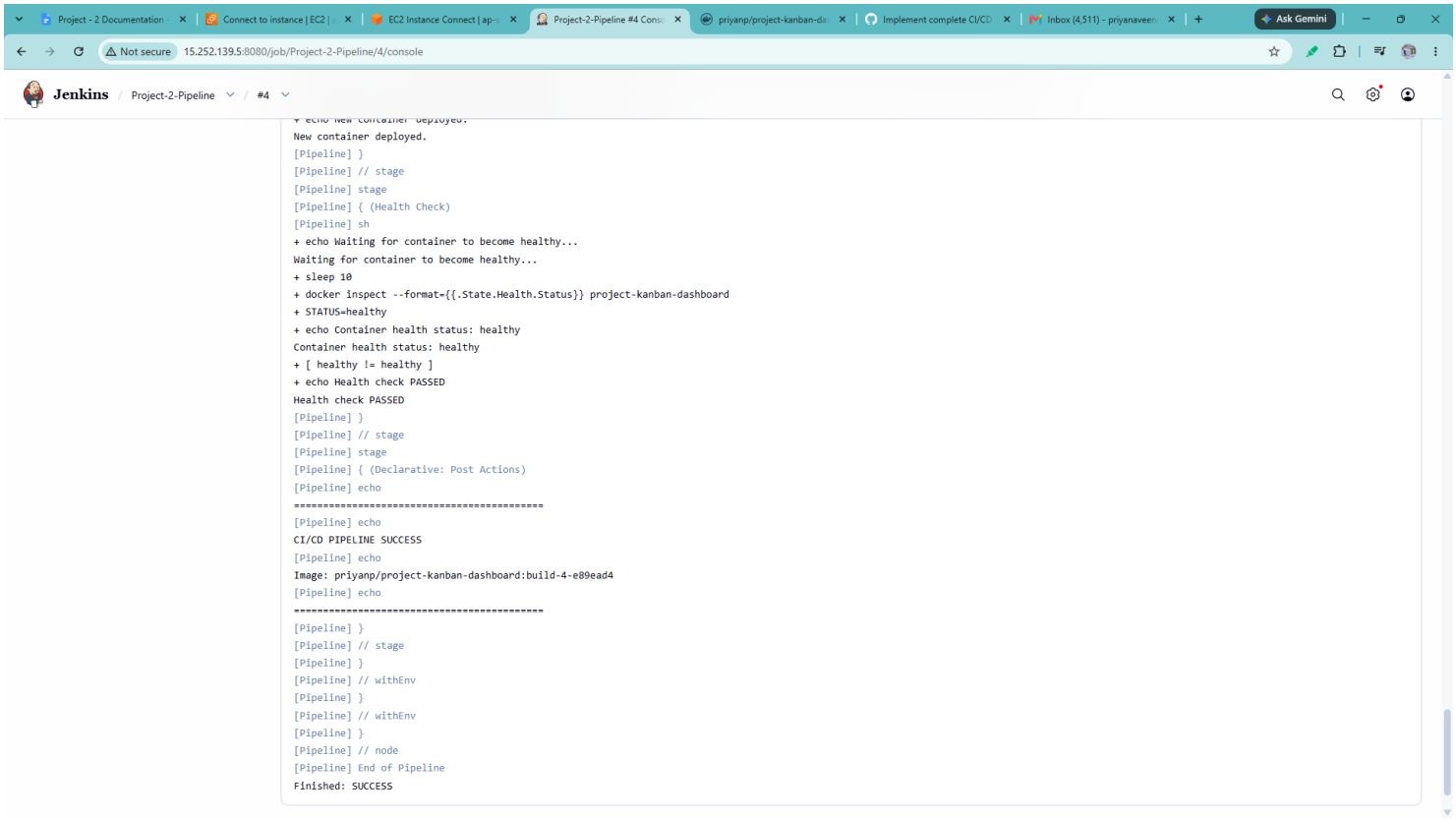
- Build #2 console showed CI/CD PIPELINE FAILED with the exact error that caused the failure
- Build #4 console showed Health check PASSED and Finished: SUCCESS
- sudo docker ps confirmed container status as **healthy**
- sudo docker logs confirmed the app is responding with 200 OK

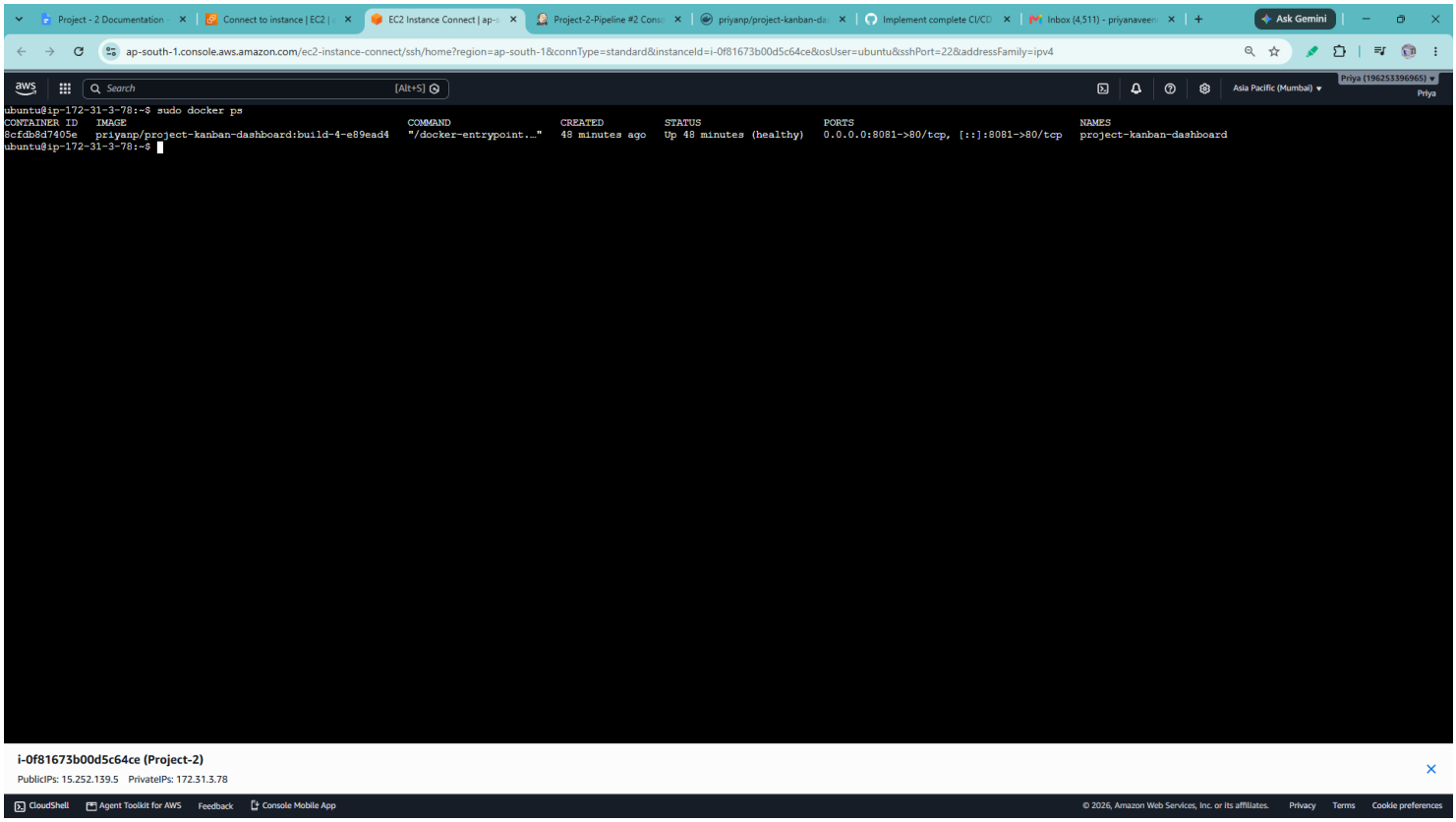
Result

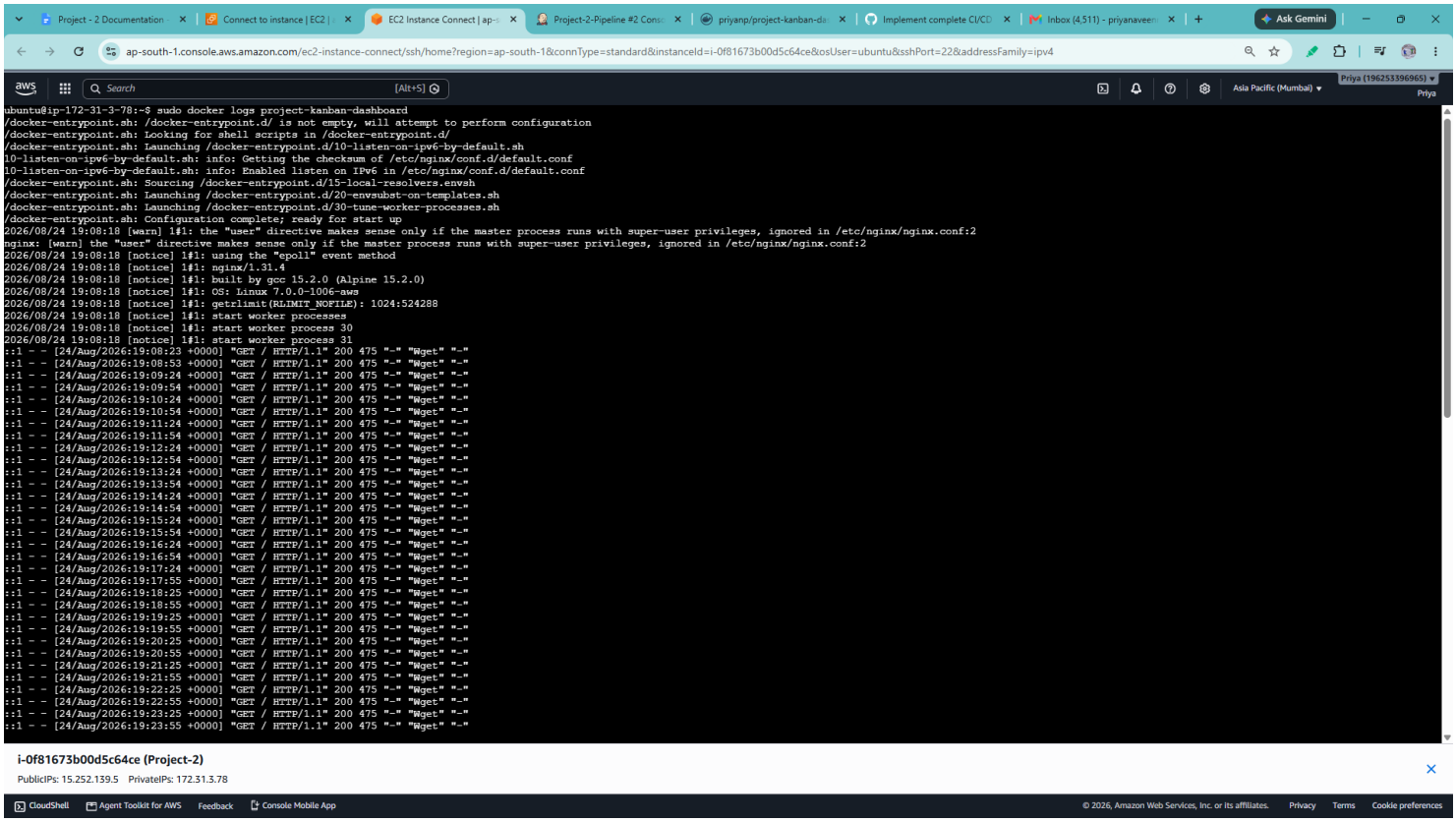
The build history shows both a failed and a successful run with enough detail in the logs to identify what went wrong and confirm when it was fixed.

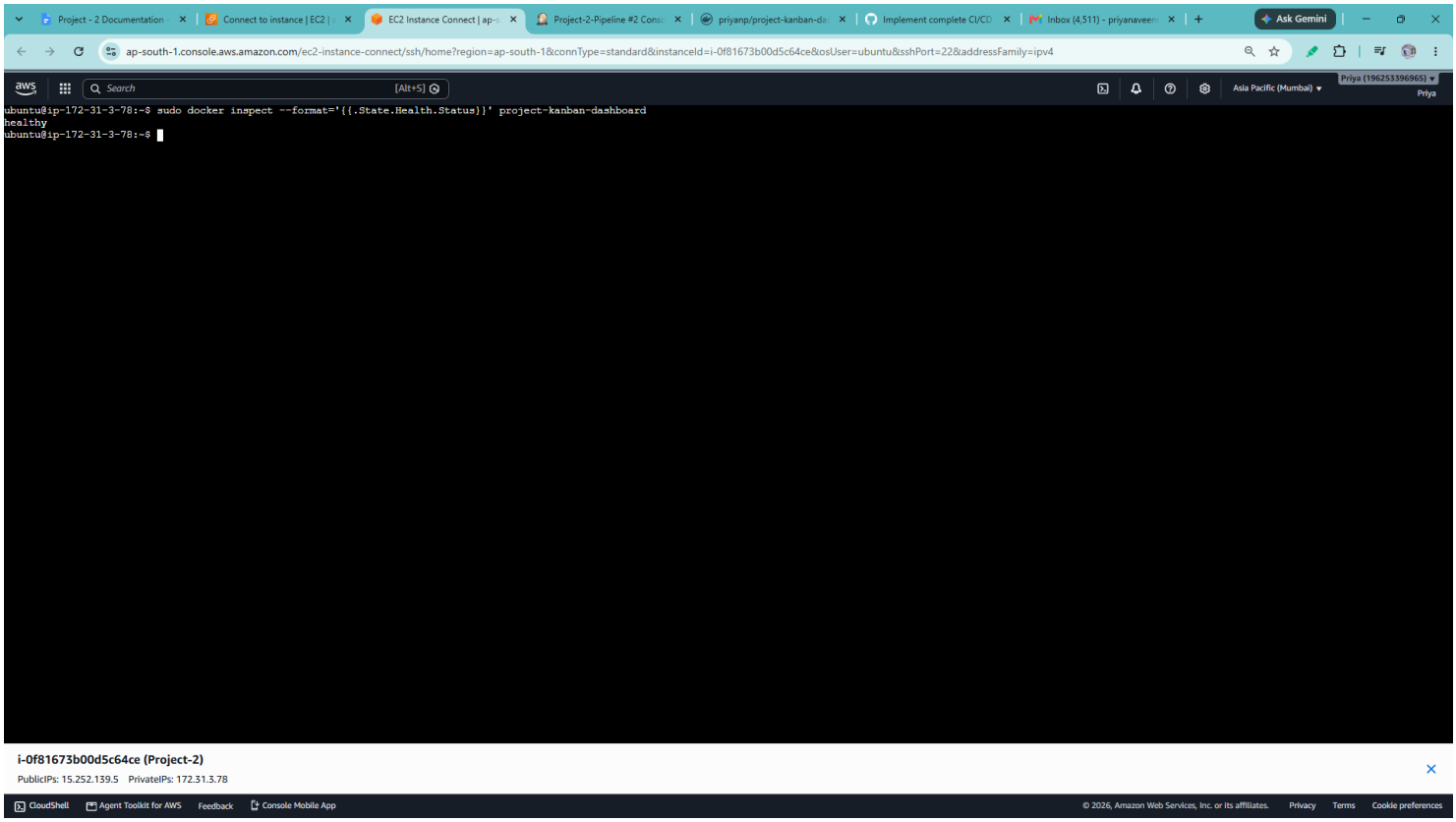

Not secure 15.252.139.5:8080/view/all/builds
Jenkins / All / Build History
New Item
Build History
Project Relationship
Check File Fingerprint
Build Queue
No builds in the queue.
Build Executor Status
(0 of 2 executors busy)
Build History of Jenkins
S
Build
Time Since
Status
Project-2-Pipeline #4
50 min
stable
Project-2-Pipeline #3
12 hr
back to normal
Project-2-Pipeline #2
12 hr
broken since this build
Project-2-Pipeline #1
2 days 9 hr
stable
Icon: S M L
REST API
Jenkins 2.568.2


Not secure 15.252.139.5:8080/job/Project-2-Pipeline/2/console
Jenkins / Project-2-Pipeline / #2
+ docker build -t priyanp/project-kanban-dashboard:build-2-98f7a77 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Registry Login + Push)
Stage "Registry Login + Push" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to EC2)
Stage "Deploy to EC2" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Health Check)
Stage "Health Check" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
==========
[Pipeline] echo
CI/CD PIPELINE FAILED
[Pipeline] echo
Starting rollback...
[Pipeline] echo
==========
[Pipeline] sh
+ [-s previous_image.txt]
+ echo No previous image available.
No previous image available.
+ echo Rollback could not be performed.
Rollback could not be performed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE


Project - 2 Documentation
Connect to instance | EC2 |
EC2 Instance Connect | ap-
Project-2-Pipeline #4 Cons
priyanp/project-kanban-da
Implement complete CI/CD
Inbox (4,511) - priyanaveen
Ask Gemini
Not secure
15.252.139.5:8080/job/Project-2-Pipeline/4/console
Jenkins
Project-2-Pipeline
#4
+ echo New container deployed.
New container deployed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Health Check)
[Pipeline] sh
+ echo Waiting for container to become healthy...
Waiting for container to become healthy...
+ sleep 10
+ docker inspect --format={{.State.Health.Status}} project-kanban-dashboard
+ STATUS=healthy
+ echo Container health status: healthy
Container health status: healthy
+ [healthy != healthy]
+ echo Health check PASSED
Health check PASSED
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
==========
[Pipeline] echo
CI/CD PIPELINE SUCCESS
[Pipeline] echo
Image: priyanp/project-kanban-dashboard:build-4-e89ead4
[Pipeline] echo
==========
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Project - 2 Documentation
Connect to instance | EC2 |
EC2 Instance Connect | ap-
Project-2-Pipeline #2 Cons
priyanp/project-kanban-da
Implement complete CI/CD
Inbox (4,511) - priyanaveen
Ask Gemini
ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&sshPort=22&addressFamily=ipv4
aws
Search
[Alt+S]
Asia Pacific (Mumbai)
Priya (196253396965)
Priya
ubuntu@ip-172-31-3-78:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
8cfdb8d7405e priyanp/project-kanban-dashboard:build-4-e89ead4 "/docker-entrypoint.…" 48 minutes ago Up 48 minutes (healthy) 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp project-kanban-dashboard
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


ubuntu@ip-172-31-3-78:~$ sudo docker logs project-kanban-dashboard
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/24 19:08:18 [warn] 1#1: the "user" directive makes sense only if the master process runs with super-user privileges, ignored in /etc/nginx/nginx.conf:2
nginx: [warn] the "user" directive makes sense only if the master process runs with super-user privileges, ignored in /etc/nginx/nginx.conf:2
2026/08/24 19:08:18 [notice] 1#1: using the "epoll" event method
2026/08/24 19:08:18 [notice] 1#1: nginx/1.31.4
2026/08/24 19:08:18 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/08/24 19:08:18 [notice] 1#1: OS: Linux 7.0.0-1006-aws
2026/08/24 19:08:18 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/08/24 19:08:18 [notice] 1#1: start worker processes
2026/08/24 19:08:18 [notice] 1#1: start worker process 30
2026/08/24 19:08:18 [notice] 1#1: start worker process 31
::1 - - [24/Aug/2026:19:08:23 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:08:53 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:09:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:09:54 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:10:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:10:54 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:11:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:11:54 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:12:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:12:54 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:13:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:13:54 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:14:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:14:54 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:15:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:15:54 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:16:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:16:54 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:17:24 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:17:55 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:18:25 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:18:55 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:19:25 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:19:55 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:20:25 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:20:55 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:21:25 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:21:55 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:22:25 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:22:55 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:23:25 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
::1 - - [24/Aug/2026:19:23:55 +0000] "GET / HTTP/1.1" 200 475 "-" "Wget" "-"
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78


ubuntu@ip-172-31-3-78:~$ sudo docker inspect --format='{{.State.Health.Status}}' project-kanban-dashboard
healthy
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78

Step 11 : Docker Resource Limits

Overview

To prevent the container from consuming all available host resources, memory and CPU limits were applied to the running container based on the EC2 instance size.

Instance Resources

The t3.small EC2 instance has 1.9 GiB total memory and 2 vCPUs (confirmed via free -h and nproc).

Resource Limits Applied

The following limits were set on the running container using docker update:

- Memory: 512m
- Memory Swap: 512m (swap disabled)
- CPUs: 1

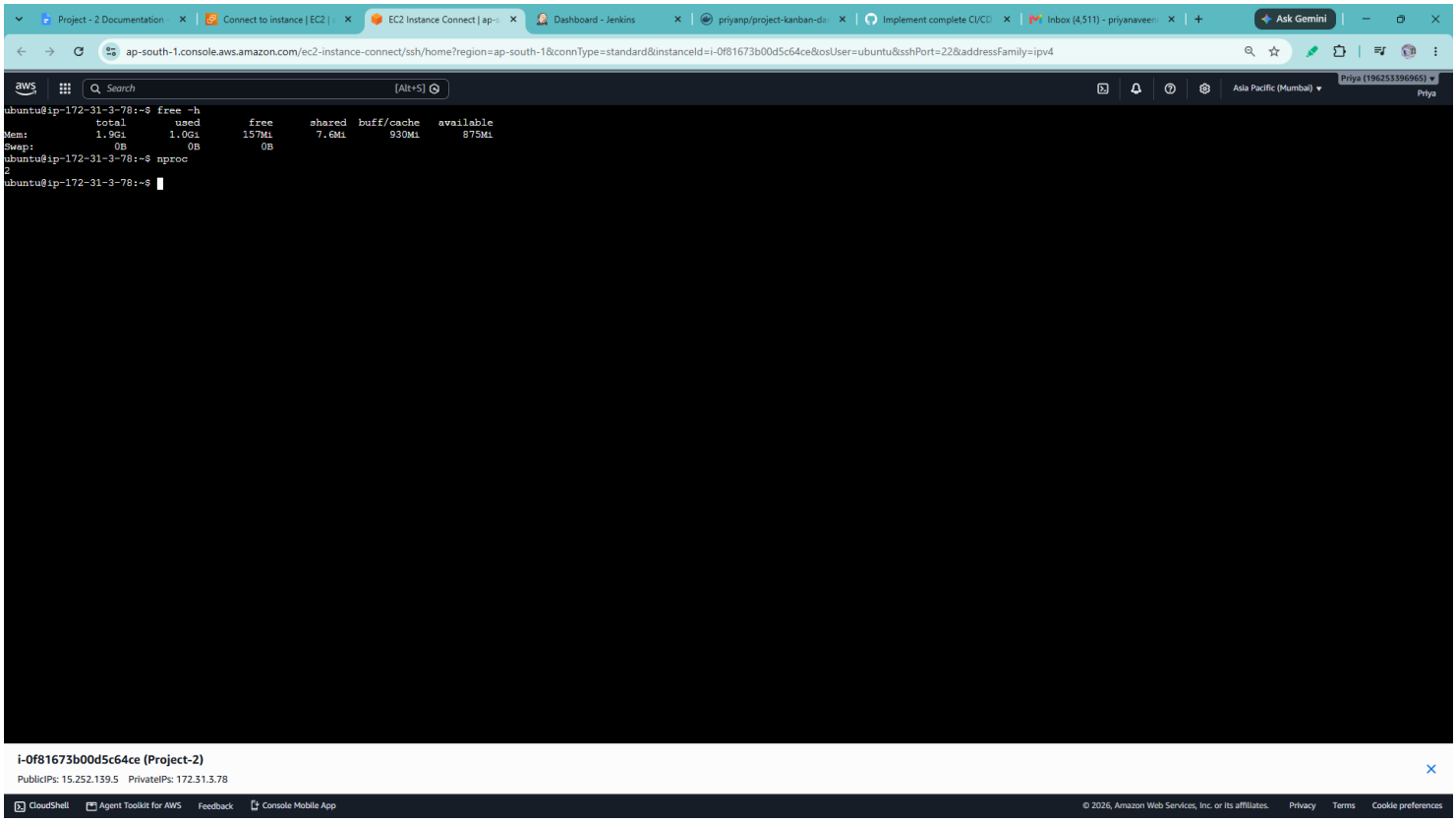
Validation

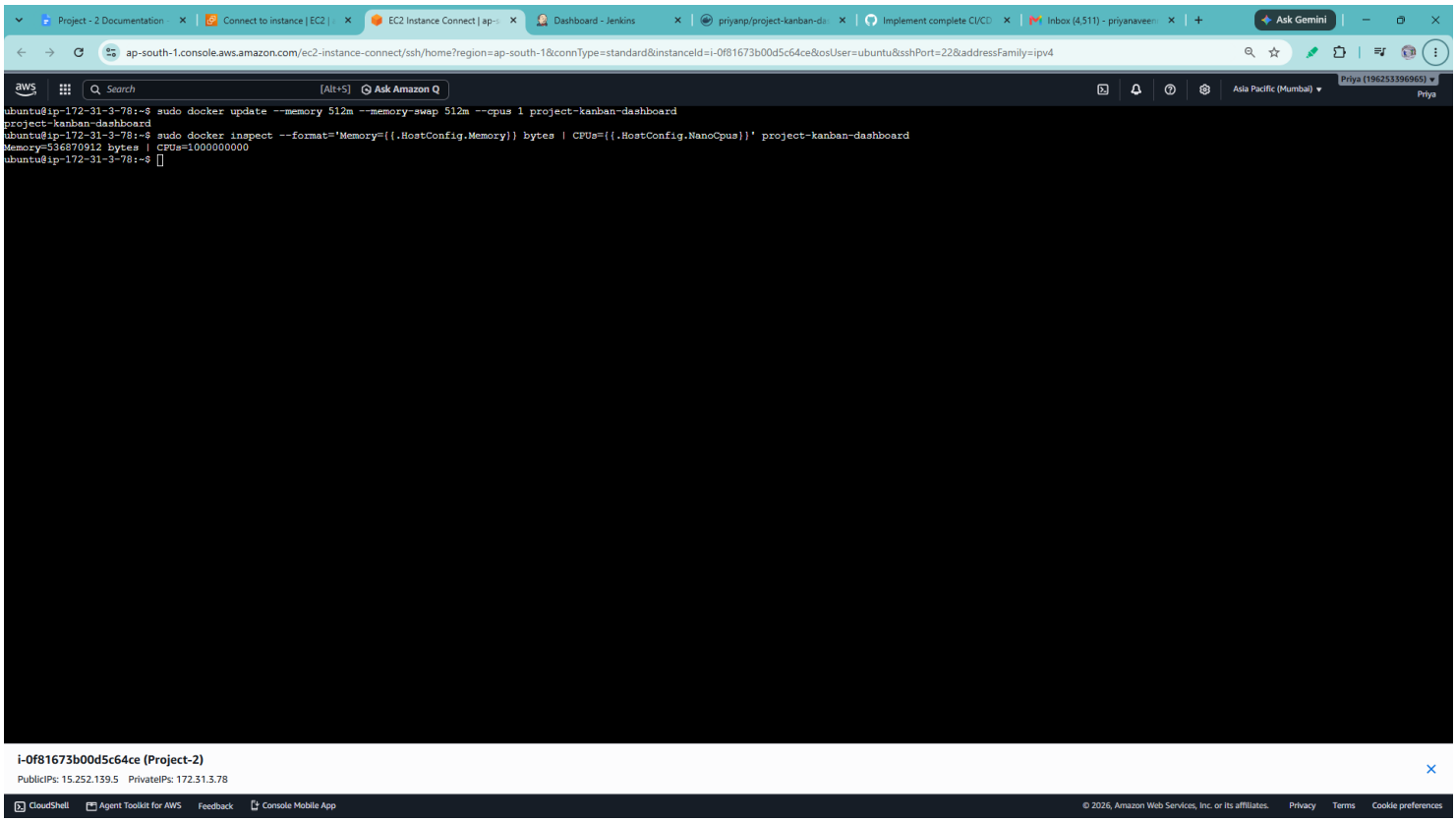
Running sudo docker inspect confirmed the applied limits:

- Memory: 536870912 bytes (512 MB)
- CPUs: 1000000000 NanoCPUs (1 CPU)

Result

Resource limits were applied successfully. The container is capped at 512 MB memory and 1 CPU, ensuring Jenkins and the application can share the t3.small instance without the container consuming all available host resources.


Project - 2 Documentation
Connect to instance | EC2 |
EC2 Instance Connect | ap-
Dashboard - Jenkins
priyanp/project-kanban-da
Implement complete CI/CD
Inbox (4,511) - priyanaveen
Ask Gemini
ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&sshPort=22&addressFamily=ipv4
aws
Search
[Alt+S]
Asia Pacific (Mumbai)
Priya (196253396965)
Priya
ubuntu@ip-172-31-3-78:~$ free -h
total used free shared buff/cache available
Mem: 1.9Gi 1.0Gi 157Mi 7.6Mi 930Mi 875Mi
Swap: 0B 0B 0B
ubuntu@ip-172-31-3-78:~$ nproc
2
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Project - 2 Documentation
Connect to instance | EC2 |
EC2 Instance Connect | ap-
Dashboard - Jenkins
priyanp/project-kanban-da
Implement complete CI/CD
Inbox (4,511) - priyanaveen
Ask Gemini
ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&sshPort=22&addressFamily=ipv4
aws
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Priya (196253396965)
Priya
ubuntu@ip-172-31-3-78:~$ sudo docker update --memory 512m --memory-swap 512m --cpus 1 project-kanban-dashboard
project-kanban-dashboard
ubuntu@ip-172-31-3-78:~$ sudo docker inspect --format='Memory={{.HostConfig.Memory}} bytes | CPUs={{.HostConfig.NanoCpus}}' project-kanban-dashboard
Memory=536870912 bytes | CPUs=1000000000
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

STEP 12 : Zero / Minimal Downtime Deployment (Bonus)

Overview

A blue-green deployment strategy was implemented to ensure the application stays reachable during a new deployment. The new container is started and health-checked before the old one is stopped, so there is no gap in availability.

Strategy

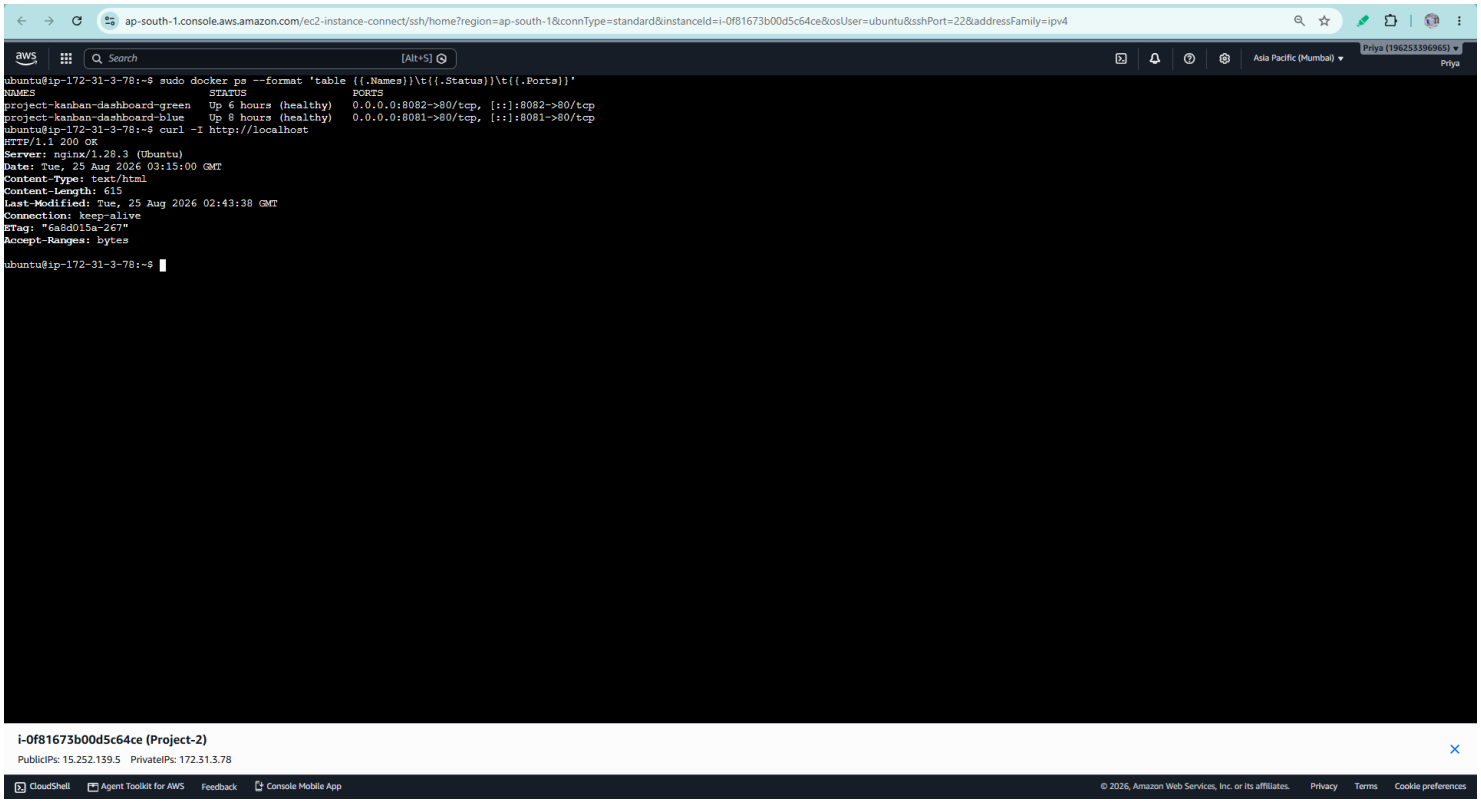
The existing container running on port 8081 was treated as the blue container. A new container named project-kanban-dashboard-green was started on port 8082 alongside it. Nginx on the EC2 host was configured with an upstream block pointing to the green container at 127.0.0.1:8082. Once the green container was confirmed healthy, traffic was switched to it via Nginx and the blue container was stopped and removed.

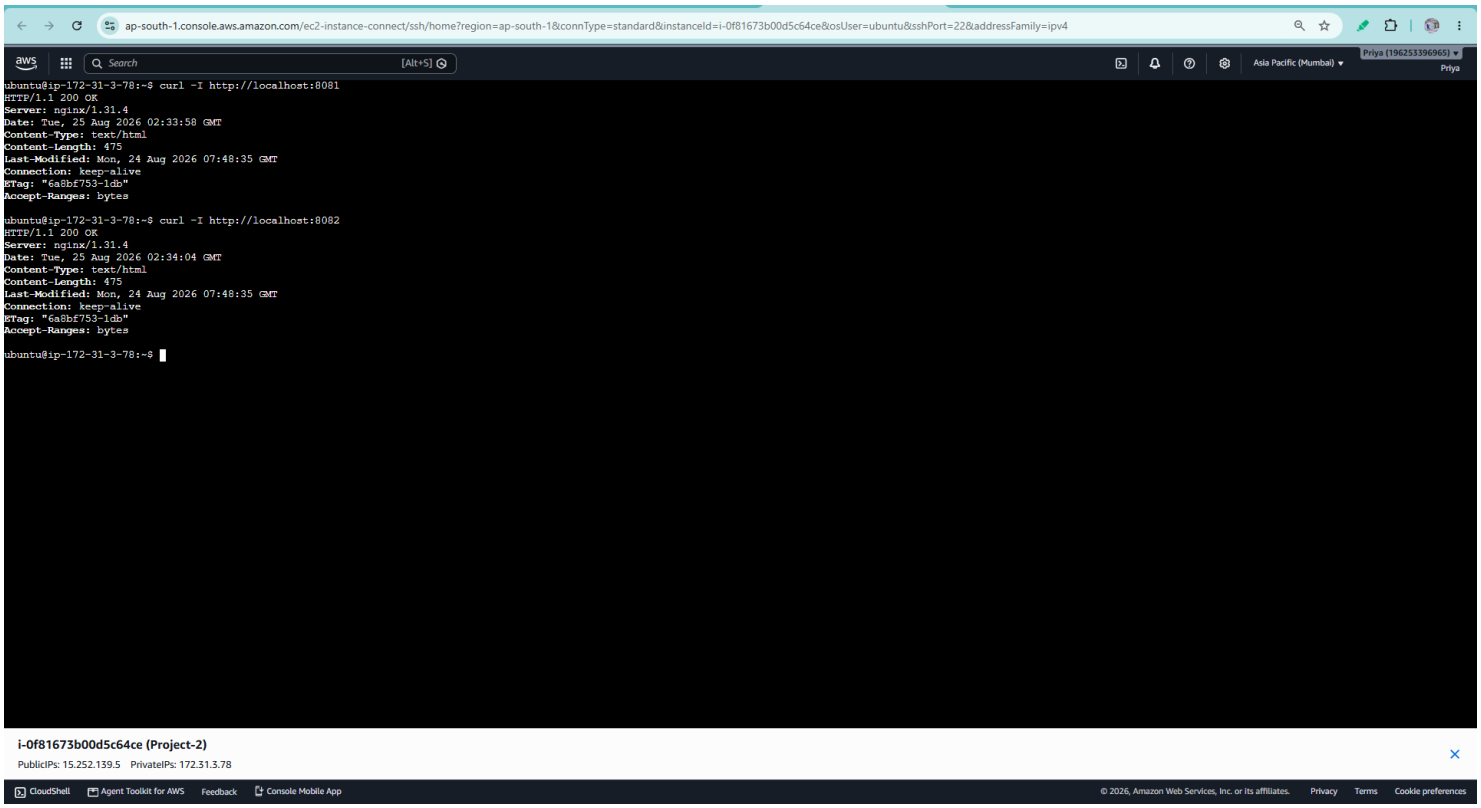
Validation

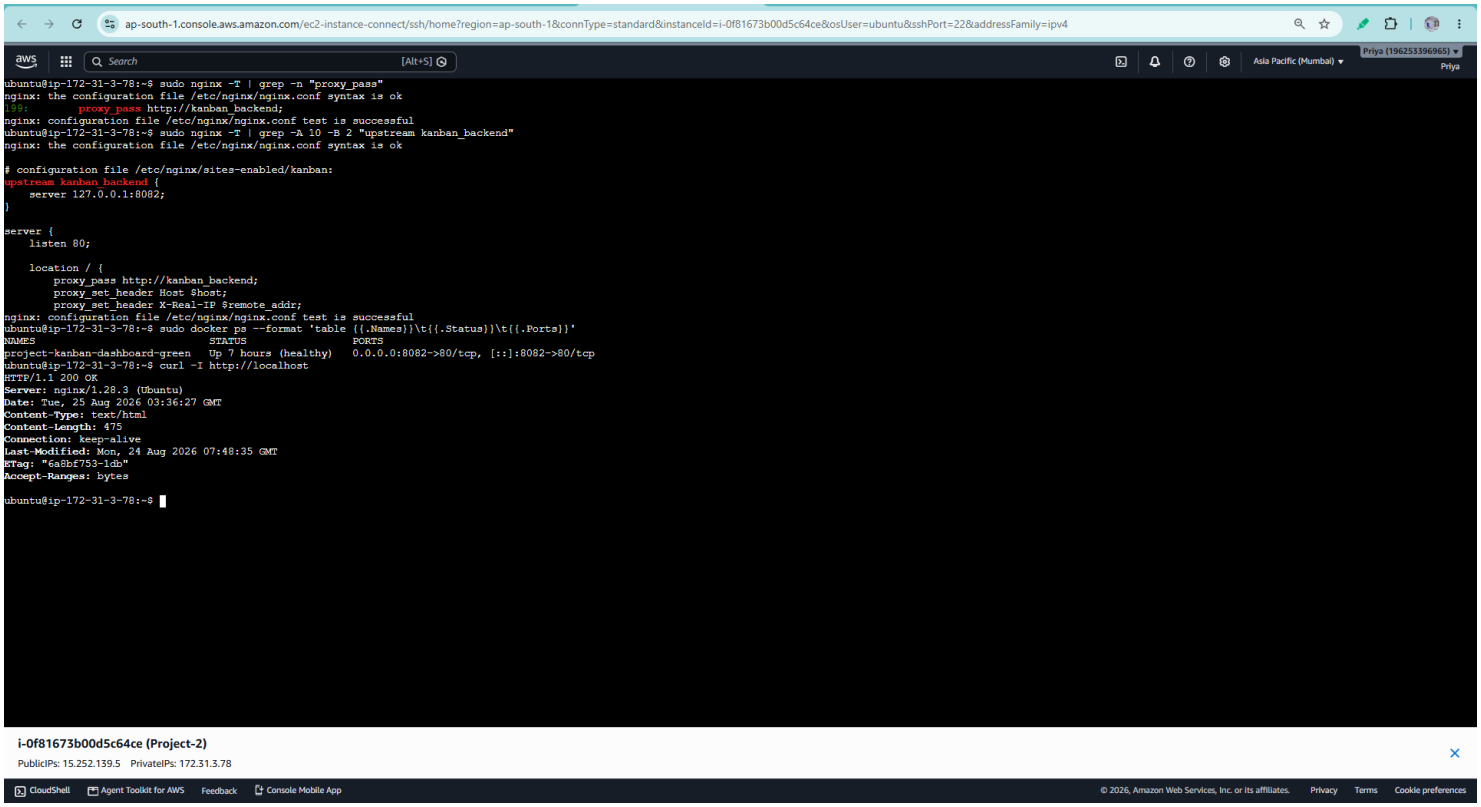
- Both containers were running at the same time — blue on port 8081 and green on port 8082, both showing status healthy
- curl -I http://localhost:8081 and curl -I http://localhost:8082 both returned HTTP/1.1 200 OK
- After switching Nginx to green, curl -I http://localhost returned HTTP/1.1 200 OK
- Blue container was stopped and removed — only green remained running as healthy
- The Kanban Task Manager loaded successfully in the browser at 15.252.139.5

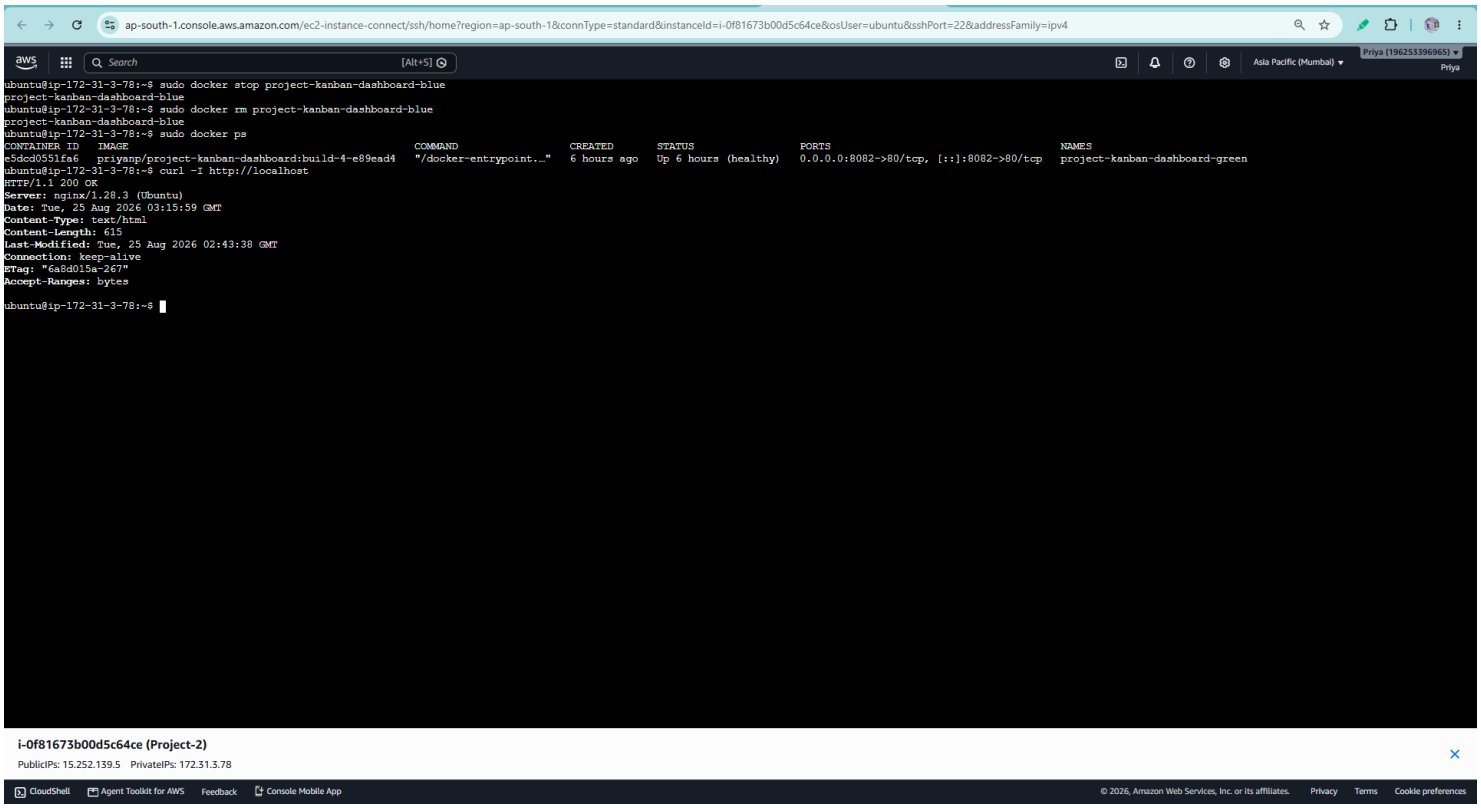
Result

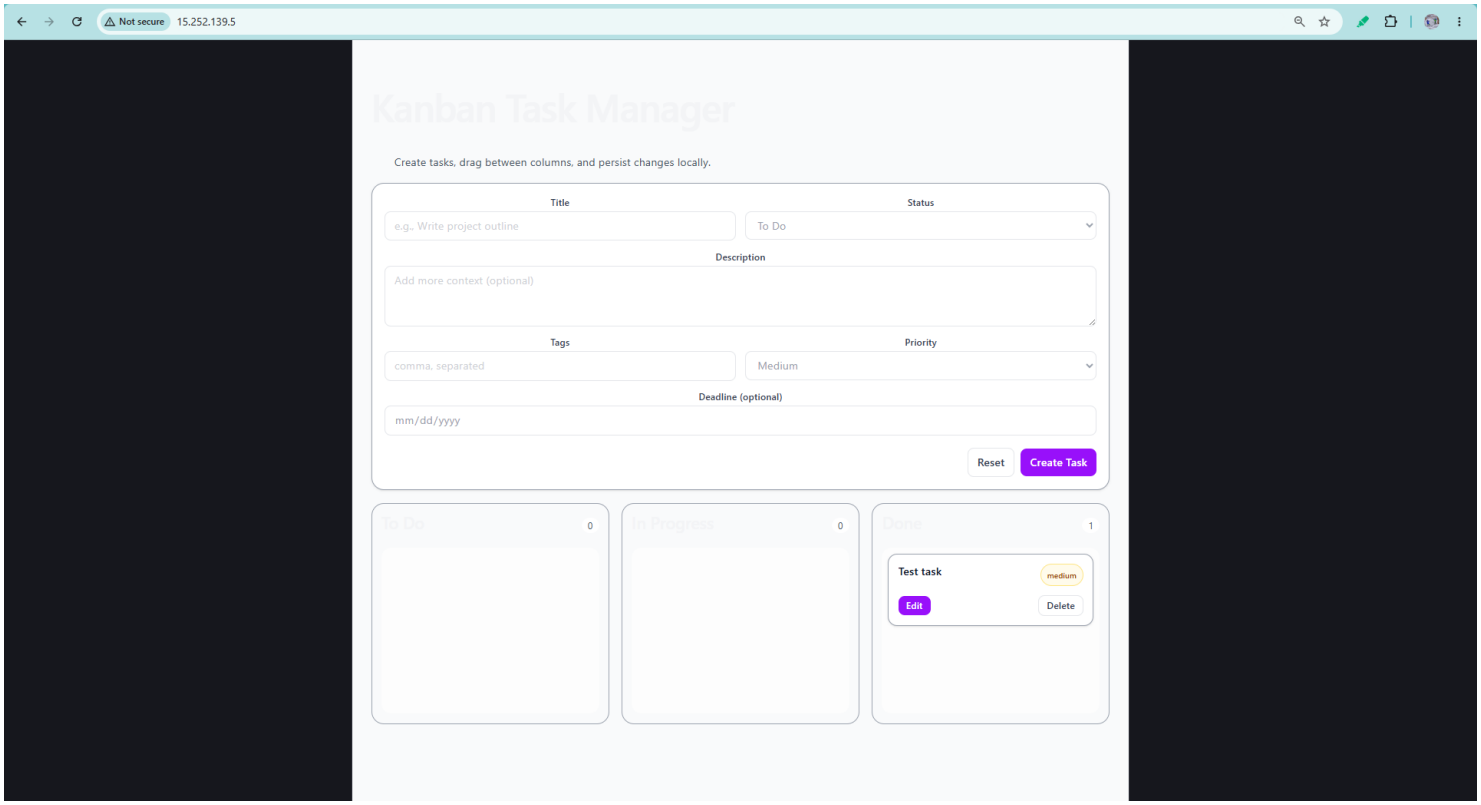
The blue-green deployment was completed successfully. The application stayed reachable throughout — the old container was only stopped after the new one was confirmed healthy and serving traffic.


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&sshPort=22&addressFamily=ipv4
Search [Alt+S]
Asia Pacific (Mumbai)
Priya (196253396965)
Priya
ubuntu@ip-172-31-3-78:~$ sudo docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
NAMES STATUS PORTS
project-kanban-dashboard-green Up 6 hours (healthy) 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
project-kanban-dashboard-blue Up 8 hours (healthy) 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
ubuntu@ip-172-31-3-78:~$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 25 Aug 2026 03:15:00 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Tue, 25 Aug 2026 02:43:38 GMT
Connection: keep-alive
ETag: "6a8d015a-267"
Accept-Ranges: bytes
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell Agent Toolkit for AWS Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&sshPort=22&addressFamily=ipv4
Search [Alt+S]
Asia Pacific (Mumbai)
Priya (196253396965)
Priya
ubuntu@ip-172-31-3-78:~$ curl -I http://localhost:8081
HTTP/1.1 200 OK
Server: nginx/1.31.4
Date: Tue, 25 Aug 2026 02:33:58 GMT
Content-Type: text/html
Content-Length: 475
Last-Modified: Mon, 24 Aug 2026 07:48:35 GMT
Connection: keep-alive
ETag: "6a8bf753-1db"
Accept-Ranges: bytes
ubuntu@ip-172-31-3-78:~$ curl -I http://localhost:8082
HTTP/1.1 200 OK
Server: nginx/1.31.4
Date: Tue, 25 Aug 2026 02:34:04 GMT
Content-Type: text/html
Content-Length: 475
Last-Modified: Mon, 24 Aug 2026 07:48:35 GMT
Connection: keep-alive
ETag: "6a8bf753-1db"
Accept-Ranges: bytes
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell Agent Toolkit for AWS Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&sshPort=22&addressFamily=ipv4
Search
[Alt+S]
Asia Pacific (Mumbai)
Priya (196253396965)
Priya
ubuntu@ip-172-31-3-78:~$ sudo nginx -T | grep -n "proxy_pass"
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
199: proxy_pass http://kanban_backend;
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-3-78:~$ sudo nginx -T | grep -A 10 -B 2 "upstream kanban_backend"
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
configuration file /etc/nginx/sites-enabled/kanban:
upstream kanban_backend {
server 127.0.0.1:8082;
}
server {
listen 80;
location / {
proxy_pass http://kanban_backend;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-3-78:~$ sudo docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
NAMES STATUS PORTS
project-kanban-dashboard-green Up 7 hours (healthy) 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
ubuntu@ip-172-31-3-78:~$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 25 Aug 2026 03:36:27 GMT
Content-Type: text/html
Content-Length: 475
Connection: keep-alive
Last-Modified: Mon, 24 Aug 2026 07:48:35 GMT
ETag: "6a8bf753-1db"
Accept-Ranges: bytes
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-0f81673b00d5c64ce&osUser=ubuntu&sshPort=22&addressFamily=ipv4
Search
[Alt+S]
Asia Pacific (Mumbai)
Priya (196253396965)
Priya
ubuntu@ip-172-31-3-78:~$ sudo docker stop project-kanban-dashboard-blue
project-kanban-dashboard-blue
ubuntu@ip-172-31-3-78:~$ sudo docker rm project-kanban-dashboard-blue
project-kanban-dashboard-blue
ubuntu@ip-172-31-3-78:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
e5dcd0551fa6 priyanp/project-kanban-dashboard:build-4-e89ead4 "/docker-entrypoint.…" 6 hours ago Up 6 hours (healthy) 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp project-kanban-dashboard-green
ubuntu@ip-172-31-3-78:~$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 25 Aug 2026 03:15:59 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Tue, 25 Aug 2026 02:43:38 GMT
Connection: keep-alive
ETag: "6a8d015a-267"
Accept-Ranges: bytes
ubuntu@ip-172-31-3-78:~$
i-0f81673b00d5c64ce (Project-2)
PublicIPs: 15.252.139.5 PrivateIPs: 172.31.3.78
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Not secure
15.252.139.5
Kanban Task Manager
Create tasks, drag between columns, and persist changes locally.
Title
e.g., Write project outline
Status
To Do
Description
Add more context (optional)
Tags
comma, separated
Priority
Medium
Deadline (optional)
mm/dd/yyyy
Reset
Create Task
To Do
0
In Progress
0
Done
1
Test task
medium
Edit
Delete

Deliverables

● GitHub Repository :
https://github.com/priya-dharshini-sankar/Kanban-Dashboard-Project.git

● Docker Hub/ECR Repository :
https://hub.docker.com/repository/docker/priyanp/project-kanban-dashboard/general

● Application URL / EC2 Public IP :
http://15.252.139.5/